Wesleyan University The Honors College

# Running Shor's Algorithm on a complete, gate-by-gate implementation of a virtual, universal quantum computer

by

Y. S. Nam

Research Advisor: Dr. Reinhold Blümel

A Dissertation submitted to the
faculty of Wesleyan University
in partial fulfillment of the requirements for the
Degree of Bachelor of Arts
with Departmental Honors in Physics

Middletown, Connecticut May, 2011

# Abstract

This Thesis presents a complete, microscopic, gate-by-gate simulation of a universal quantum computer, implemented on a classical PC. The resulting virtual quantum computer is then put to work running Shor's algorithm, a quantum program that is capable of cracking banking, military, and government codes. The virtual quantum computer is an ideal laboratory for testing quantum computing under realistic conditions such as the influence of noise on the quantum computing process.

We obtain the surprising result that quantum computing is unexpectedly robust against the influence of noise. We propose an analytical scaling relation, verified by actual quantum runs of Shor's algorithm, that have the potential to result in considerable hardware savings. In addition, an error in the literature is pointed out.

# Acknowledgments

I would like to thank my research advisor, Professor Reinhold Blümel, in guiding me and always being there whenever I was in a conundrum. I couldn't have had the insight, knowledge, and understanding of physics necessary for this paper without your instruction. I am lucky to have such an able mentor.

I would like to thank the Wesleyan Physics Department for all its support. I learned a great deal in interacting with both the faculty and the students.

A special thanks to my friend and colleague Stefan Kramel for the digital diagrams in this thesis.

# Contents


**1 Introduction** **1**

1.1 Philosophy . . . . . 1

1.2 History . . . . . 2

1.3 Quantum Nature . . . . . 3

1.4 Quantum Information . . . . . 5

1.5 Overview . . . . . 6

**2 Brief History of Quantum Computing** **8**

**3 Shor's Algorithm** **13**

3.1 RSA Encryption . . . . . 13

3.2 Miller's Algorithm . . . . . 15

3.3 Quantum Factoring . . . . . 17

**4 Evaluation of the Analytical Result of Shor's Algorithm** **22**

**5 Gate-by-Gate Implementation of Shor's Algorithm: Ideal $f$-mapping** **26**

5.1 Basic Single-Qubit Quantum Gates . . . . . 27

5.1.1 Single-Qubit Rotation Gate . . . . . 28

5.1.2 Hadamard Gate . . . . . 28

5.2 Construction of Gates . . . . . 29

5.2.1 Initialization of Qubits . . . . . 29

5.2.2 Quantum Fast Fourier Transform . . . . . 30
5.3 Structure of Simulation Code . . . . . 31
5.3.1 Initialization and Equivalence Classes . . . . . 31
5.3.2 Limit of Quantum Fast Fourier Transform . . . . . 32
5.4 Probability Spectrum and Scaling Property . . . . . 33
5.4.1 Ideal Spectrum . . . . . 33
5.4.2 Effect of the Limit imposed on the Quantum Fourier Transform . 35
5.4.3 Scaling . . . . . 40

**6 Periphery of Shor's Algorithm: Ideal $f$-mapping 43**
6.1 Introduction of Noise . . . . . 43
6.2 Probability Spectrum . . . . . 44
6.2.1 Influence of Noise . . . . . 44
6.2.2 Effect of the Combination of a Quantum Fourier Transform Limit and the Noise . . . . . 46

**7 Gate-by-Gate construction of $f$-mapping 49**
7.1 Methodology . . . . . 50
7.2 Building Blocks . . . . . 52
7.2.1 NOT gate . . . . . 52
7.2.2 CNOT gate . . . . . 53
7.2.3 CCNOT Gate . . . . . 55
7.2.4 Block Diagram of the SUM Gate . . . . . 56
7.2.5 Block Diagram of the CARRY Gate . . . . . 57
7.2.6 Block Diagram of the ADDER Gate . . . . . 58
7.3 Construction of the $f$-mapping operation . . . . . 59
7.3.1 Block Diagram of the ADD-MOD Gate . . . . . 59
7.3.2 Block Diagram of the CTRL-MULTI-MOD Gate . . . . . 60
7.3.3 Block Diagram of the EXP-MOD Gate . . . . . 61

**8 Complete Implementation of Shor's Algorithm: Non-unitary Approach 64**

8.1 Unitary Gates . . . 65

8.1.1 Single-Qubit Gates . . . 66

8.1.2 CNOT Gate . . . 66

8.1.3 CCNOT Gate . . . 67

8.1.4 SUM Gate . . . 67

8.1.5 CARRY Gate . . . 67

8.2 Non-Unitary Gates . . . 67

8.2.1 ADDER Gate . . . 68

8.2.2 ADD-MOD Gate . . . 70

8.2.3 CTRL-MULTI-MOD Gate . . . 73

8.2.4 EXP-MOD Gate . . . 76

8.3 Non-Unitary Shor's algorithm without Noise . . . 76

8.3.1 Non-Unitary $f$-mapping . . . 76

8.3.2 Unitary Peripheral Shor's algorithm and Non-Unitary $f$-mapping without Noise . . . 77

8.4 Scaling of the number of basic quantum gates . . . 78

**9 Complete Implementation of Shor's Algorithm with Noise 80**

9.1 Noise Introduction . . . 80

9.2 Probability Spectrum . . . 81

9.2.1 Equivalence Class . . . 81

9.2.2 Final Measurement . . . 83

**10 Dicussion 85**

**11 Conclusion 87**

**A Unitary N xN Transformations 89**

**B A New Method for Matrix Diagonalization 97**

# List of Figures


3.1 Effort of factoring a semi-prime number. . . . . 15
3.2 Flow chart of Miller's algorithm. . . . . 16
3.3 A schematic diagram of Shor's algorithm. . . . . 19

4.1 Probability Spectra of integer states via the evaluation of the analytical formula of Shor's algorithm. . . . . 23

5.1 Circuit representation of the single-qubit rotation gate. . . . . 28
5.2 Circuit representation of the Hadamard gate. . . . . 28
5.3 Logic circuit of the quantum Fourier transform. . . . . 31
5.4 Probability spectrum of Shor's algorithm obtained via a gate-by-gate implementation of the periphery of Shor's algorithm. . . . . 34
5.5 Peak probability of the periphery of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. . . . . 36
5.6 Log-scaled peak probability loss of the periphery of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. . . 36
5.7 Function fit of peak probability of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. . . . . 37
5.8 Geometry of the interference of the peak phases in the quantum Fourier transform. . . . . 38

5.9 Analytical expression for the peak probability of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. . . . 39
5.10 Extrapolated behavior of peak probability vs. number of qubits with various limits imposed on the quantum Fourier transform. . . . . 40

6.1 Probability spectra of Shor's algorithm via the gate-by-gate implementation of the peripheral Shor's algorithm with noise. . . . . 45
6.2 Peak probability of peripheral Shor's algorithm vs. noise level. . . . . 46
6.3 Peak probability of peripheral Shor's algorithm with fixed noise vs. size of the system (number of qubits). . . . . 47
6.4 Normalized peak probability of the peripheral Shor's algorithm with noise vs. normalized maximal qubit spacing in the quantum Fourier transform. 48

7.1 Circuit representation of the NOT gate. . . . . 53
7.2 Circuit representation of the CNOT gate. . . . . 55
7.3 Logic circuit of the CCNOT gate. . . . . 56
7.4 Logic circuit of the SUM gate. . . . . 57
7.5 Logic circuit of the CARRY gate. . . . . 57
7.6 Logic circuit of the ADDER gate. . . . . 58
7.7 Logic circuit of the ADD-MOD gate. . . . . 60
7.8 Logic circuit of the CTRL-MULTI-MOD gate. . . . . 61
7.9 Logic circuit of the EXP-MOD gate. . . . . 62
7.10 Logic circuit of the modified EXP-MOD gate. . . . . 63

8.1 Stage-by-stage decomposition of the logic circuit of the ADDER gate. . 69
8.2 Stage-by-stage decomposition of the logic circuit of the ADD-MOD gate. 71
8.3 Stage-by-stage decomposition of the logic circuit of the CTRL-MULTI-MOD gate. . . . . 74
8.4 Probability spectrum of an equivalence class obtained from a complete, non-unitary implementation of Shor's algorithm. . . . . 77

8.5 Pprobability spectrum of Shor's algorithm via the complete, non-unitary implementation. . . . . 78

9.1 Probability spectrum of equivalence class with noise via the complete, non-unitary implementation. . . . . 82
9.2 Peak probability of non-unitary Shor's algorithm vs. noise level. . . . . 83
9.3 Probability spectrum of Shor's algorithm with noise via the complete, non-unitary implementation. . . . . 84

A.1 Two different ways to understand an Euler rotation. . . . . 90
A.2 Visualization of the decomposition process. . . . . 91
A.3 Infinitesimal unitary transformations and the number of real parameters. 96

B.1 Depiction of the steepest descent method in a quadratic function. . . . . 99
B.2 Comparison of the execution time for diagonalizaing an $N \times N$ matrix between the conventional method and the new method. . . . . 101
B.3 Convergence behavior of the steepest descent method. . . . . 102
B.4 A possible geometry of a function that may cause an increase in the error in step number. . . . . 102
B.5 Trapping by local minima in the steepest descent method. . . . . 103
B.6 Trapping by maxima in the steepest descent method. . . . . 104
B.7 $N \times N$ matrix representing an approximated system. . . . . 105

C.1 A schematic diagram of a realizable quantum network. . . . . 109

# Chapter 1

# Introduction

## 1.1 Philosophy

"I am walking with two men; each of them will serve as my teacher. I will pick out the good points of the one and imitate them, and the bad points of the other and correct them in myself." (Confucius) Meant as good advice for the personal improvement of people, it is eminently applicable to physics.

It is not hard to find an example: Bohr's model of the atom. Bohr was working with two tools at hand, empirical data and classical mechanics. As the data was reliable (good), he tried to build on (imitate) the irrefutable nature of the data. As the classical theory did not fit the data (bad) he tried to improve (correct) the unreliable nature of the classical theory. The result was quantization! In short, we have the physicist's version of the quote: "I work with two different parts of physics, each of them will serve as my tools. I will identify the strengths of the one and make great use of it; I will identify the weak parts of the other and try to improve it."

The development of a quantum computer is an excellent demonstration of how powerful this philosophy can be. As a physicist, one's ultimate goal is to describe nature. If

quantum mechanics is the most fundamental theory of physics at our disposal and, therefore, nature indeed behaves as quantum mechanics tells us, then nature itself is the most natural and the most excellent candidate to simulate physics. Combine this with the rise of the ever more powerful, yet not powerful enough, classical computer, and we have arrived at a novel paradigm: the quantum computer.

## 1.2 History

One may wonder who came up with the idea of a quantum computer and how it came about. It is Richard Feynman, an inspiring physicist, who suggested the notion in the form of a quantum simulator. The way he introduces this new kind of idea is evident from his article in 1982 [1]. As to the question of what type of universal computer to use to simulate physics, specifically the computer that simulates everything, exactly, that happens in a finite volume of space and time, Feynman asserts,

- "The discovery of computers and the thinking about computers has turned out to be extremely useful in many branches of human reasoning."
- "I am not happy with all the analyses that go with just the classical theory, because nature isn't classical."
- "If you want to make a simulation of nature, you'd better make it quantum mechanical."
- "The real use of it (computer) would be with quantum mechanics."

We see that, implicitly, Feynman gave birth to the quantum computer, applying the philosophical idea introduced in the previous section. However, the idea of a quantum computer was put on a firm foundation only after David Deutsch defined it in analogy to the universal, classical Turing machine [2].

In 1985, Deutsch proposed the physical and quantum mechanical version of the Church-Turing principle, which once and for all defined the universality of a quantum computer. The original Church-Turing principle reads as follows: Every 'function which naturally can be regarded as computable' can be computed by a universal Turing machine [2]. As it is clear in the statement, the universality plays a key role in computing in that it allows us to use any architecture of a universal computer to perform a given computational task. Deutsch's quantum Turing principle is, therefore, worth mentioning: 'Every finitely realizable physical system can be perfectly simulated by a universal model computing machine operating by finite means' [2].

## 1.3 Quantum Nature

A classical Turing machine has a finite set of input states. Hence, an inherently quantum mechanical system, which has infinite dimension, cannot be imitated, and not even efficiently simulated. On the other hand, a quantum computer has an infinite-dimensional state space and only a finite-dimensional unitary evolution is needed to simulate nature! Of course, a classical Turing machine is equivalent to a universal quantum computer in terms of computability itself. Because a quantum computer can make use of many input states at the same time (parallelism) and execute a program (via unitary evolution) simultaneously on different states, a classical Turing machine can be seen as a special type of quantum computer in the sense that a classical computer can only run a program on one state at a time. In other words, a classical Turing machine is a universal quantum computer that has no interference, no non-separability, or no indeterminism of computational states at the end of each elementary computational step.

Bell's states [2–4] are excellent examples that demonstrate the superiority of a quantum computer over its classical counterpart. Such states bear the two qualitatively new features of quantum mechanics: entanglement and superposition. Bell's inequality [2–4] proves that a classical computer that requires prior classical knowledge, or a classical

probability machine, cannot imitate nature at all, while a quantum computer has the ability to simulate nature exactly. Therefore, a quantum computer is a fundamentally superior simulator to any classical computer.

In addition, entanglement and superposition, exhibited, e.g., by Bell's states, enables massively parallel processing that outperforms classical computers by an exponential margin. For instance, Peter Shor, in 1994 [5], presented an immensely powerful algorithm that factors a semi-prime integer [3, 4, 6] into two prime factors, using the intrinsic massive parallelism present in a quantum computer that arises from the superposition and entanglement of quantum physics; details will be discussed in Chapter 3. Compared to a classical computer's most efficient integer factorization algorithm known to us, Shor's algorithm is exponentially faster [5].

As we shall see in the following chapters, Shor's algorithm is a particularly interesting, yet dangerous algorithm, due to the fact that the encryption method used in a secure networking, e.g. RSA encryption, is based on the practical impossibility of cracking a large semi-prime integer into its two factors. The ramifications of the downfall of the RSA encryption method range from the malfunction of online banking systems to a severe threat to national security.

According to the principles of the philosophy introduced in section 1.1, the following idea can be thought of. If quantum technology is powerful, that it is indeed so mighty that the current RSA encryption standard can be easily broken, then we should use yet another quantum technological scheme to repair the broken security! The potential danger of the demolition of a securely encrypted network can now be remedied by using another quantum mechanical method: quantum encryption. Quantum encrypted messages may then be sent using yet another new type of technology: quantum networks (see appendix C).

## 1.4 Quantum Information

It becomes now necessary to quantify information in the quantum mechanical sense so that quantum information can be dealt with quantitatively. The most commonly used unit of information in quantum information is the qubit, in analogy to the classical bit. A qubit is fundamentally different from a classical bit [3, 4, 6]:

- A qubit can be in a superposition state:

$$|\text{state}\rangle = \cos(\phi)|0\rangle + \sin(\phi)|1\rangle$$

  .

- Qubits can be entangled with one another:

$$|\text{state}\rangle = \frac{1}{\sqrt{2}}\left[|0\rangle|1\rangle + |1\rangle|0\rangle\right]$$

  .

- A measurement on a qubit necessarily puts the state of the qubit in the measured state:

$$M|\text{state}\rangle = m|m\rangle$$

  .

- Quantum mechanics strictly forbids the cloning of an unknown qubit state. This means that, for arbitrary states $|A\rangle$ and $|B\rangle$, no Hermitian operator $\mathcal{C}$ exists with the property

$$\mathcal{C}|A\rangle|B\rangle \rightarrow |A\rangle|A\rangle$$

  .

A quantum network can now be built if we can manage to deliver a qubit state from one place to another. With the help of a quantum teleportation protocol (see Appendix C), such a delivery of a quantum state is possible. This type of network scheme is secure

due to several reasons. If a quantum state cannot be cloned, the only way to eavesdrop is to intercept the state. The receiver of the information, then, will notice the loss or the alteration of information while checking with the sender of the information, so the attempt to steal the information fails. A similar networking scheme, called quantum key distribution, makes use of a photon transfer through an optical fiber [7, 8]. Several attacks have been made to crack the key distribution method but none of them were particularly successful at a fundamental level, despite some achievements in corrupting some commercial systems [9–12].

## 1.5 Overview

Quantum computing is a sub-field of quantum information, an important and growing discipline of modern physics. As shown above, the applications of quantum computing and quantum information are very powerful. Equipped with a deep appreciation of the power of quantum computers and the principles of quantum information, we can now move forward to the next step: the realization of quantum computing.

In this thesis, we would like to demonstrate how robust a quantum computer is under realistic conditions. As Shor's algorithm occupies a central position in the field of quantum information, we will investigate a quantum computer running Shor's algorithm. The structure of the thesis is as follows:

- In Chapter 2, a brief history of quantum computing will be presented. This will help us understand how the field of quantum computing has developed and what the current state of the art is. It will also serve as background information on why and how we would like to investigate the robustness of a quantum computer.
- In Chapter 3, we will introduce Shor's algorithm in detail. The material discussed in this chapter will serve as the foundation in understanding how Shor's algorithm works at a fundamental level. It will also demonstrate how a quantum computer

beats its classical counterpart.

- In Chapter 4, we will evaluate the analytical result obtained from Shor's algorithm. We will discuss why this result is useless for integer factorization and why that uselessness is important in understanding Shor's algorithm. The Chapter also includes the significance of the analytical result itself.

- In Chapter 5, an actual simulation of a part of Shor's algorithm will be introduced. We will discuss the implementation scheme and the difficulty of performing actual quantum computations, specifically that of a quantum Fourier transform. Along with the scaling property in execution time and the memory requirement of the simulation, we will discover the exact mechanism according to which a quantum computer acquires its tremendous efficiency.

- In Chapter 6, we introduce hardware errors/noise in order to investigate the influence of such imperfections on our probability spectrum. We shall also determine how the difficulty of performing a quantum Fourier transform affects the probability distribution in the presence of noise.

- In Chapter 7, the way in which the rest of Shor's algorithm may be implemented will be introduced. We will also demonstrate why it is not desirable to implement this in the introduced way.

- In Chapter 8, we will present a unitarity-broken version of the simulation shown in Chapter 7, without any noise. How such a simulation is structured in breaking the unitarity will be shown in detail.

- In Chapter 9, a unitarity-broken, complete implementation of Shor's algorithm will be presented. How the final probability spectrum of Shor's algorithm looks like under realistic conditions will be demonstrated.

- A brief discussion of the results and conclusions are presented in Chapters 10 and 11, respectively.

# Chapter 2

# Brief History of Quantum Computing

Quantum computing started in 1961 [13] with the question of how much heat is produced in performing a single computational step. This question is important for many reasons. Primary is the question of how densely one can integrate computers. Obviously this finds its limitation when the chip gets too hot because too much heat is generated by performing computational steps. So the question of how much heat is produced is fundamental both from the physics/science point of view and from the technology/applications point of view. The question was answered by Landauer in 1961 [13]. He found that only irreversible computing steps generate heat.

Landauer's discovery then led to the question of whether reversible computing is possible, which would not generate any heat at all (since it could be conducted "adiabatically"). This was answered in the positive by Bennett, who discovered reversible computation [14] with classical logic gates. While these discoveries pertained to the classical level of computing, the question of dissipation and reversibility is already pointing directly at quantum mechanics.

Following the discovery of classical reversible computing by Bennett, the question now became how to construct actual reversible computing gates. The first answer was given by Toffoli, who suggested the reversible CONTROL-NOT gate (classical), also known as the CNOT gate [15]. Toffoli also invented the 3-bit gate CONTROL-CONTROL-NOT (CCNOT) or Toffoli gate, which he showed to be universal for classical digital computing. At this point, then, it was possible to construct completely reversible classical computers made out of CCNOT gates.

Now, with the idea of universal reversible computing fully established, it was a short step to the idea of a quantum computer, since quantum systems under the influence of a Hermitian Hamiltonian are also fully reversible (in the absence of coupling to an uncontrolled environment). Indeed, at the beginning of the 1980s, Benioff [16] and Feynman [1] introduced the notion of the quantum computer as a logical consequence of classical reversible computing. However, the quantum computing process was still envisioned as some large unitary operator, accomplishing the unitary transformation from the input states to the output states. What was needed was the concept of a universal quantum gate that would allow the modular construction of quantum computers.

Such a universal gate was introduced in 1989 in the form of Deutsch's three-qubit universal gate [17]. In addition, Deutsch showed that, because of the new possibilities of superposition and entanglement, quantum computing is much more powerful than classical computing.

The first actual quantum computing scheme was then introduced by Lloyd in 1993 [18]. It consisted of atoms perturbed by electromagnetic pulses. At this point, however, quantum computing was still a relatively obscure direction pursued by very few researchers.

An explosion of activity, however, occurred, when in 1994 Shor presented his quantum algorithm for the factorization of large semi-primes [5]. This discovery catapulted quantum computing from a niche research subject to the main stream of physics. The reason for this is that Shor's algorithm is tremendously powerful. While classical computer

codes need an execution time proportional to $\exp(L^{1/3})$ for the factorization of an $L$-digit semi-prime $n$ (i.e. $n = p \times q$, where $p$ and $q$ are prime), Shor's algorithm needs an execution time proportional to $L^2$ only, i.e. polynomial time versus exponential time. This qualitative difference is so profound that if quantum hardware existed that could run Shor's algorithm, all military and government secrets could be read like an open book!

In 1995, Schumacher [19] provided the notion of qubits as the quantum analog of classical bits, and the computing process, then, became the application of unitary operators (quantum gates) on qubits and qubit combinations.

A further advance in the simplification of quantum gates occurred in the same year, when Barenco et al. [20] showed that the two-qubit CNOT gate, together with the one-qubit rotation gate, form a complete set of gates from which all quantum logical processes can be constructed. So these one- and two-qubit gates replaced Deutsch's universal three-qubit gate as the smallest necessary universal gates. Following this discovery, the focus of the quantum computing community naturally shifted towards the experimental realization of a CNOT gate.

A possible experimental realization of a CNOT gate was then proposed in the same year (1995) by Cirac and Zoller [21] in the form of a string of ions with a phonon acoustic bus. Since the CNOT gate is universal, the scheme did more than just present a possible realization of the CNOT gate: It also defined one of the first architectures for a universal quantum computer: The ion trap quantum computer [22, 23].

Following the theoretical proposal by Cirac and Zoller, Monroe et al. [24] then implemented the Cirac-Zoller scheme experimentally and presented the world's first realization of a CNOT gate. Another promising quantum computer architecture, still investigated today as the basis of a quantum computer, was proposed at the same time by Turchette et al. [25], the cavity electrodynamic quantum computer [26, 27].

As we all know, classical computing would be impossible without error correction and

error correction codes. The same is true for quantum computing. Due to environmental influences, such as electrodynamic fields, cosmic rays, acoustic vibrations, and thermal noise, errors in classical and quantum computers invariable and unavoidably occur! These errors need to be corrected. Since errors, however, destroy the unitary time evolution of a quantum computer, it was believed before 1995 that even though the quantum computer is in principle much faster than a classical computer, the quantum computer, because of the unavoidable errors, will never work. In classical computers success is guaranteed by error correction, which was thought to be impossible to implement for the unitary quantum time evolution.

This assumption was incorrect. The first scheme for quantum error correction was suggested in 1995 by Shor [28]. While the exponential speed-up of quantum computers was nice, it was the possibility of quantum error correction that finally proved that quantum computing was not just a pipe dream, but was serious business. From this point on, industry as well as military and government funding agencies got seriously interested in quantum computing, an interest that has continued gathering momentum even today.

While in 1995 only Shor's algorithm existed as a practical non-academic application of quantum computers, this quickly changed in 1996 when Grover developed fast quantum algorithms for sorting of data bases ("quantum Google") and pattern recognition [29].

Following the advances of the last decade of the 20th century, many exciting developments occurred during the first decade of the 21st century, culminating in the present state of universal and adiabatic quantum computers.

Concerning the field of adiabatic quantum computers, on May 11, 2011, the private computer company D-Wave announced "D-Wave One", a 128-qubit adiabatic quantum computer, which is certainly a world record, that performs a single task, i.e. discrete optimization [30]. Lockheed-Martin bought this quantum computer at the sticker price

of 10 million dollars, which demonstrates that a quantum computer has begun to be a commercial success already.

Concerning the field of universal quantum computers, one of the latest developments is a universal quantum simulator based on the ion-trap architecture [31]. In contrast to D-Wave One, it has only 5 qubits, but it can run any (5-qubit) quantum program, whereas D-Wave One is a special-purpose quantum computer that runs only a single, specialized task.

Given these latest developments in the quantum computing field, including IBM's advancement in quantum device performance presented at the annual meeting of the American Physical Society in 2012 [32], it is fair to say that the field of quantum computing is flourishing and developing faster than anyone expected at the turn of the past century.

# Chapter 3

# Shor's Algorithm

In this chapter, Shor's Algorithm will be investigated in detail. Since Shor's Algorithm is designed to decrypt RSA encrypted messages, the RSA encryption mechanism will be introduced in section 3.1. Then, a classical algorithm on which Shor's algorithm is based, Miller's algorithm, will be presented. Exactly how Shor's algorithm works and why it is immensely powerful will be analyzed in detail, thereafter.

## 3.1 RSA Encryption

With the unprecedented development in information technology (IT) in the past decades, it is hard to imagine a life without the convenience of having access to Internet. IT is at the heart of the modern lifestyle; ranging from emails and social network services to online banking transactions at a personal level, and from local office management to international intelligence warfare. In order to keep all these transactions secret, it is inevitable, therefore, to use a fast, yet secure, way of encrypting information. The most popular of currently used encryption methods is the RSA encryption method.

The power of the RSA encryption method is based on number theory: currently no fast

way exists to factor a large semi-prime number (a number, which is the product of two different primes) into its two constituent prime numbers. As it will be shown shortly, one can make an encrypting tool publicly available, without compromising the security, by using this fact: it is easy to multiply two prime numbers, but it is hard to factor the product into its constituent prime numbers. This is precisely how RSA encryption enables a fast encoding in a secure manner.

The way RSA works is the following. Suppose we have the modulus number $n$, which is a semi-prime number, whose prime factors are $p$ and $q$. We choose a public key, $e$, which is smaller than $(p-1)\times(q-1)$ (Euler's totient function) and relatively prime with $n$. Then, a private key, $d$, may be found using the following relation: $d \times e = 1 \mod (p-1)\times(q-1)$ ($d$ is the multiplicative inverse of $e \mod totient$). With the two keys, encoding and decoding formula are constructed as follows.

$$(Cipher) = (Number\ or\ information\ to\ be\ secured)^e \mod n,$$

$$(Number\ or\ information\ to\ be\ secured) = (Cipher)^d \mod n.$$

Hence, the receiver of the information would publish the public key, $e$, and the modulus number, $n$, so that the senders of the information can cipher the message without delay, and keep the private key, $d$, so that the received information may be decoded right away. It is the difficulty of finding $d$, the private key, which makes the RSA mechanism secure. Therefore, if $d$ can be easily found by using the publicly available numbers, $e$ and $n$, the RSA encryption method would no longer be safe.

As $d$ is the multiplicative inverse of $e \mod totient$, if $n$ can be factored into its two prime factors, $p$ and $q$ so that the totient can be computed, the RSA encryption scheme is broken. A natural question would now be how hard it is to factor a semi-prime number into its two prime factors. Figure 3.1 demonstrates how difficult the task of prime factoring can be. Even with the best classical algorithm known to us, for a $k$ digit semi-prime number $n$, the number of computations needed to factor this number is exponential in $k$ [33–38].

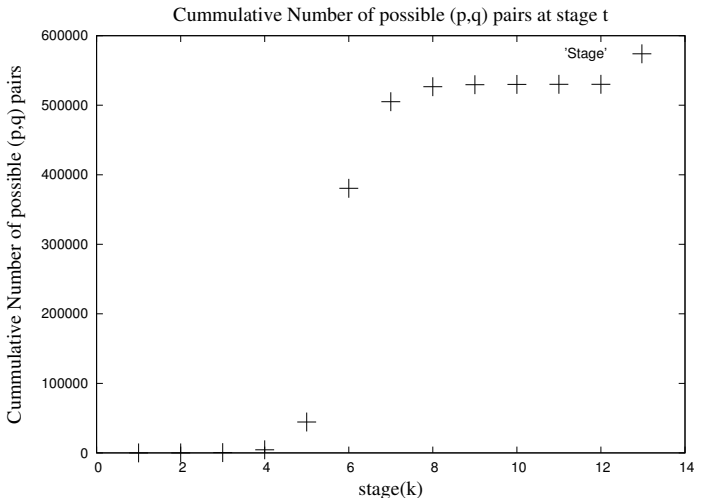



Figure 3.1: Effort of factoring a semi-prime number. The semi-prime number factored is 491293416799. In order to factor, we start from the least significant digit of the semi-prime number and generate pairs of prime numbers of minimal length $t$, whose product matches the last $t$ digits of the semi-prime number.

## 3.2 Miller's Algorithm

One of many classical algorithms that may be used for factoring semi-primes is Miller's algorithm. This algorithm is of particular interest due to the fact that a quantum version of the algorithm is Shor's algorithm. Hence, understanding Miller's algorithm is crucial for investigating Shor's algorithm. Even though, as a classical algorithm, Miller's algorithm is not efficient in factoring a semi-prime number, the details discussed in this section will serve as a steping stone to Shor's algorithm.

Suppose we pick two integers, $x$ and $\omega$, which satisfy the following condition: $(x^{\omega} - 1)$ mod $n = 0$, where $\omega$ is the smallest of all such exponents. When the following three additional conditions are met, i.e. $(i)$ $\omega$ is even, $(ii)$ $x$ is larger than 1 and smaller than the modulus number $n$, and $(iii)$ the greatest common divisor between $x$ and $n$ is 1, then $x$ and $\omega$ are referred to as the seed and the order of the seed, respectively. By defining an integer quantity $M = x^{\frac{\omega}{2}}$, if there exist integers $A$ and $B$ such that $M = A + 1 = B - 1$

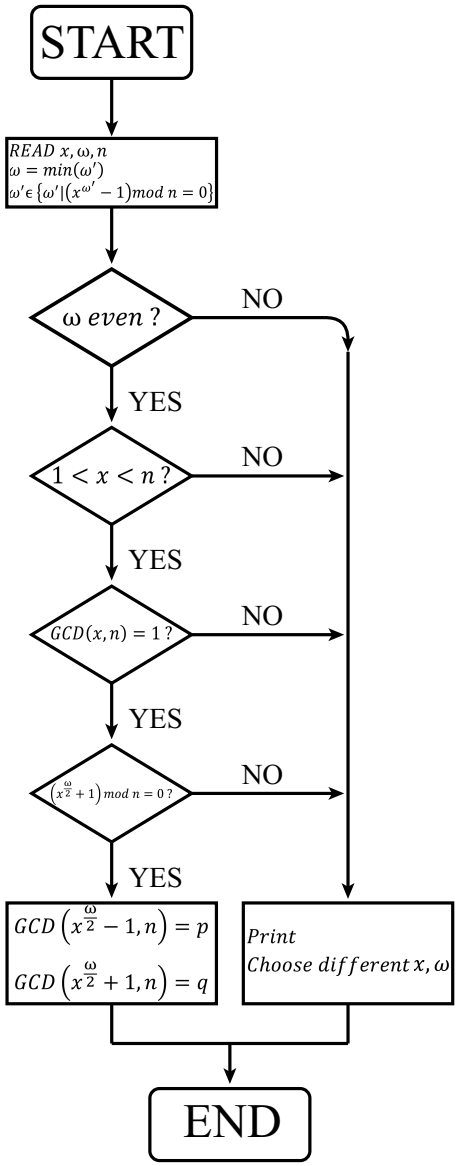



Figure 3.2: Flow chart of Miller's algorithm. The chart demonstrates that it is finding the suitable seed and the order that enables factoring of a semi-prime number in Miller's algorithm.

and $B \mod n = 0$, then the two prime factors $p$ and $q$ of the modulus number $n$ are the greatest common divisor of $A$ and $n$ and that of $B$ and $n$. This is so, because the quantity $x^{\omega} - 1 = (x^{\frac{\omega}{2}} - 1) \times (x^{\frac{\omega}{2}} + 1) = A \times B$ is an integer multiple of $n$. Since it is guaranteed that the quantities $(x^{\frac{\omega}{2}} - 1) = A$ and $(x^{\frac{\omega}{2}} + 1) = B$ are not divisible by $n$ (due to the way $\omega$ is defined and the conditions stated above), the desired prime factors $p$ and $q$ are found with certainty. A flow chart of Miller's algorithm is shown in figure 3.2.

The difficulty of factoring via Miller's algorithm arises in finding the order, $\omega$. When the number of digits of the modulus number increases by 1, there is an exponential increase in the number of allowed values of $(x^{\omega} - 1)$. Given a suitable seed $x$ such that there exists a proper $\omega$, finding the method to efficiently obtain the order of the seed now becomes the core task in cracking RSA. We shall see in the next section that quantum computing provides an extremely powerful way of determining the order $\omega$.

## 3.3 Quantum Factoring

In order to transform Miller's algorithm into a quantum version, the following mathematical observation is useful: $x^{\omega} \mod n = 1 \leftrightarrow f(r + \omega) = f(r)$ where $f(r) = x^r \mod n$. In this way, we immediately observe that $\omega$ is a period of the function $f$. Hence, constructing an algorithm that returns the value of the period of the function $f$ becomes our interest.

One may ask why we even need quantum computing to find the period $\omega$ to begin with. This is so, since a classical computer fails miserably in finding $\omega$. Suppose we have a 300-digit semi-prime number to be factored. Then, $f(r)$ is indistinguishable from a random function on $\sim 10^{300}$ data points. If we attempt to find the periodicity, because of $f(r)$'s randomness, which looks like white noise, only a Fourier transform can determine the period, $\omega$. It is practically impossible, however, to perform a Fourier transform on

$\sim 10^{300}$ data points. To load and execute a Fourier transformation on $\sim 10^{300}$ data points and then to store the transformed data demands unimaginably long time, which easily exceeds the age of the universe.

A quantum computer, however, loads multiple inputs and performs a Fourier transform on all numbers simultaneously, by using the massive parallelism that arises from superposition and entanglement. This is precisely the point where a quantum computer beats a classical computer and ultimately results in a polynomial scaling in the number of computational steps whereas the classical counterpart scales like an exponential in the number of digits to be factored.

A complete Shor's algorithm is performed in the following way. Suppose we have two registers, denoted by register I and II, with at least $N$ qubits each, where $N$ is the number of qubits needed to load $n$, i.e. the semi-prime number to be factored. We initialize the states of each qubit in the superposition state $\left(\frac{|0\rangle+|1\rangle}{\sqrt{2}}\right)_{Reg\ I}$ in register I and the zero state $|0\rangle_{Reg\ II}$ in register II. Then, we perform a function mapping on register II in such a way that $|Superposition\ State\rangle_{Reg\ I}|0\rangle_{Reg\ II} \rightarrow |Superposition\ State\rangle_{Reg\ I}$ $|f(Superposition\ State)\rangle_{Reg\ II}$. Finally, we operate a quantum Fourier transform (QFT) on register I to obtain the final state, $|\psi\rangle_{final}$.

$$|\psi\rangle_{final} = |QFT\ of\ Superposition\ State\rangle_{Reg\ I}|f(Superposition\ State)\rangle_{Reg\ II}.$$

A full mathematical expression is presented below.

- Define a quantum Fourier transform (QFT):

$$\mathcal{U}^{QFT}|s\rangle = \frac{1}{\sqrt{2^{N'}}} \sum_{l=0}^{2^{N'}-1} e^{\frac{2\pi isl}{2^{N'}}}|l\rangle$$

(in analogy to $y_s = \sum_{l=0}^{2^{N'}-1} e^{\frac{2\pi isl}{2^{N'}}} x_l$, a discrete, classical, Fourier transform).

1. Initialization of register I and II ($N'(\geq N)$ qubits in register I, $2^{N'}$ possible states):

$$|\psi\rangle_{initial} = \frac{1}{\sqrt{2^{N'}}} \sum_{a=0}^{2^{N'}-1} |a\rangle_{Reg\ I}|0\rangle_{Reg\ II}.$$

2. $f$-mapping on register II:

$$|\psi\rangle = \frac{1}{\sqrt{2^{N'}}} \sum_{a=0}^{2^{N'}-1} |a\rangle_{Reg\ I}|f(a)\rangle_{Reg\ II}.$$

3. QFT on Reg I:

$$|\psi\rangle_{final} = \frac{1}{2^{N'}} \sum_{a=0}^{2^{N'}-1} \sum_{y=0}^{2^{N'}-1} e^{\frac{2\pi i a y}{2^{N'}}} |y\rangle_{Reg\ I}|f(a)\rangle_{Reg\ II}.$$

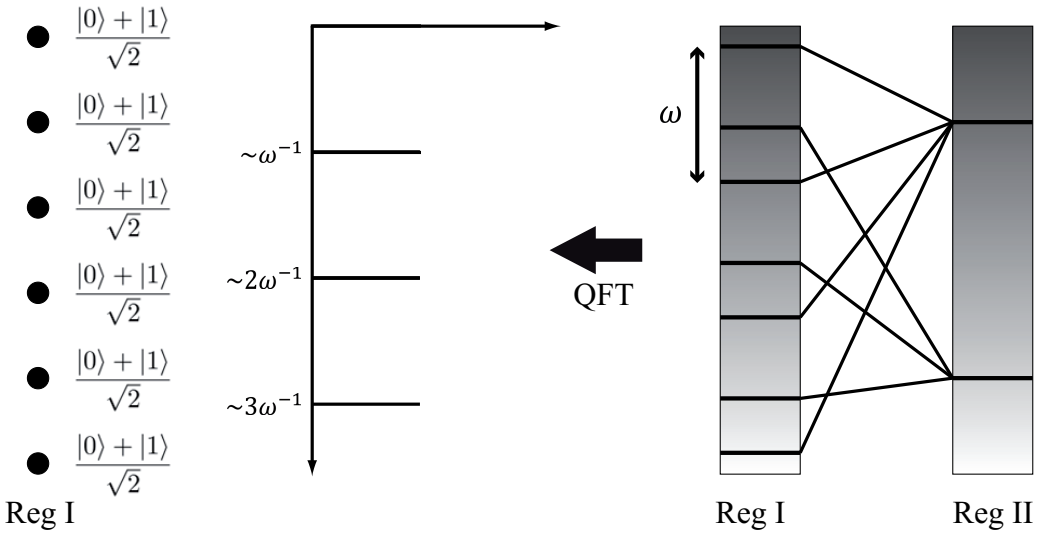



Figure 3.3: A schematic diagram of Shor's algorithm. The initialization step of qubits in register I requires only one operation per qubit that results in a superposition state $\frac{|0\rangle+|1\rangle}{\sqrt{2}}$ (see left). Once the $f$-mapping is performed (see right), for any equivalence class we choose in register II, we obtain the same periodicity of the equivalence class elements in register I. A quantum Fourier transform, then, picks up the periodicity (see middle).

It is important to understand the way in which we have arrived at the final state in order to fully appreciate the power of quantum computing. In the initialization process,

for instance, we have loaded a full spectrum of numbers from 0 to $2^{N'}-1$, using the superposition states (see figure 3.3 for a schematic diagram). This provides us with a linear scaling of the number of computation steps in the loading stage per additional digit in the modulus number. It is precisely the utilization of simultaneous operations on all number states via the quantum nature of qubits that boosts the computing power exponentially.

The way we constructed the final state now becomes pivotal in determining the periodicity $\omega$. By inspecting the final state, we immediately notice that the second register is periodic with periodicity $\omega$. This allows us to rewrite the final state using the following fact.

- Using the periodicity, $\omega$, in $f(a)$ in the second register, define:

$$a = \tilde{a} + l\omega \; ; \; 0 \leq \tilde{a} < \omega \; ; \; L = \max(l).$$

With the defined quantities above and $\tilde{y}$'s, which are the associated $y$'s for given $\tilde{a}$'s, we obtain an alternative expression of the final state $|\psi\rangle_f$.

$$|\psi\rangle_f = \frac{1}{2^{N'}} \sum_{\tilde{a}=0}^{\omega-1} \sum_{\tilde{y}=0}^{2^{N'}-1} \sum_{l=0}^{L} e^{\frac{2\pi i(\tilde{a}+l\omega)\tilde{y}}{2^{N'}}} |\tilde{y}\rangle_{Reg\ I} |f(\tilde{a})\rangle_{Reg\ II}. \tag{3.1}$$

It is quantum entanglement that allows us to extract the periodicity from the final state. Inspection of equation (3.1) shows that, for any state in register II, or for any value of $\tilde{a}$, the periodicity of the entangled states in register I is $\omega$. This is because the entanglement due to the $f$-mapping between a state in register II and the elements in register I that results in the same state in register II, guarantees the same periodicity for any set of equivalence classes in the first register. A visualization of this process is shown in figure 3.3. The final reading, or measurement, of the state $|\tilde{y}\rangle_{Reg\ I}|f(\tilde{a})\rangle_{Reg\ II}$ yields the amplitude $A(\tilde{y}, \omega, \tilde{a})$, where

$$A(\tilde{y}, \omega, \tilde{a}) = \frac{1}{2^{N'}} \sum_{l=0}^{L} \exp \frac{2\pi i(\tilde{a} + l\omega)\tilde{y}}{2^{N'}}. \tag{3.2}$$

One may wonder why we still have an $\tilde{a}$ dependence in our amplitude shown in equation (3.2). But this dependence disappears when we compute the probability, $P(\tilde{y}, \omega)$, which reads,

$$P(\tilde{y},\omega) = |A(\tilde{y},\omega,\tilde{a})|^2 = \left(\frac{1}{2^{N'}}\right)^2 \left|\sum_{l=0}^{L} \exp\frac{2\pi i(\tilde{a}+l\omega)\tilde{y}}{2^{N'}}\right|^2 = \left(\frac{1}{2^{N'}}\right)^2 \left|\sum_{l=0}^{L} \exp\frac{2\pi i l\omega\tilde{y}}{2^{N'}}\right|^2. \tag{3.3}$$

We see that the probability shown in equation (3.3) is similar to a Dirac Comb structure. Hence, the probability is suppressed to near 0 whenever the exponent is not an integer multiple of $2\pi$. The condition that the probability has a peak is, indeed, very similar to that of a Dirac Comb: $\frac{\omega\tilde{y}}{2^{N'}}$ is an integer. When the condition is met, the probability equation reduces to a simpler form, $P(\tilde{y},\omega) = \left(\frac{L+1}{2^{N'}}\right)^2$.

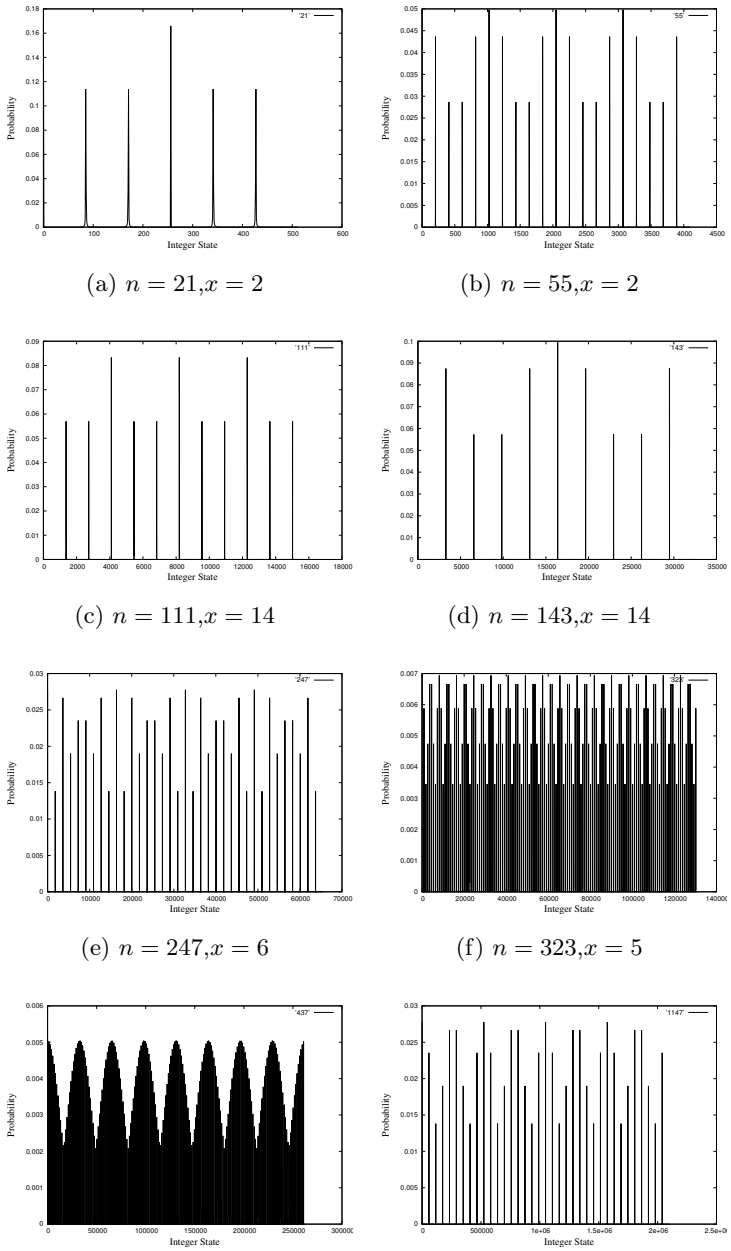
The computation of $\omega$ is now straightforward. Recalling that the value of $\omega$ is the desired output, we immediately observe that $\omega$ must be an integer multiple of the quantity, $\frac{2^{N'}}{\tilde{y}}$. Since we know the number of qubits in register I, $N'$, and we measure $\tilde{y}$ from the first register, the quantity of $\omega$ can now be found from the results of multiple runs of Shor's algorithm. This process is sometimes called "classical post processing", which is an inevitable part of Shor's algorithm. A quick mathematical reasoning behind the inevitability is due to the underdetermined unitary matrix that comprises the entire Shor's algorithm.

# Chapter 4

# Evaluation of the Analytical Result of Shor's Algorithm

As far as the factoring of a semi-prime number is concerned, the analytical formula (3.3) by itself is useless in that we must have prior knowledge of the order of a given seed. However, the insights that the formula provides are extremely valuable. In this chapter, we shall discuss the role of the formula in regards to the simulation of Shor's algorithm and investigate what the formula tells us about quantum computing.

The analytical formula plays a crucial role, since, under ideal conditions, it provides us with the probability spectrum of $\tilde{y}$, given the semi-prime number to be factored, the number of qubits, the seed, and the order. The evaluation of the formula, therefore, provides us with a means to check the analytical formula, and ultimately serves as a template for the expected result of an actual quantum computation. The peak structure obtained from the formula, then, serves as the comparison group when Shor's algorithm is run on a quantum computer under realistic conditions. An example of such a peak structure is shown in figure 4.1. We note that we use the number of qubits in register I as the integer floor value of $2\ln(N) + 1$.




(a) $n = 21$,$x = 2$ (b) $n = 55$,$x = 2$

(c) $n = 111$,$x = 14$ (d) $n = 143$,$x = 14$

(e) $n = 247$,$x = 6$ (f) $n = 323$,$x = 5$

(g) $n = 437$,$x = 5$ (h) $n = 1147$,$x = 5$

Figure 4.1: Probability Spectra of integer states via the evaluation of the analytical formula of Shor's algorithm. Note the peak structure, which is similar to a Dirac Comb.

A first glimpse of the peak structure demonstrates how a quantum computer is a curious hybrid between an analog computer (quantum, superposition, and amplitudes, which are all continuous) and a digital computer, since the act of measurement returns digital information, i.e. actual integer numbers such as, in our case, $\tilde{y}$. Suppose a quantum computer were a completely analog device. Then, we would obtain $\tilde{y} \pm \Delta\tilde{y}$ as a real number. If we have $\tilde{y}$ that is of the order of $10^{200}$, then unless $\frac{\Delta\tilde{y}}{\tilde{y}} \simeq \frac{1}{10^{200}}$, which is impossible to obtain with an analog device, there would be too many integers included in the spread, which results in the impossibility of factoring. This is so, because a typical analog device has $\frac{\Delta\tilde{y}}{\tilde{y}} \gtrsim 10^{-5}$, which returns $10^{200} \times 10^{-5} = 10^{195}$ integers in the spread (i.e. orders of magnitude more than there are particles in the universe!).

We recall from the last chapter that classical post processing is necessary for computing the order, $\omega$. Based on figure 4.1, we immediately observe that there is a high probability of observing the *integer* (not real) multiple of $\frac{2^N}{\omega}$ for $\tilde{y}$ values. If we happen to observe $\tilde{y}$ values of the third and the fifth peaks, for instance, then $\frac{2^N}{\tilde{y}}$ will be numerically obtained to be $\frac{\omega}{3}$ and $\frac{\omega}{5}$, respectively. Repeating this process, we obtain multiple values that are $\omega$ divided by integers, which can then be used to find the value of $\omega$ itself.

As the mere evaluation of formula (3.3) is not a simulation of a quantum computer, it now becomes necessary to truly simulate Shor's algorithm at the microscopic level in order to investigate the influence of noise on the computational process. The most realistic way of accomplishing this is to implement, gate by gate, an actual quantum computer. It is not necessary to specify the physical implementation of the gates. This allows the flexibility of specifying the noise once the actual physical realization of a particular gate is known. Such a realization may be in the form of two-level atoms, nuclear spins, photons in cavities, etc., depending on the quantum computer hardware used to implement the gates.

An actual implementation of a quantum computer with gates is done in two steps.

- Step1 : A quantum computer running Shor's algorithm without gate-by-gate im-

plementation of the $f$-mapping.

- Step2 : A full microscopic implementation of Shor's algorithm on a simulated quantum computer.

# Chapter 5

# Gate-by-Gate Implementation of Shor's Algorithm: Ideal $f$-mapping

As shown in the previous chapter, the way in which we use gates in simulating a quantum computer is analogous to the assembly language in a classical computer. Even though a physical implementation of the commands of the quantum gate may differ depending on the architecture of a computer, the desired results of the operation are identical, which enable the construction of a logic circuit. Since we would like to investigate how a universal quantum computer performs in running Shor's algorithm under realistic conditions, there is no better way than implementing quantum gates for the simulation of a quantum computer in that such a way of simulating allows full flexibility in the architecture of a quantum computer.

In this chapter, we would like to use such a gate-by-gate implementation for the initial register-loading step and the following quantum Fourier transform. The purpose of such a simulation is to investigate the scaling of memory requirements and the execu-

tion time, which provides us with a rough estimate of the maximal number of qubits that can be simulated in practice. In addition, a classical simulation of a quantum computer will demonstrate how inefficient a classical computer is in running a quantum algorithm.

We should keep in mind that this simulation is not a complete implementation of Shor's algorithm. This is so, because it is the $f$-mapping that sorts all the states into their appropriate equivalence classes. Without $f$-mapping, the periodicity, $\omega$, cannot be determined and, therefore, the factoring of a semi-prime number is not possible. Hence, for the purpose of investigating the initialization of qubits and the quantum Fourier transform in Shor's algorithm (i.e. the periphery part of Shor's Algorithm), we assume pre-knowledge of the equivalence classes in order to obtain the final probability spectrum for measuring $\tilde{y}$.

Since we will make heavy use of certain basic single-qubit quantum gates during the implementation that will be shown in this chapter, the first section will be devoted to the introduction of such basic quantum gates. Then, we will use these gates to construct the initialization step and the quantum Fourier transform parts of Shor's algorithm. Subsequently, the structure of the simulation code will be laid out. The shape of the probability spectrum and the scaling of the resources needed for the simulation will be presented thereafter.

## 5.1 Basic Single-Qubit Quantum Gates

In order to achieve consistency, we now choose a systematic notation to analyze quantum gates. Throughout this thesis, we will use the conventional Pauli matrices and their associated Lie-Algebra for the transitions among the computational states. We note that this is directly applicable to an ion-trap quantum computer, as such matrices correspond to the action of laser beams shun on a two-level atom.

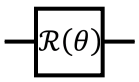



Figure 5.1: Circuit representation of the single-qubit rotation gate. The gate rotates the phase of $|1\rangle$ by $\theta$ and leave the phase of $|0\rangle$ unchanged.

### 5.1.1 Single-Qubit Rotation Gate

A single-qubit rotation gate performs a phase shift on one state, whereas the other state's phase is not affected. A matrix form of such a gate reads

$$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}, \tag{5.1}$$

where $\theta$ is the angle of rotation. A circuit diagram of the single-qubit rotation gate is shown in figure 5.1. Since only one of the two computational states is affected by (5.1), constructing this gate by purely using Pauli matrices is not feasible. This is so, because Pauli matrices always act on both states. Therefore, for the case of a two-level atom, for instance, a single-qubit rotation gate can be built using auxiliary states other than the two computational states. One can show that such a gate may be constructed, e.g., with the help of circularly polarized light [39].

### 5.1.2 Hadamard Gate




Figure 5.2: Circuit representation of the Hadamard gate. The gate performs the Hadamard operation on the input state.

A Hadamard operation (see figure 5.2 for a circuit diagram representation) makes the following transitions: $|0\rangle \rightarrow \frac{|0\rangle+|1\rangle}{\sqrt{2}}$, $|1\rangle \rightarrow \frac{|0\rangle-|1\rangle}{\sqrt{2}}$. In matrix form, this transformation

is represented by

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}. \tag{5.2}$$

Up to a global phase of $i$, a Hadamard matrix can be constructed by using Pauli matrices. Using Pauli-$z$ and -$y$ matrices,

$$e^{-i\sigma_z\omega}e^{-i\sigma_y\epsilon} = \begin{pmatrix} e^{-i\omega}\cos(\epsilon) & -e^{-i\omega}\sin(\epsilon) \\ e^{i\omega}\sin(\epsilon) & e^{i\omega}\cos(\epsilon) \end{pmatrix}.$$

For $\omega = \frac{\pi}{2}$ and $\epsilon = \frac{3\pi}{4}$, the above matrix becomes

$$i\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{pmatrix}.$$

If we use a single-qubit rotation on both computational states with an angle of $-\frac{\pi}{2}$, the global phase $i$ is eliminated. It is important to notice that this global phase is not the global phase of the entire qubit system but of a single qubit. If a gate operation happens to involve multiple qubits at a time, then the single-qubit global phase cannot be ignored.

## 5.2 Construction of Gates

### 5.2.1 Initialization of Qubits

From Chapter 3 we know what the initial states of the qubits in registers I and II should be: a superposition state of $\frac{|0\rangle+|1\rangle}{\sqrt{2}}_{Reg\ I}$ in register I and $|0\rangle_{Reg\ II}$ in register II. Assuming we can set the states of all qubits to $|0\rangle$, i.e. we measure every single qubit to be in the state of $|0\rangle$, the only additional step needed to complete the initialization is to make a transition from the state $|0\rangle$ to the state $\frac{|0\rangle+|1\rangle}{\sqrt{2}}$. The most effective way to perform this is to use a Hadamard gate as shown in equation (5.2). This is natural, since the Hadamard gate is one of the most heavily used gates in quantum computing.

### 5.2.2 Quantum Fast Fourier Transform

By inspecting the definition of a quantum Fourier transform (see section 3.3), we make the following important observations.

- The integer state to be transformed is a product state involving each qubit state according to

$$\mathcal{U}^{QFT}|s\rangle = \mathcal{U}^{QFT}(|s_{N'-1}\rangle|s_{N'-2}\rangle...|s_0\rangle).$$

- The resultant qubit states span the entire number space, which are also product states of the single-qubit states:

$$\frac{1}{\sqrt{2^{N'}}}\sum_{l=0}^{2^{N'}-1} e^{\frac{2\pi isl}{2^{N'}}}|l\rangle = \frac{1}{\sqrt{2^{N'}}}\prod_{m=0}^{N'-1}\sum_{l_m=0}^{1}(phase\ factor)|l_m\rangle.$$

- Comparing the two equations above, the phase factor must be proportional to the exponential of the product of the input and the output qubit states:

$$(phase\ factor) = e^{const.\times function(s_0,s_1,...,s_{N'-1})\times l_m}.$$

Suppose the phase factor of $|l_m\rangle$ is $e^{2\pi i[.s_{N'-1-(m)}s_{N'-1-(m-1)}\cdots s_{N'-1}]\times l_m}$. Using the method of induction, one can easily prove that this choice of phase leads to the desired quantum Fourier transform

$$\mathcal{U}^{QFT}(|s_{N'-1}\rangle|s_{N'-2}\rangle...|s_0\rangle) = \frac{1}{\sqrt{2^{N'}}}\prod_{m=0}^{N'-1}\sum_{l_m=0}^{1} e^{2\pi i[.s_{N'-1-(m)}s_{N'-1-(m-1)}\cdots s_{N'-1}]\times l_m}|l_m\rangle. \tag{5.3}$$

First, we notice that the order of subscripts is reversed. This means a binary number embedded in the qubit states are to be read in the reversed order after a quantum Fourier transform is performed. We also note that the expression (5.3) can now be decomposed into combinations of Hadamard gates and single-qubit rotation gates. This can be shown most easily by writing out the product in equation (5.3):

$$\{|0\rangle + e^{2\pi i[.s_{N'-1}]}|1\rangle\}_0\{|0\rangle + e^{2\pi i[.s_{N'-2}s_{N'-1}]}|1\rangle\}_1...\{|0\rangle + e^{2\pi i[.s_0s_1...s_{N'-1}]}|1\rangle\}_{N'-1}. \tag{5.4}$$

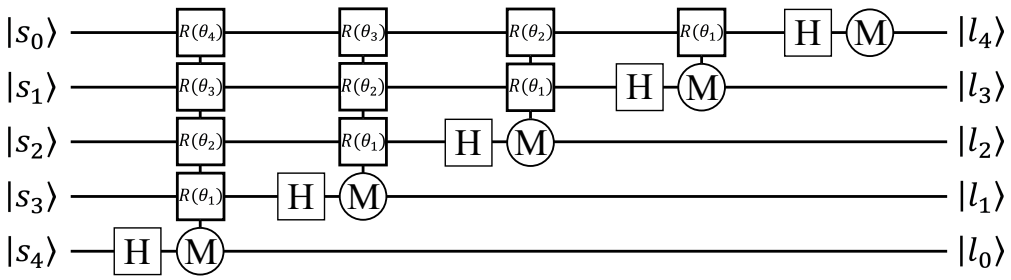



Figure 5.3: Logic circuit of the quantum Fourier transform. Gate $M$ performs the measurement of a qubit state. The single qubit rotation gate's argument $\theta_j$ is $2\pi\frac{s_{N'-1-m}}{2^{j+1}}$, where $s_{N'-1-m}$ is from the measurement of the $m$'th qubit (enumerated from the bottom; see right). Notice that $\theta_j$ is dependent of the spacing between the reference qubit that is measured and the operated qubit, which is $j$ spacings away from the reference qubit. The diagram is inspired by Mermin [6].

The above expression (5.4) clearly shows that a single-qubit rotation gate needs to be performed on state $|1\rangle$ with a rotation angle $2\pi\frac{s_{N'-1-m}}{2^{j+1}}$, where $j$ runs from 1 to $N'-1-m$ and $m$ runs from 0 to $N'-2$. One may wonder why we excluded the angle $2\pi i[.s_{N'-1-m}]$ on the $m$'th qubit. This is because we want both states $|0\rangle$ and $|1\rangle$ as our result and the phase of the state $|1\rangle$ to be $e^{2\pi i[.s_{N'-1-m}]}$, which is precisely a Hadamard gate. The logic circuit of a quantum fast Fourier transform operation is shown in figure 5.3 to help visualization. We note that the number of gates is quadratic in the number of qubits.

## 5.3 Structure of Simulation Code

### 5.3.1 Initialization and Equivalence Classes

As shown in section 5.2, the coding is done in the following way. First, we ensure all of the qubit states are $|0\rangle$'s. In order to complete the initialization step, we apply a Hadamard operation to each qubit in register I. This can be coded on a classical computer very easily by declaring a matrix with two columns where each row represents each qubit

and each column represents states $|0\rangle$ and $|1\rangle$, where each element in the matrix is a complex number that corresponds to the phase of each state. First, we set the first column to be 1 and the second column to be 0. Then, we create a loop that runs over each row of matrix, which calls a subroutine that performs a Hadamard gate shown in subsection 5.1.2. Once the loop is completely run, we are left with a matrix with $\frac{1}{\sqrt{2}}$ in both columns for the rows that belong to register I. In order to make the next step easier, we produce an output array with complex numbers in each array element, where rows correspond to each integer number, ranging from all 0's to all 1's in register I, in binary form. Each array element's complex value, or the phase of each integer state, is computed by multiplying the matrix elements that correspond to each integer row number of the output array.

Since the $f$-mapping must be done classically, we need a separate program to produce the equivalence classes. Using the pre-knowledge of the seed and the associated order, we divide the output array produced by the initialization code into the appropriate equivalence classes. As we already know from Chapter 3, it is not important which one of the equivalence classes we choose. Therefore, we choose any one of the equivalence classes to proceed to the next step in Shor's algorithm. Such a choice is equivalent to a measurement of the second register and, therefore, the phases of the states must be properly re-normalized.

### 5.3.2 Limit of Quantum Fast Fourier Transform

The way in which the quantum fast Fourier transform is implemented entails a difficulty. As shown in subsection 5.2.2, in order to implement the quantum Fourier transform, it is necessary to execute a single-qubit rotation gate with an angle that is inversely proportional to the exponential of the spacing between the reference qubit and the rotated qubit. Hence, we hit a technical limit for performing such a small rotation. In addition, for a small enough angle, there is a physical limit that prevents us from

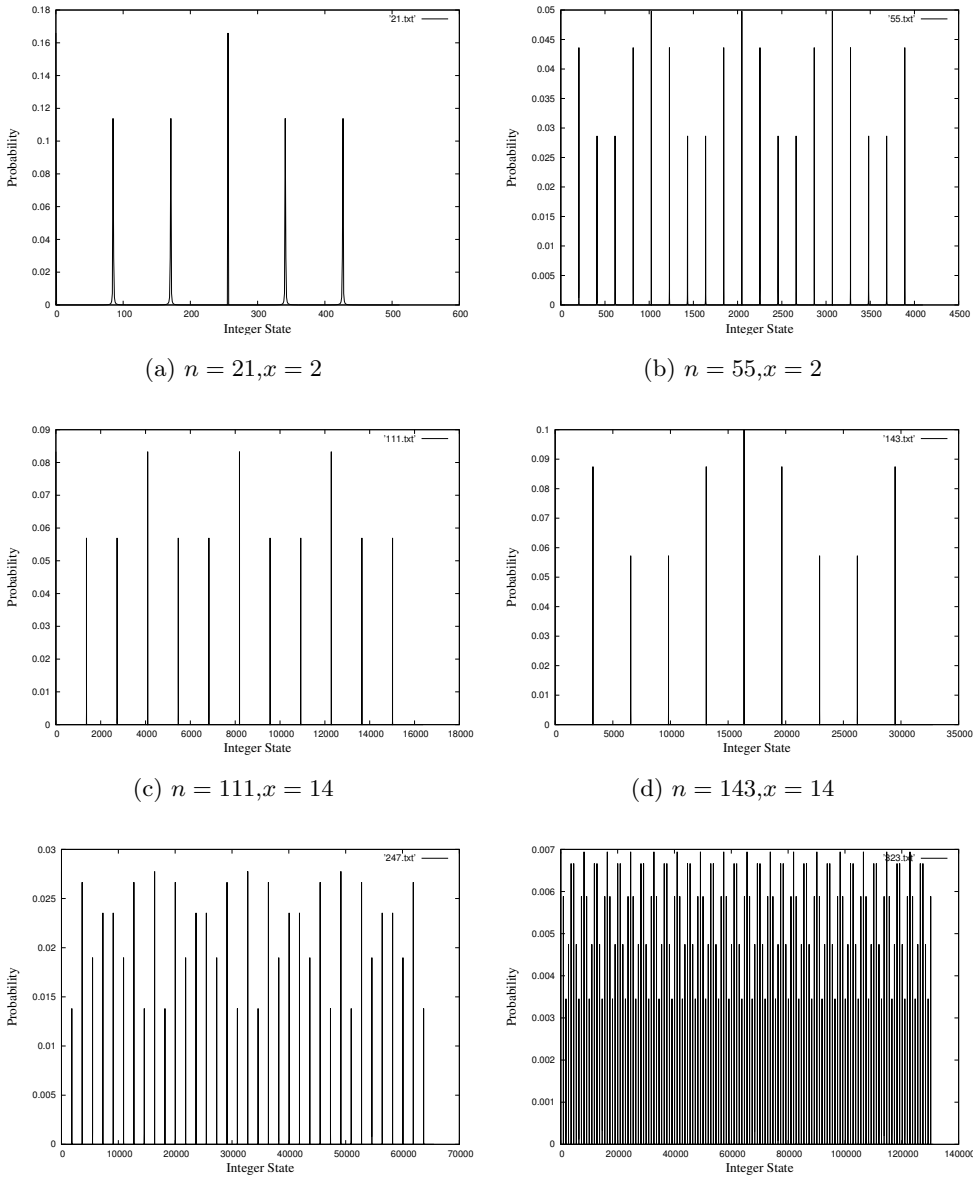
executing the rotation. Suppose we shine a laser beam on a two-level atom. Then, a small-angle rotation is realized by a short laser pulse. As there is an experimental floor for the time duration of the laser shot, it may not be possible to perform the required rotation in practice. Moreover, when the shot needs to become shorter than the period of the laser light itself, such a rotation may hit yet another physical limitation in that we are forced to confront Heisenberg's uncertainty principle, which makes the frequency of the laser to be indeterminate for a very short laser shot. Hence, such a limit can be best modeled by admitting the impossibility of performing a rotation gate. This can be coded by not rotating the operation-qubit state when the rotation angle is smaller than the minimum angle that is determined by the maximal qubit spacing (see figure 5.3).

Coding a quantum fast Fourier transform, implementing the limits discussed above, is now straightforward. By loading each element in an equivalence class at a time, we construct our code according to the logic circuit shown in figure 5.3. The limit can be imposed by telling the program to skip rotations with an angle smaller than a certain cut-off value. Once the circuit is run, we read the final output in reverse order, and record them as the phases of integer states. After running the circuit for all elements in an equivalence class, we compute the phase of a certain integer state by adding the phases of the integer state from all elements of the equivalence class. A probability spectrum can then be produced by taking the absolute square value of the phases.

## 5.4 Probability Spectrum and Scaling Property

### 5.4.1 Ideal Spectrum

A series of probability spectra that are obtained by gate-by-gate implementation of the periphery of Shor's algorithm is shown in figure 5.4. For the purpose of comparing with the analytical result of Shor's algorithm, the limits on the quantum Fourier transform are




(a) $n = 21$,$x = 2$ (b) $n = 55$,$x = 2$

(c) $n = 111$,$x = 14$ (d) $n = 143$,$x = 14$

(e) $n = 247$,$x = 6$ (f) $n = 323$,$x = 5$

Figure 5.4: Probability spectrum of Shor's algorithm obtained via a gate-by-gate implementation of the periphery of Shor's algorithm. The classical knowledge of equivalence classes is used. Limits on the quantum Fourier transform are not imposed.

ignored. We immediately observe that the result from the classical simulation is identical to the analytical result (see figure 4.1); the peak structure is well-demonstrated.

### 5.4.2 Effect of the Limit imposed on the Quantum Fourier Transform

In an attempt to investigate the effect of the limit imposed on the quantum Fourier transform, we recall that it is the probability of landing on the ideal peaks that we are mainly interested in, as such ideal peaks are the ones that return correct $\tilde{y}$ values, which enables the factoring of a semi-prime number (see Chapter 3 and 4). Hence, it is fair to say that the factoring is no longer possible if the probability of landing on the correct peaks falls below a certain threshold value. Therefore, it is our particular interest to investigate how the sum of the correct peak probabilities behaves in general, as the imposed limits on the quantum Fourier transform change.

The way in which we investigate such a behavior is the following. We try factoring multiple semi-prime integers with varying limiting angles for the quantum Fourier transform. Since such limiting angles directly result in the maximal spacing between the reference qubit and the manipulated qubit, as shown in section 5.3.2 and in figure 5.3, we produce probability spectra of the periphery of Shor's algorithm for the limit-spacing ranging from 1 to the total number of qubits in register I. The probability of landing on the correct peaks as a measure of the possibility of factoring can, then, be extracted.

In figure 5.5, we show how the peak probability behaves as the maximal qubit spacing varies. We note that both axes are scaled: When all the gates are implemented, the peak probability values on the $y$-axis reads 1, and the scaled maximal spacing between the reference qubit and the manipulated qubit, represented on the $x$-axis, also reads 1.

At a first glance, we observe excellent performance of the quantum Fourier transform without implementing all of the gates, which indicates potential hardware savings. In

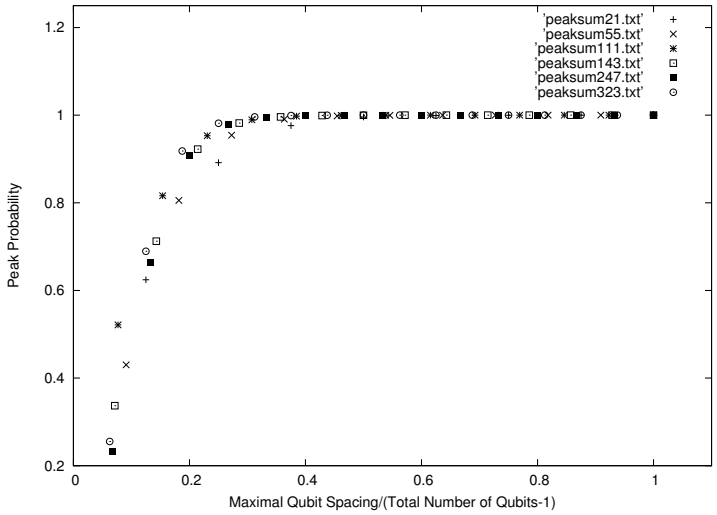



Figure 5.5: Peak probability of the periphery of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. The plot necessarily converges to $(1, 1)$, since the probability spectrum converges to the ideal spectrum when there is no limit imposed on the quantum Fourier transform.

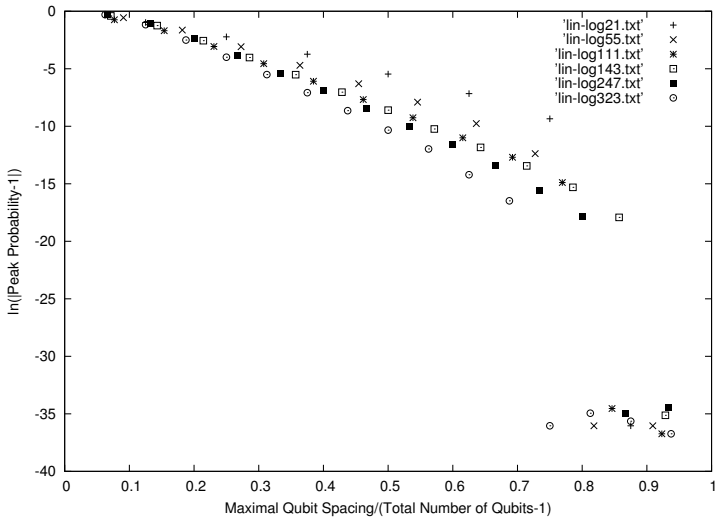



Figure 5.6: Log-scaled peak probability loss of the periphery of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. Linear behavior is observed from the plot. A set of points located at the bottom-right corner is due to limited machine precision, which may be improved by using quadruple precision.

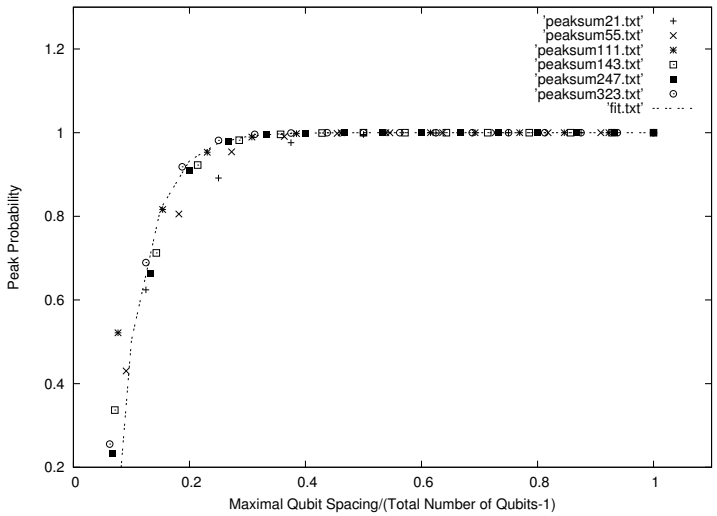



Figure 5.7: Function fit of peak probability of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. The fitted function $y = f(x)$ is $1 - e^{-20(x-.065)}$.

addition, we see scaling behavior in figure 5.5. In an attempt to extract the scaling law, we try $(i)$ adjusting $y$ to be the distance between the convergence value 1 and the raw data, then $(ii)$ applying a logarithm on the transformed $y$. The resulting plot after the transformation, $y \to \ln|y-1|$, is shown in figure 5.6: We observe linear behavior of the transformed plot.

The linear behavior shown in figure 5.6 suggests a particular form of the fit function of the raw data in figure 5.5: $y = 1 - e^{-\mu(x-A)}$, where $\mu$ and $A$ are our adjustable parameters. We find that $\mu = 20$ and $A = 0.065$ produce a well-fitting function to our raw data (see figure 5.7).

The behavior of the raw data can also be captured analytically. From the circuit diagram shown in figure 5.3, we observe that the cumulative angle rotated by the Hadamard gate and the single-qubit rotation gates amounts to $\sim [.s_{N'-1-\tilde{m}} s_{N'-1-(\tilde{m}-1)} ... s_{N'-1}]$ for the $\tilde{m}$'th qubit. If we define $k$ be the cutoff qubit spacing such that the applied rotation amounts to $\sim [.s_{N'-1-(\tilde{m})} s_{N'-1-(\tilde{m}-1)} ... s_{N'-1-(\tilde{m}-k)}]$, then the missing rotation angle

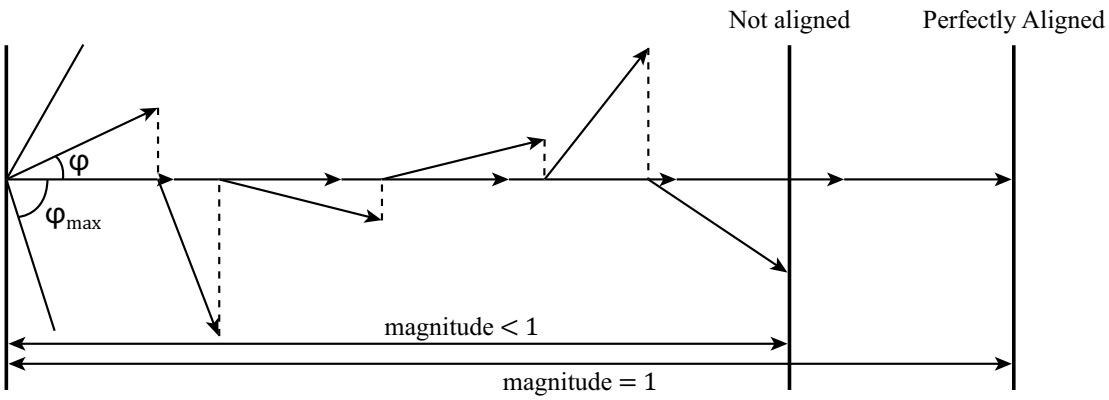



Figure 5.8: Geometry of the interference of the peak phases in the quantum Fourier transform. The limit imposed on the quantum Fourier transform induces the imperfect alignment of each phase or vector.

for the $\tilde{m}$'th qubit becomes (scaling implicit)

$$\sum_{i=k+1}^{\tilde{m}} \frac{s_{N'-1-(\tilde{m}-i)}}{2^{\tilde{m}+1}} = 2\pi(2^{-k-1} - 2^{-\tilde{m}-1}),$$

where the equality is obtained if all $s_{N'-1-(\tilde{m}-i)}$'s are 1 (the assumption is made to maximize the angle offset of the $\tilde{m}$'th qubit). If we now consider the integer space represented in qubits, the maximal phase-angle offset for an integer state reads,

$$\phi_{max} \equiv \left( \sum_{m=0}^{k} \{0\} + \sum_{m=k+1}^{N-1} \{2^{k-1} - 2^{-m-1}\} \right) = 2^{-k-1}(N-k) + \left(2^{-N} - 2^{-k}\right). \quad (5.5)$$

From equation (5.5), we can attempt to derive an analytical expression for the peak probability behavior shown in figure 5.5 in the following way. Since the way in which the peaks are formed in Shor's algorithm (after the operation of the quantum Fourier transform) is through interference of the phases of the integer states, i.e. via constructive interference of the phases, we can translate the process into a vector addition problem: If the phases, or unit vectors, are aligned so that they point in the same direction, then their sum will be 1 when normalized. A pictorial description of this process is shown in figure 5.8.

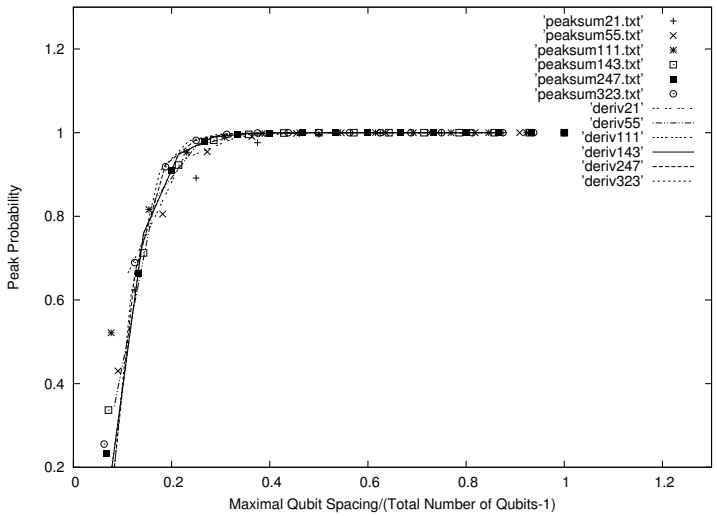



Figure 5.9: Analytical expression for the peak probability of Shor's algorithm vs. normalized maximal qubit spacing in the quantum Fourier transform. The lines represent the analytical expression we obtained in equation (5.6)

Using the geometry shown in figure 5.8, we can immediately translate equation (5.5) into a peak probability expression. Since equation (5.5) provides us with the maximal angle offset of the vector in figure 5.8, assuming we have $(i)$ a uniform probability distribution of the vector direction for the given maximal offset, and $(ii)$ independent events in determining the angle of each vector, we obtain

$$\begin{aligned}(Peak\ Probability) = \langle \cos\phi \rangle_{\phi_{max}} &= \frac{\sin\phi_{max}}{\phi_{max}} \\ &= \frac{\sin\left(2^{-k-1}(N-k) + \left(2^{-N} - 2^{-k}\right)\right)}{2^{-k-1}(N-k) + (2^{-N} - 2^{-k})}.\end{aligned} \tag{5.6}$$

To determine how well the obtained result in equation (5.6) fits with the raw data, we plot them together (see figure 5.9). We observe that equation (5.6) fits our data very well.

We note that the peak probability convergence to 1 occurs rapidly enough that the convergence of the peak probability of 95% is achieved on 300 qubits with a maximal qubit spacing of only 8 qubits, as shown in figure 5.10. Since the maximal spacing of 8

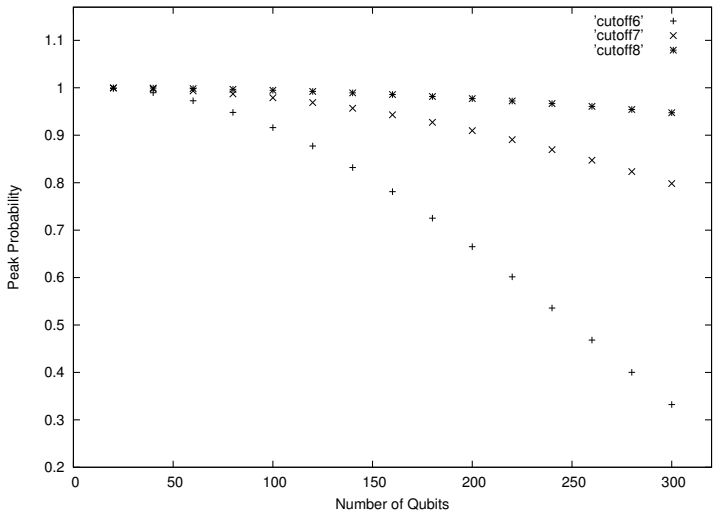



Figure 5.10: Extrapolated behavior of peak probability vs. number of qubits with various limits imposed on the quantum Fourier transform.

qubits corresponds to the minimal rotation angle of $\frac{2\pi}{2^9}$, the hardware requirement for performing the quantum Fourier transformation in Shor's algorithm now becomes a laser that can perform $\frac{2\pi}{2^9}$ phase rotation on a two-level atom, for instance.

### 5.4.3 Scaling

The execution time and the memory requirement can now be estimated analytically because the number of qubits determine the number of operations or computations to be conducted in a quantum Fourier transform and in the initialization process, and the order determines how many input values or equivalence class elements we have. Assuming we have $N$ qubits in register I, we have $N$ Hadamard operations and $N \times 2^N$ multiplications for generating the integer state array, in the initialization step. For a given order value, $\omega$, the number of input states, then, reduces to $\frac{2^N}{\omega}$ for a quantum Fourier transform. In a quantum Fourier transform, we have $N$ Hadamard operations and $\frac{N(N-1)}{2}$ single qubit rotation gates per input number. As we consecutively add the resultant phase values at the end of each quantum Fourier transform, this leads to

additional $2^N$ additions per input in a quantum Fourier transform.

The memory requirement is now straightforward to estimate. As we use double precision throughout the code, each stored number costs 8 bytes. Since it is the quantum Fourier transform that requires the most memory and the initialization is done on a separate program, we do not need to take the initialization step into account for the memory requirement. Now, we have $\frac{2^N}{\omega}$ input values to be imported, $2^N$ phases to be computed per input value, and another $2^N$ slots to be used for storing updated phases, as described above. Since the program performs a quantum Fourier transform on each input at a time, in terms of random access memory requirements, we do not need to count the number of input values. Hence, we end up with a minimal RAM requirement of $8 \times 2^{N+1}$ bytes.

If we assume that the computation time required for performing an addition is $A$, for a multiplication is $M$, for a Hadamard operation is $H$, and for a single-qubit rotation is $R$, then the total computational time required scales like

$$\{H \times N + M \times (N \times 2^N)\}_{init} + \frac{2^N}{\omega}\{H \times N + R \times \frac{N(N-1)}{2} + A \times 2^N\}_{QFT}.$$

Since $A$,$M$,$H$,and $R$ vary from computer to computer due to their different Central Processing Units or Bus speed and so on, they cannot be further specified. It has been observed, from the computer used to generate the probability spectra in this thesis, that the limiting number of qubits that can be simulated is about 17 to 20, given a maximally allowed computational time of a couple of days.

The obtained expression for the computational time helps us understand how inefficient a classical computer is in running even the periphery part of Shor's algorithm. For a quantum computer, such an expression is

$$\{H \times N\}_{init} + \{H \times N + R \times \frac{N(N-1)}{2}\}_{QFT}.$$

Even if we disregard simple multiplications or additions, due to the exponentially scaling number of elements in an equivalence class, the number of gates to be performed

on a classical computer necessarily exceeds exponential scaling. We truly observe, now in a mathematical sense, that it is the massive parallelism present in a quantum computer that results in polynomial scaling for running Shor's algorithm on a quantum computer.

# Chapter 6

# Periphery of Shor's Algorithm: Ideal $f$-mapping

In section 5.3.2, we discussed the physical limit in implementing a quantum fast Fourier transform. In this chapter, we study the effect of hardware imperfections in our experimental apparatus. We note that this is, again, still not a complete implementation of Shor's algorithm, as the $f$-mapping is still ideal, i.e. classical.

## 6.1 Introduction of Noise

From section 5.2 we recall how we initialize qubits and perform a quantum Fourier transform. The gates used for these steps are $(i)$ a Hadamard gate and $(ii)$ a single qubit rotation gate (see section 5.1). As both gates are rotation-based gates, we introduce noise to our gates by setting an error margin, $\delta$, in their rotational angles, $-\delta \leq \Delta\theta \leq \delta$. This is best simulated by using a random number generator.

A single qubit rotation gate, then, becomes,

$$\begin{pmatrix} 1 & 0 \\ 0 & e^{i(\theta+\Delta\theta)} \end{pmatrix}, \tag{6.1}$$

where $\theta$ is the rotation angle and $\Delta\theta$ is the noise as described above. A Hadamard gate can also be constructed in a similar way. With $\omega = \frac{\pi}{2}$ and $\epsilon = \frac{3\pi}{4}$, a Hadamard matrix becomes,

$$\mathcal{H} = e^{-i\sigma_z(\omega+\Delta\omega)}e^{-i\sigma_y(\epsilon+\Delta\epsilon)} = \begin{pmatrix} e^{-i(\omega+\Delta\omega)}\cos(\epsilon+\Delta\epsilon) & -e^{-i(\omega+\Delta\omega)}\sin(\epsilon+\Delta\epsilon) \\ e^{i(\omega+\Delta\omega)}\sin(\epsilon+\Delta\epsilon) & e^{i(\omega+\Delta\omega)}\cos(\epsilon+\Delta\epsilon) \end{pmatrix}. \tag{6.2}$$

## 6.2 Probability Spectrum

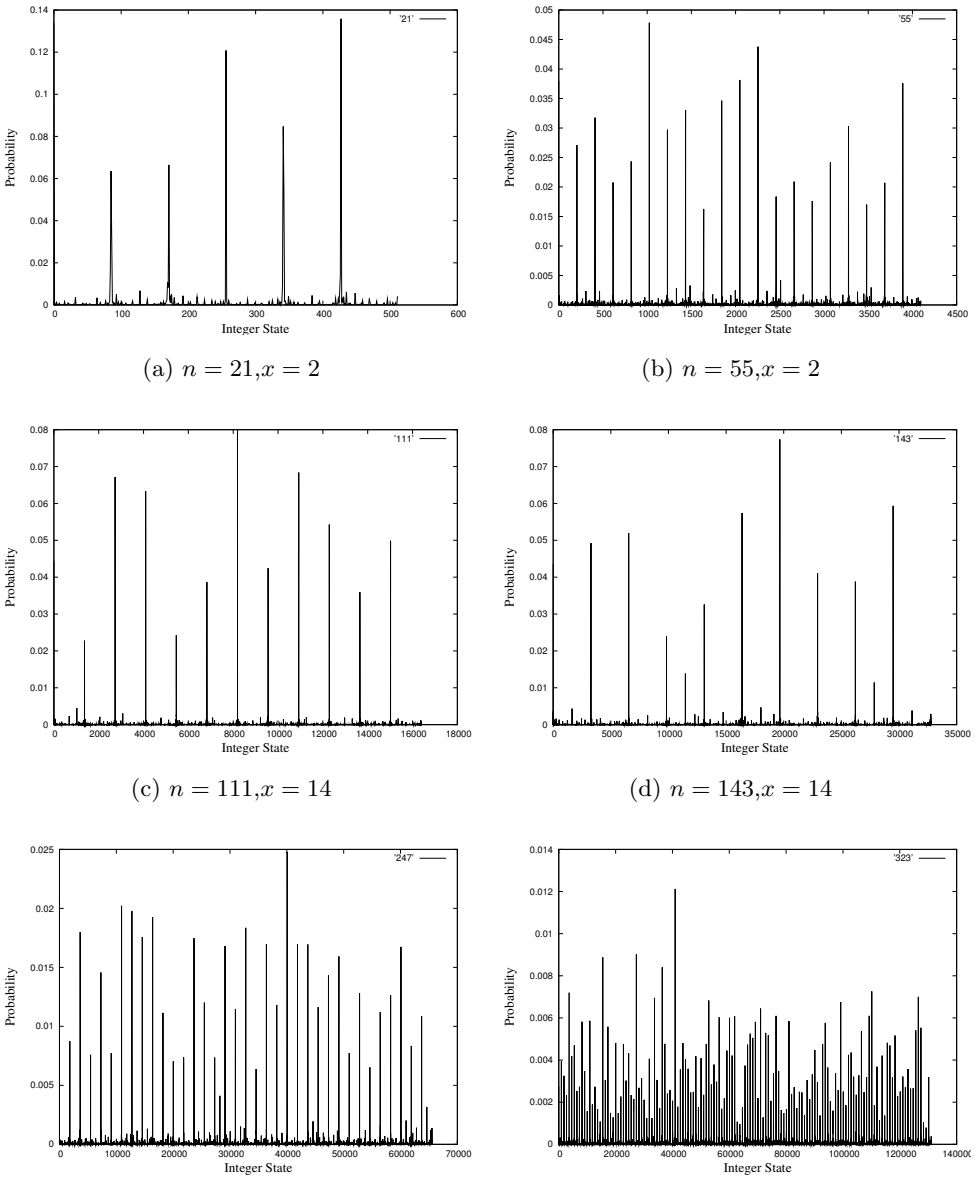
### 6.2.1 Influence of Noise

A set of probability spectra for measuring $\tilde{y}$ with a fixed noise level is shown in figure 6.1. For the purpose of isolating the effect of noise, we eliminated the limit of the quantum Fourier transform. We immediately observe the effect of noise in the probability spectra; in general, the introduced noise results in the following properties.

- The Dirac Comb-like peak structure fades away.
- The probability of landing on the correct peaks drops.
- Spurious peaks emerge.

In an attempt to study how the peak probability drops as the noise level varies, we simulate the peripheral Shor's algorithm with a variety of noise margins, where no limit in the quantum Fourier transform is introduced. The results obtained are shown in figure 6.2.

From figure 6.2, we see that the effect of noise follows a "stretched" Gaussian distribution function: $e^{-\frac{x^\beta}{\sigma}}$, where $\sigma$ is the standard deviation and $\beta$ ranges from 1 to 2. The




(a) $n = 21$,$x = 2$

(b) $n = 55$,$x = 2$

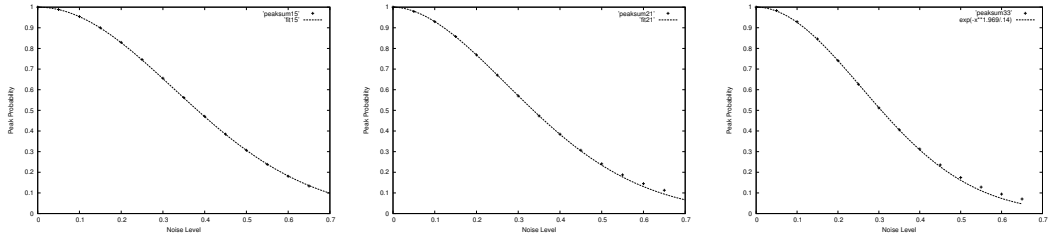
(c) $n = 111$,$x = 14$

(d) $n = 143$,$x = 14$

(e) $n = 247$,$x = 6$

(f) $n = 323$,$x = 5$

Figure 6.1: Probability spectra of Shor's algorithm via the gate-by-gate implementation of the peripheral Shor's algorithm with noise. No limit in the quantum Fourier transform is assumed for the purpose of isolating the influence of the noise.




(a) $n = 15$,$x = 2$ (b) $n = 21$,$x = 2$ (c) $n = 33$,$x = 5$

Figure 6.2: Peak probability of peripheral Shor's algorithm vs. noise level. The fitted functions are "stretched" Gaussians; $e^{-\frac{x^\beta}{\sigma}}$. $\beta$ values are 2, 1.86, and 1.97, and $\sigma$ values are 0.21, 0.19, and 0.14 for the cases 15, 21, and 33, respectively. Notice the fit is not perfect: The tail of the curve slightly deviates from the data.

tail of the curve slightly deviates from the curve, but the region of interest is towards higher peak probability. In order to investigate how the influence of noise scales as we increase the size of the system or the number of qubits, we compute the peak probability for various semi-prime numbers with fixed noise and no limit in the quantum Fourier transform. The result is shown in figure 6.3. Provided with the fact that, in the quantum Fourier transform, the minimum angle of rotation that the single qubit rotation gate shifts, exponentially decreases as the number of qubits increases (see subsection 5.2.2), the slight trend of decaying probability, given the fixed noise level, for increasing number of qubits (see figure 6.3), is understood.

### 6.2.2 Effect of the Combination of a Quantum Fourier Transform Limit and the Noise

As we now have a complete picture of the periphery of Shor's algorithm, we finally introduce a fully realistic condition, i.e. the combination of a limit on the quantum Fourier transform and the noise. Since, in subsections 5.4.2 and 6.2.1, we have already shown how the limit of the quantum Fourier transform and the noise affect the probability distribution, we study their combined influence in this subsection.

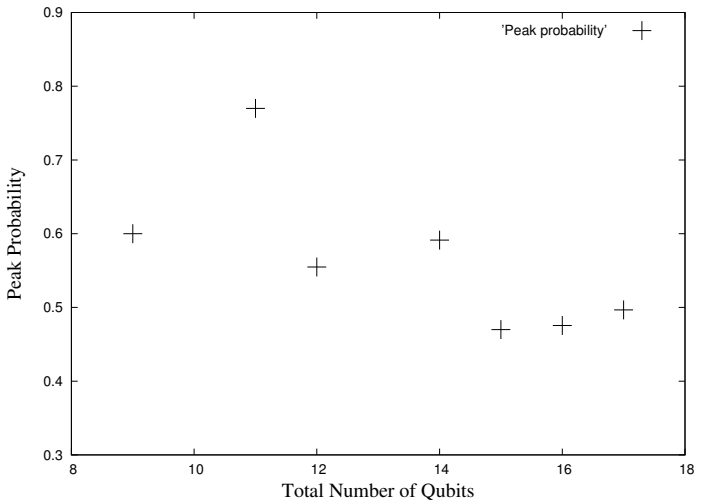



Figure 6.3: Peak probability of peripheral Shor's algorithm with fixed noise vs. size of the system (number of qubits). The introduced noise level is .2 radian in the Hadamard gates and .25 radian in the single-qubit rotation gates. Notice the plot does not exhibit any strong scaling behavior.

Using the peak probability method established to capture the influence of the limit of the quantum Fourier transform (see subsection 5.4.2), we investigate the effect of the combination of a quantum Fourier transform limit and the noise. How the limiting angle for the quantum Fourier transform, given a fixed level of noise, influences the peak probability is shown in figure 6.4.

We remark that the noise introduced in our simulation results in the fluctuation in the peak probability for the varying limiting angles in the quantum Fourier transform, as is clearly demonstrated in figure 6.4. This is so, because, if the noise happens to compensate for the lost rotation angle due to the limiting angle in the quantum Fourier transform, we obtain higher peak probability. Similarly, if the noise deepens the effect of the lost angle in the quantum Fourier transform, we obtain lower peak probability.

It is noteworthy to mention how robust a quantum computer is against such realistic noise conditions. For instance, the error margin, $\delta$, used to produce figure 6.4 is more

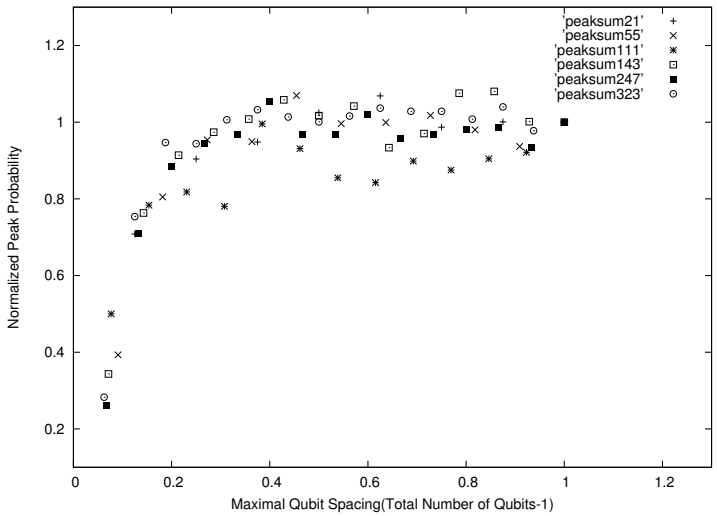



Figure 6.4: Normalized peak probability of the peripheral Shor's algorithm with noise vs. normalized maximal qubit spacing in the quantum Fourier transform. Peak probability on the $y$-axis is normalized with the peak probability without a limit in the quantum Fourier transform. The cumulative probability of the correct peaks may be larger than 1 if noise compensates errors introduced by the limit imposed on the quantum Fourier transform. Due to normalization imposed on both $x$ and $y$ axes, the plot necessarily converges to $(1, 1)$.

than an eighth of the ideal angle in the Hadamard gates and a half or even more than the ideal angle itself in single-qubit rotation gates. We make a special note that a quantum computer is immune to hardware imperfections present in the single-qubit rotation gates in the quantum Fourier transform when running Shor's algorithm. This is so, because (*i*) the single-qubit rotation gates affect only the phases of qubits, not the probabilities, (*ii*) the hardware imperfections in the single-qubit rotation gates affects the manipulated qubits in a parallel way, due to the massive parallelism that exists in a quantum computer, (*iii*) the quantum Fourier transform is the last operation in Shor's algorithm and, therefore, the parallel phase shift does not affect the final probability reading when measurements are performed.

# Chapter 7

# Gate-by-Gate construction of $f$-mapping

In Chapters 5 and 6, we have assumed pre-knowledge of the equivalence-class elements in order to isolate and investigate the periphery of Shor's algorithm. Such an equivalence class was, in fact, obtained through the classical $f$-mapping, using the classical knowledge of the order, $\omega$. For this reason, the simulation was not a full implementation of a quantum computer.

In this Chapter, we introduce a full, gate-by-gate construction of the quantum $f$-mapping. As the $f$-mapping is the last piece that needs to be put in place for a complete implementation of Shor's algorithm on a quantum computer, the gate-by-gate construction will lead us to a full implementation of a universal quantum computer running Shor's algorithm on a classical computer.

The implementation of the quantum $f$-mapping used in this Chapter closely follows the method introduced by Vedral et al.[40]. To explain how the method works, we introduce the methodology of the implementation in the first section. Since the method requires certain gates as basic building blocks of a quantum $f$-mapping circuit, we introduce

such quantum gates in the following section. Using the methodology as a guide and the basic quantum gates as building blocks, we will present the construction mechanism of a quantum $f$-mapping in the last section of the Chapter.

It is important to notice that we will not implement the ideal quantum $f$-mapping on a classical computer. As we will see later in this Chapter, the number of qubits that need to be implemented in order to simulate the $f$-mapping is at least seven times the minimum number of qubits required to load the semi-prime number to be factored, $n$. If the required qubit number is 3, for instance, we already have at least 21 qubits to simulate, which exceeds the practical maximal number of qubits that can be simulated, as we have shown in Chapter 5. However, three qubits are not even enough to load $15 = 3 \times 5$, and, therefore, the implementation of the ideal $f$-mapping on a classical computer would be pointless. The resolution of this issue will be presented in Chapter 8.

## 7.1 Methodology

From section 3.3, we recall the role of the $f$-mapping (i.e. an exponentiation-modulo or EXP-MOD operation):

$$f(r) = x^r \bmod n,$$

where $x$ is the seed, $r$ is the integer state in register I, and $n$ is the semi-prime number to be factored. Using the fact that $r$ may be written in binary form as

$$r = 2^0 r_0 + 2^1 r_1 + ... + 2^{N'-1} r_{N'-1},$$

where $N'$ is the number of qubits in the register I and $r_j \in \{0, 1\}$, we obtain the following expression

$$f(r) = x^{2^0 r_0 + 2^1 r_1 + ... + 2^{N'-1} r_{N'-1}} \bmod n. \tag{7.1}$$

From equation (7.1), we immediately notice that $f(r)$ can now be decomposed into multiplications of $x^{2^j r_j}$, where modulo $n$ is to be executed at each multiplication step.

This can be mathematically written as

$$f(r) = (...((1 \times x^{2^0 r_0}) \bmod n \times x^{2^1 r_1}) \bmod n \times ... \times x^{2^{N'-1} r_{N'-1}}) \bmod n. \tag{7.2}$$

Hence, by inspecting equation (7.2), the quantum gate of interest to be used in order to construct the $f$-mapping, or an EXP-MOD gate, now becomes a controlled-multiplier-modulo or CTRL-MULTI-MOD gate, which performs

$$(Number) \rightarrow \left(Number \times \left(x^{2^j}\right)^{r_j}\right) \bmod n,$$

where the value $r_j$ is to be taken from the $j$'th qubit in register I as a control-qubit state.

The construction mechanism of a CTRL-MULTI-MOD gate can now be found in an analogous way: As the exponentials can be decomposed into repeated multiplications, the multiplication can be decomposed into many additions. This time, we use the following equality:

$$x^{2^j r_j} = \left(\left(x^{2^j}\right)_0 + \left(x^{2^j}\right)_1 + ... + \left(x^{2^j}\right)_{N''-1}\right)^{r_j},$$

where $N''$ is the number of qubits in register II. Since $r_j$ acts as a conditional operation switch, i.e. $r_j = 0$ leads to the multiplication of 1 and $r_j = 1$ leads to the multiplication of $x^{2^j}$, we can now decompose a CTRL-MULT-MOD gate (with $\alpha = x^{2^j}$ for a cleaner notation), as

$$\begin{aligned} &\# \times \alpha^{r_j} \bmod n \\ &= \left(... \left(\#_0\{\alpha_0\}^1 \bmod n + \#_1\{\alpha_1\}^1\right) \bmod n + ... + \#_{N''-1}\{\alpha_{N''-1}\}^1\right) \bmod n \ (if\ r_j = 1) \\ &= \#\ (if\ r_j = 0). \end{aligned} \tag{7.3}$$

By inspecting equation (7.3), we notice a CTRL-MULTI-MOD gate can be further decomposed with gates that perform

$$(Number) \rightarrow (Number + \alpha_k) \bmod n, \tag{7.4}$$

where $\alpha_k$ is a classically known value, as it is determined from the seed value, $x$, and the control-qubit number $j$ in register I. We call such a gate an addition-modulo or ADD-MOD gate.

It is now clear how to construct the quantum $f$-mapping. First, we construct a quantum gate that performs an ADD-MOD operation. Then, we follow the methodology developed above in the reverse direction to obtain an EXP-MOD gate.

## 7.2 Building Blocks

In this section we introduce a series of gates that are needed to construct an ADD-MOD gate and ultimately the $f$-mapping. Since we would like to use Pauli-matrix convention, consistent with Chapter 5, the construction mechanism of the gates can be directly translated to the action of a laser beam on a two-level atom.

### 7.2.1 NOT gate

A NOT gate makes the following transition: $|0\rangle \leftrightarrow |1\rangle$. A matrix form of such a gate reads

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

This can be generated by using a Pauli-$x$ matrix. Up to a global phase of $i$,

$$e^{-i\frac{\pi}{2}\sigma_x} = \begin{pmatrix} \cos\left(\frac{\pi}{2}\right) & i\sin\left(\frac{\pi}{2}\right) \\ i\sin\left(\frac{\pi}{2}\right) & \cos\left(\frac{\pi}{2}\right) \end{pmatrix}. \tag{7.5}$$

As in the case of the Hadamard gate (see subsection 5.1.2), the global phase $i$ is the global phase of a single qubit, not of the entire system. If a gate operation involves multiple qubits at a time, the single-qubit global phase cannot be ignored. For the purpose of the $f$-mapping, however, this issue does not arise, and, hence, we will use equation (7.5) as our NOT operation in the $f$-mapping. A NOT gate representation in



Figure 7.1: Circuit representation of the NOT gate.

a circuit diagram is shown in figure 7.1.

### 7.2.2 CNOT gate

The way in which a CNOT gate is realized with a two-level atom is demonstrated in Monroe et al. [41] and Schmidt-Kaler et al. [42]. We closely follow their work to present the construction of a CNOT gate in this subsection.

A CNOT gate performs the following transitions:

$$\begin{aligned}
|0\rangle_{control}|0\rangle_{target} &\rightarrow |0\rangle_{control}|0\rangle_{target}, \\
|0\rangle_{control}|1\rangle_{target} &\rightarrow |0\rangle_{control}|1\rangle_{target}, \\
|1\rangle_{control}|0\rangle_{target} &\rightarrow |1\rangle_{control}|1\rangle_{target}, \\
|1\rangle_{control}|1\rangle_{target} &\rightarrow |1\rangle_{control}|0\rangle_{target}.
\end{aligned} \tag{7.6}$$

We note that a CNOT gate is equivalent to an XOR gate in a classical circuit.

Since a CNOT gate is a two-qubit gate, we have an additional quantum state from their vibrational modes, or phonon modes. As it will be shown shortly, it is the phonon that will be used as the bus between two qubits, which enables the realization.

In order to physically realize the CNOT gate, first, we initialize our phonon to be in the state $|0\rangle_{phonon}$, i.e. the ground state, by ensuring that the temperature is low enough to not excite the vibrational mode of the qubits. Then, we make a transition between $|1\rangle_{control}|0\rangle_{phonon}$ and $|0\rangle_{control}|1\rangle_{phonon}$ with the help of a sideband beam, which is equivalent to a Pauli-$x$ pulse between the two states resulting in a $\frac{\pi}{2}$ rotation up to a global phase of $i$. Such a global phase may be ignored due to its cancellation with another global phase, which will be shown later. Since the control qubit state is now

copied to the phonon state, without the global phase $i$,

$$|0\rangle_{control}|0\rangle_{phonon} \rightarrow |0\rangle_{control}|0\rangle_{phonon},$$
$$|1\rangle_{control}|0\rangle_{phonon} \rightarrow |0\rangle_{control}|1\rangle_{phonon},$$

we can use the phonon state, which is shared between the control and the target qubit, as the bus state, and the target qubit state to perform the controlled-NOT operation.

To perform the bus mode CNOT, we first apply a $\frac{\pi}{4}$ pulse with a Pauli-$y$ matrix. The transitions made from this operation are

$$|0\rangle_{phonon}|0\rangle_{target} \rightarrow \frac{|0\rangle_{phonon}|0\rangle_{target} + |0\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}},$$
$$|0\rangle_{phonon}|1\rangle_{target} \rightarrow \frac{-|0\rangle_{phonon}|0\rangle_{target} + |0\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}},$$
$$|1\rangle_{phonon}|0\rangle_{target} \rightarrow \frac{|1\rangle_{phonon}|0\rangle_{target} + |1\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}},$$
$$|1\rangle_{phonon}|1\rangle_{target} \rightarrow \frac{-|1\rangle_{phonon}|0\rangle_{target} + |1\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}}.$$

We know the desired result is exchanging $|1\rangle_{phonon}|0\rangle_{target}$ and $|1\rangle_{phonon}|1\rangle_{target}$. We immediately recognize that this can be achieved by a $2\pi$ Rabi rotation of the phase of the state $|1\rangle_{phonon}|0\rangle_{target}$ and a subsequent Pauli-$y$ pulse, resulting in a $-\frac{\pi}{4}$ rotation:

$$\begin{aligned}
|0\rangle_{phonon}|0\rangle_{target} &\rightarrow \frac{|0\rangle_{phonon}|0\rangle_{target} + |0\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \\
|0\rangle_{phonon}|1\rangle_{target} &\rightarrow \frac{-|0\rangle_{phonon}|0\rangle_{target} + |0\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \\
|1\rangle_{phonon}|0\rangle_{target} &\rightarrow \frac{|1\rangle_{phonon}|0\rangle_{target} + |1\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \\
|1\rangle_{phonon}|1\rangle_{target} &\rightarrow \frac{-|1\rangle_{phonon}|0\rangle_{target} + |1\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \\
&\rightarrow \frac{|0\rangle_{phonon}|0\rangle_{target} + |0\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \rightarrow |0\rangle_{phonon}|0\rangle_{target}, \\
&\rightarrow \frac{-|0\rangle_{phonon}|0\rangle_{target} + |0\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \rightarrow |0\rangle_{phonon}|1\rangle_{target}, \\
&\rightarrow \frac{-|1\rangle_{phonon}|0\rangle_{target} + |1\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \rightarrow |1\rangle_{phonon}|1\rangle_{target}, \\
&\rightarrow \frac{|1\rangle_{phonon}|0\rangle_{target} + |1\rangle_{phonon}|1\rangle_{target}}{\sqrt{2}} \rightarrow |1\rangle_{phonon}|0\rangle_{target}.
\end{aligned}$$

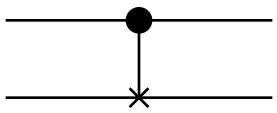


Figure 7.2: Circuit representation of the CNOT gate.

By applying a $-\frac{\pi}{2}$ sideband beam, which restores the global phase to 1,

$$|0\rangle_{control}|0\rangle_{phonon} \rightarrow |0\rangle_{control}|0\rangle_{phonon},$$
$$|0\rangle_{control}|1\rangle_{phonon} \rightarrow |1\rangle_{control}|0\rangle_{phonon},$$

we finally obtain a full CNOT operation as shown in equation (7.6). A CNOT gate representation as a circuit diagram is shown in figure 7.2.

### 7.2.3 CCNOT Gate

A CCNOT gate performs the following transitions:

$$|0\rangle_{control1}|0\rangle_{control2}|0\rangle_{target} \rightarrow |0\rangle_{control1}|0\rangle_{control2}|0\rangle_{target},$$
$$|0\rangle_{control1}|0\rangle_{control2}|1\rangle_{target} \rightarrow |0\rangle_{control1}|0\rangle_{control2}|1\rangle_{target},$$
$$|0\rangle_{control1}|1\rangle_{control2}|0\rangle_{target} \rightarrow |0\rangle_{control1}|1\rangle_{control2}|0\rangle_{target},$$
$$|0\rangle_{control1}|1\rangle_{control2}|1\rangle_{target} \rightarrow |0\rangle_{control1}|1\rangle_{control2}|1\rangle_{target},$$
$$|1\rangle_{control1}|0\rangle_{control2}|0\rangle_{target} \rightarrow |1\rangle_{control1}|0\rangle_{control2}|1\rangle_{target},$$
$$|1\rangle_{control1}|0\rangle_{control2}|1\rangle_{target} \rightarrow |1\rangle_{control1}|0\rangle_{control2}|0\rangle_{target}.$$
$$|1\rangle_{control1}|1\rangle_{control2}|0\rangle_{target} \rightarrow |1\rangle_{control1}|1\rangle_{control2}|1\rangle_{target},$$
$$|1\rangle_{control1}|1\rangle_{control2}|1\rangle_{target} \rightarrow |1\rangle_{control1}|1\rangle_{control2}|0\rangle_{target}.$$

The implementation of such a gate is shown in the circuit diagram in figure 7.3. As we know how to implement all three quantum gates used in the circuit (see subsections 5.1.1, 5.1.2, and 7.2.2), the implementation of a CCNOT gate is known.

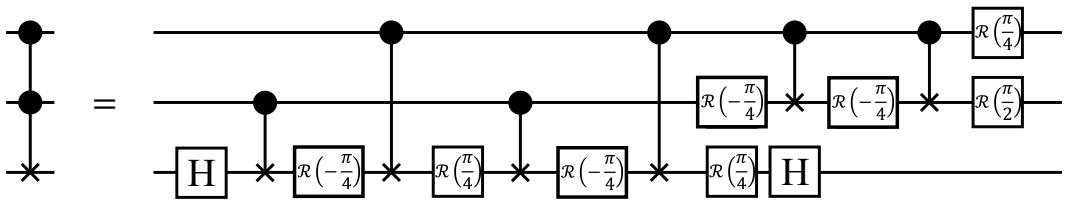



Figure 7.3: Logic circuit of the CCNOT gate. The diagram is inspired by the book of Nielsen and Chuang [3]. The circuit diagram shows how to construct a CCNOT gate using the single-qubit rotation gates, the Hadamard gates and the CNOT gates.

### 7.2.4 Block Diagram of the SUM Gate

A SUM gate performs the following transitions:

$$|0\rangle_{control1}|0\rangle_{control2}|0\rangle_{target} \rightarrow |0\rangle_{control1}|0\rangle_{control2}|0\rangle_{target},$$
$$|0\rangle_{control1}|0\rangle_{control2}|1\rangle_{target} \rightarrow |0\rangle_{control1}|0\rangle_{control2}|1\rangle_{target},$$
$$|0\rangle_{control1}|1\rangle_{control2}|0\rangle_{target} \rightarrow |0\rangle_{control1}|1\rangle_{control2}|1\rangle_{target},$$
$$|0\rangle_{control1}|1\rangle_{control2}|1\rangle_{target} \rightarrow |0\rangle_{control1}|1\rangle_{control2}|0\rangle_{target},$$
$$|1\rangle_{control1}|0\rangle_{control2}|0\rangle_{target} \rightarrow |1\rangle_{control1}|0\rangle_{control2}|0\rangle_{target},$$
$$|1\rangle_{control1}|0\rangle_{control2}|1\rangle_{target} \rightarrow |1\rangle_{control1}|0\rangle_{control2}|1\rangle_{target}.$$
$$|1\rangle_{control1}|1\rangle_{control2}|0\rangle_{target} \rightarrow |1\rangle_{control1}|1\rangle_{control2}|1\rangle_{target},$$
$$|1\rangle_{control1}|1\rangle_{control2}|1\rangle_{target} \rightarrow |1\rangle_{control1}|1\rangle_{control2}|0\rangle_{target}.$$

It is called a SUM gate because it performs the summation of three qubits modulo 2.

The circuit diagram in figure 7.4 shows how to implement a SUM gate. We note that the SUM gate involves three qubits and consists of two CNOT gates, which were already studied in subsection 7.2.2. The position of a thick black bar is used to denote the order of the CNOT operations, i.e. if positioned to the left, we operate in reversed order.

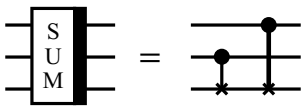



Figure 7.4: Logic circuit of the SUM gate. The diagram is inspired by Vedral et al. [40]. The implementation of the SUM gate using the CNOT gates is shown. The thick black bar on the right is to indicate the sequence of the implementation. If the bar is located on the right, the operations run in the forward direction, i.e. first, the CNOT operation between the second and the third qubit, then another CNOT operation between the first and the third qubit. If the bar is located on the left, the operation is performed in the reverse order, i.e. first, the CNOT operation between the first and the third qubit, then another CNOT operation between the second and the third qubit.

### 7.2.5 Block Diagram of the CARRY Gate

The CARRY gate is used to compute an arithmetic carry value resulting from the addition of two qubit states. Since there may be a carry generated by the previous operation, we have a total of four qubits, including the storage qubit of a new carry, involved in the CARRY gate. The circuit diagram of a CARRY gate is shown in figure 7.5. Since the constituent gates such as the CNOT gate and the CCNOT gate have already been investigated in subsection 7.2.2 and 7.2.3, the realization is straightforward. The role of the thick bar is the same as described in subsection 7.2.4 and assumed to be the same throughout this Chapter.

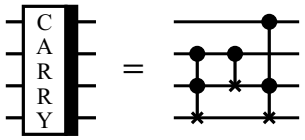



Figure 7.5: Logic circuit of the CARRY gate. The diagram is inspired by Vedral et al. [40]. The role of the thick black bar is the same as in the case of the SUM gate (see figure 7.4).

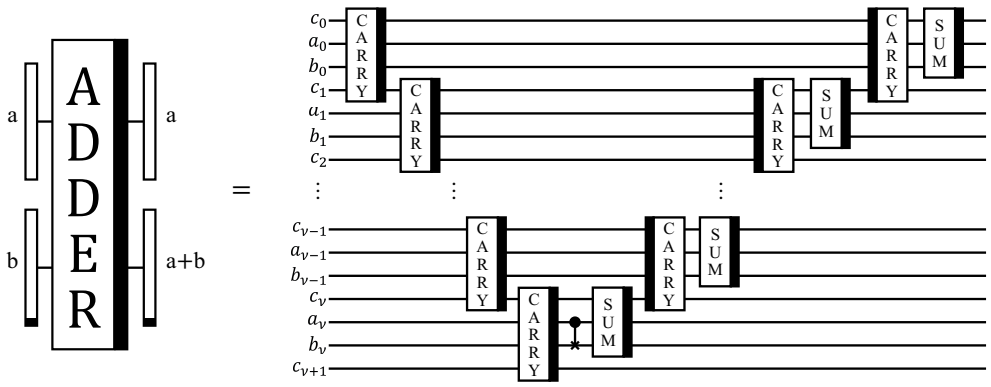



Figure 7.6: Logic circuit of the ADDER gate. The diagram is inspired by Vedral et al. [40]. An auxiliary qubit array $c$ is used to store arithmetic carry values. Notice that the most significant carry value (the overflow information) is stored in $c_{\nu+1}$, the most significant (highest) carry qubit.

### 7.2.6 Block Diagram of the ADDER Gate

In this subsection, we introduce a plain adder, or ADDER, gate. As the name indicates, the gate performs a simple addition between two numbers embedded in two arrays of qubits. The circuit representation of the gate in figure 7.6 shows the input and the output states of the ADDER network. We note that the ADDER gate requires three qubit arrays of the same size and an extra qubit.

The addition is done in roughly three steps: $(i)$ we produce the arithmetic carry value, $c_{j+1}$, between the carry value transferred over from the previous CARRY operation, $c_j$, and the two qubits with the same subindex from the two different numbers to be added (i.e. $a_j$ and $b_j$), where $0 \leq j \leq \nu$, to ultimately obtain the most significant carry value, $c_{\nu+1}$, $(ii)$ we undo the CARRY operation performed in $(i)$ by applying the reverse-ordered CARRY gate to reset the produced arithmetic carry value $c_{\tilde{j}+1}$, where $0 \leq \tilde{j} \leq \nu - 1$, $(iii)$ we apply the SUM gate at the end of each reverse CARRY operation to obtain the summation of the not-yet-reset carry value $c_{\tilde{j}}$ and two qubits of the same order from the two numbers, $a_j$ and $b_j$, and store the summed value in $b_j$.

As the most significant carry value, $c_{\nu+1}$, is not to be undone, we use a CNOT gate to restore the affected $b_\nu$ state from the CARRY operation. Since the network makes use of the CARRY gate, the SUM gate, and the CNOT gate only, the implementation of the ADDER gate is straightforward. We notice that the additional qubit, $c_{\nu+1}$, in the ADDER network provides us with a means to check the overflow produced in the addition process.

We note that the reverse operation of the ADDER gate results in a subtractor gate. When the minuend is larger or equal to the subtrahend, the subtractor gate returns the subtrahend and the difference, $d$, and the overflow qubit is in $|0\rangle$. On the other hand, when the minuend is smaller than the subtrahend, the subtractor gate returns the subtrahend and the difference of $2^{\nu+2}$ and $d$, meaning the overflow qubit will be in $|1\rangle$. This particular observation, then, will be used in constructing the ADD-MOD gate.

## 7.3 Construction of the $f$-mapping operation

### 7.3.1 Block Diagram of the ADD-MOD Gate

Using the ADD gates as building blocks, we now construct the ADD-MOD gate. Since we want to take the modulo $n$, we introduce another qubit array to load the number $n$. As the modulo $n$ results in two different outcomes, depending on whether the addition of two numbers is smaller than $n$ or not, we introduce one additional qubit that can differentiate the two different cases. Including the three qubit arrays and one extra qubit from the ADDER gate, the ADD-MOD gate, then, requires two extra qubits and four qubit arrays whose size is at least $N$, where $N$ is the minimum number of qubits required to represent $n$. The circuit diagram of the ADD-MOD gate is shown in figure 7.7.

The mechanism to differentiate the two different cases can be obtained by using the

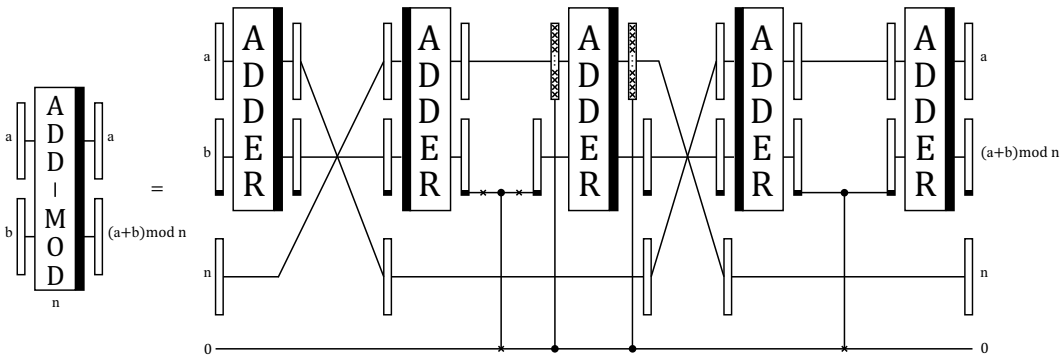



Figure 7.7: Logic circuit of the ADD-MOD gate. The diagram is inspired by Vedral et al. [40]. An auxiliary qubit array is used to load the semi-prime number $n$, which is indicated below the circuit representation of the ADD-MOD gate (see left). An additional qubit is used to import the information of the most significant carry qubit that results from the ADDER gates.

overflow qubit discussed in section 7.2.6. Since the addition of the two numbers embedded in the ADD-MOD input state becomes the minuend, and $n$ is the subtrahend of the second ADDER gate, depending on the sizes of the minuend and the subtrahend, the resultant overflow qubit state differs. We, then, use this difference on the third ADDER gate, in order to obtain the ADD-MOD operational result shown in expression (7.4). The last two ADDER gates in figure 7.7 are used to unitarily reset the additional qubit state to $|0\rangle$.

### 7.3.2 Block Diagram of the CTRL-MULTI-MOD Gate

Using section 7.1 as a guide, we construct the CTRL-MULTI-MOD gate in a straightforward manner as shown in figure 7.8. Since we would like to consecutively add values if $r_j$ is 1, we let the qubit $r_j$ in the register I to be the control qubit. As we have the number to be multiplied loaded in the register II, all we need is a conditional circuit on $r_j$ and $\#_k$ that results in the consecutive addition of $\alpha_k$'s, where $k$ ranges from 0 to $N'' - 1$, as shown in equation (7.3). Such an $\alpha_k$ value is a classically well-determined value, so we

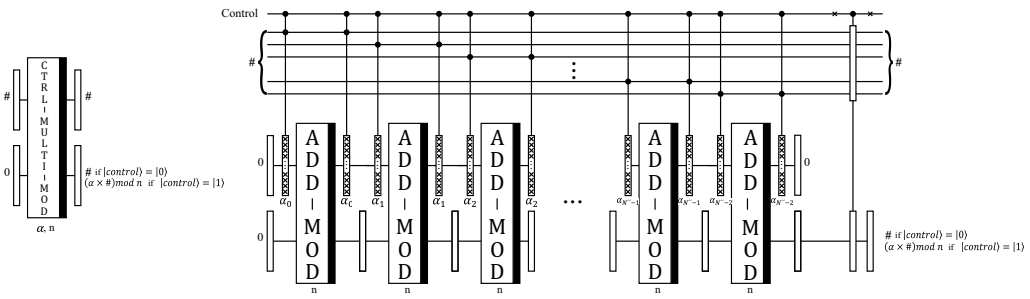



Figure 7.8: Logic circuit of the CTRL-MULTI-MOD gate. The diagram is inspired by Vedral et al. [40]. An auxiliary qubit array is used to load and remove $\alpha_k$, as indicated in the diagram.

can use the CCNOT gates to conditionally implement the $\alpha_k$ value on the input state of the ADD-MOD gate. With $N''$ consecutive operations of the ADD-MOD gates, in association with the conditional implementation of the $\alpha_k$'s, we obtain the desired result for the case $r_j = 1$. As we are to obtain # when $r_j = 0$, we make use of the NOT gates and the CCNOT gates to perform the conditional copying at the end to implement the CTRL-MULTI-MOD gate. We note that such a circuitry requires five qubit arrays of minimum size $N$, where we ignored the number of additional qubits.

### 7.3.3 Block Diagram of the EXP-MOD Gate

As we have the implementation of the CTRL-MULTI-MOD gate (see subsection 7.3.2), we can now construct the EXP-MOD gate to realize the $f$-mapping. Following the methodology described in section 7.1, we consecutively apply the CTRL-MULTI-MOD gate $N'$ times with the control qubit of the CTRL-MULTI-MOD gate running from the 0'th qubit to the $N' - 1$'st qubit in register I, as shown in figure 7.9. If we let $N'$ be at least $2N$, as is shown in Chapter 4, the minimum number of qubits required to implement the EXP-MOD Gate is now $7N$.

Since the result of a CTRL-MULTI-MOD gate is stored in a different array of qubits from the input states, one can attempt to unitarily restore the output state into the original input-qubit array, while the output qubit is re-initialized in order to recycle

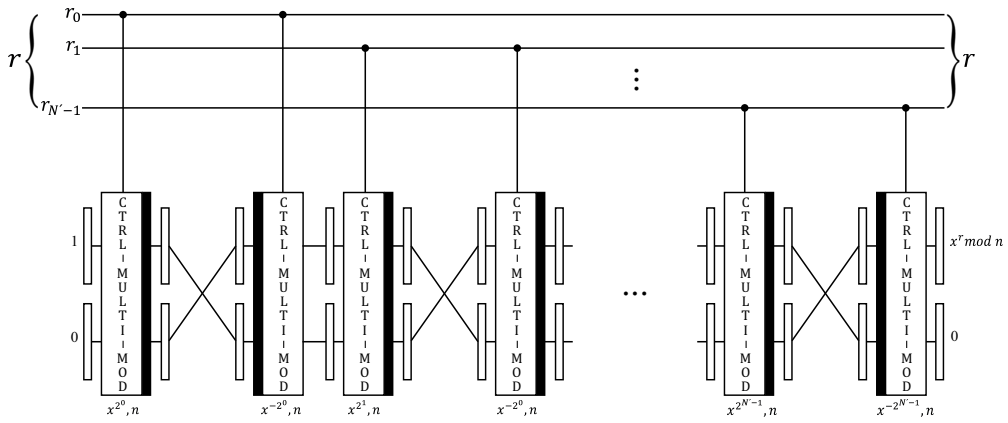



Figure 7.9: Logic circuit of the EXP-MOD gate. The diagram is inspired by Vedral et al. [40]. An auxiliary qubit array is used to carry over intermediate computational results. Notice the circuit requires the implementation of the $x^{-2^j}$'s, which are real numbers.

the qubits. The way in which this restoration is done in Vedral et al. [40], however, is wrong due to the following reason: a rational number $x^{-2^j}$ cannot be implemented on an integer state represented by qubits. Formally, the restoration mechanism can still be saved by using an integer $\xi_j$ instead of $x^{-2^j}$ if $\xi_j$ satisfies

$$x^2 \times \xi_j = 1 \bmod n. \tag{7.7}$$

The usefulness of the formally correct method can now be discussed.

- If $\xi_j$ is not easily determined algebraically, the method is useless.
- If $\xi_j$ is trivially determined algebraically, the method is useful.

The answer to this question is presently unknown. However, we consider the following case to show that it is likely that $\xi_j$ is hard to obtain. Suppose the $\xi_j$'s are easily determined. Then, knowing that $x^\omega$ is 1 mod $N$, we immediately notice

$$x^\omega = (integer) \times x^{2^j} \xi_j.$$

This, in fact, means $x^\omega$ can be easily determined if $\xi_j$ can be easily found. Therefore, finding the value of $\xi_j$ is not likely to be trivial. If this is the case, the restoration

mechanism may not be saved.

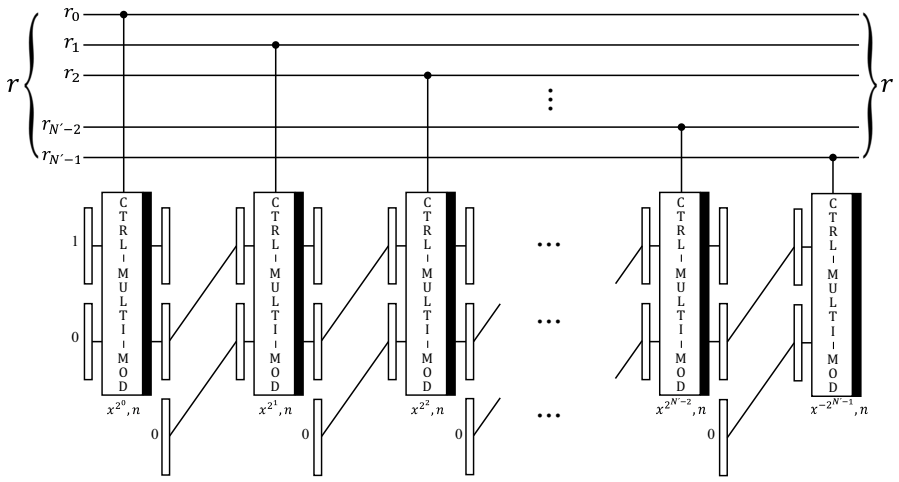



Figure 7.10: Logic circuit of the modified EXP-MOD gate. By employing multiple auxiliary qubit arrays, we avoid the implementation of the $\xi_j$'s, which may be hard to obtain.

In an attempt to save the mechanism, we use a modified EXP-MOD gate as shown in figure 7.10. We resolve the issue by simply choosing not to recycle the used qubits but to feed new qubits that are initialized to $|0\rangle$'s. We note that this method can be very costly due to the large number of qubits required.

# Chapter 8

# Complete Implementation of Shor's Algorithm: Non-unitary Approach

In Chapter 7 we discussed why we cannot implement the ideal quantum $f$-mapping: the minimum number of states to be simulated is at least $2^{7N}$ where $N$ is the minimum number of qubits necessary to represent the semi-prime number $n$. For example, the absolute minimum number of qubits required for factoring $n = 15$ is $N = 4$. This means that a complete implementation of Shor's algorithm, including the $f$-mapping, requires a minimum of $7 \times 4 = 28$ qubits, far exceeding the maximal number of qubits (around 21) that can be simulated on a currently available machine, a 32-bit PC. To cope with the limited resource we currently have, in this chapter, we present a method that allows us to simulate the $f$-mapping on a classical computer by means of breaking the unitarity, which will lead us to a complete, non-unitary implementation of Shor's algorithm.

The protocol that we will follow to simulate the $f$-mapping is the following.

- We do not introduce any "illegal" paths: Only the elimination of possible outcomes

will be employed.

- There is no probability leakage out of computational states: Non-computational bases, or atomic levels, will be ideal.
- Auxiliary qubits or phonons will always return to the ideal states, at least by the end of the non-unitary gates: We reduce our system of interest to computational qubits at the non-unitary gates level.
- Qubit states of register I are to be unaffected after the initialization step: Register I is to be treated ideal in the $f$-mapping process.
- The ideal path is to be carried out at all times: We focus on the probability that flows in and out of the ideal path.

We note that there is a potential probability loss encoded in our non-unitary method. This is so, because $(i)$ the auxiliary qubits may not return to their ideal states, $(ii)$ register I qubits may not return to their supposed states. Since none of the illegal paths are to be incorporated, the method we use to break the unitarity of our system will provide us with the minimal probability of measuring each state in register II.

By limiting our system to simulate register II only, the number of states to be explicitly simulated now reduces to $2^{N''}$, where $N''(\geq N)$ is the number of qubits in register II. Even though the auxiliary qubits, phonons, and register I qubits are included in our simulation, by choosing the ideal paths for such qubits, we dramatically reduce the resources required to simulate. The practical limit of $N''$ of the classical computer we used is estimated to be around 7.

## 8.1 Unitary Gates

As the difficulty of implementing the ideal quantum $f$-mapping arises from the size of the number of states to be simulated, a gate may still be simulated unitarily, if the

simulated number of states is sufficiently small. In this section, we present the gates that are unitarily constructed under the proposed protocol shown above.

### 8.1.1 Single-Qubit Gates

Single-qubit gates such as the single-qubit rotation gate (see subsection 5.1.1), the Hadamard gate (see subsection 5.1.2), and the NOT gate (see subsection 7.2.1) are straightforward to implement. Since the single-qubit rotation gate in Pauli matrix convention requires auxiliary levels, in so far as the probability does not leak to such non-computational states, the implementation of the gate is complete: We use equation (5.1). Since both the Hadamard gate and the NOT gate only require two computational states in addition to the single-qubit rotation gate when the global phase is to be matched, we can immediately apply equations (5.2) and (7.5) to our simulation of the $f$-mapping. An array with two rows, each representing $|0\rangle$ and $|1\rangle$, suffices to be the input and the output of such gates.

### 8.1.2 CNOT Gate

The CNOT gate can be easily implemented unitarily by following the proposed protocol. As all the operations involved in constructing the CNOT gate (see subsection 7.2.2) are performed on the computational states and the phonon state, except for the $2\pi$ rotation of the state $|1\rangle_{phonon}|0\rangle_{target}$, which requires an auxiliary level, the implementation of the CNOT gate becomes straightforward: We follow the method introduced in subsection 7.2.2. The only part that needs to be replaced from subsection 7.2.2 is the $2\pi$ rotation, since the probability leakage out of the computational states is not allowed. By replacing the $2\pi$ Rabi rotation by the sign flip of the phase of the state $|1\rangle_{phonon}|0\rangle_{target}$, we obtain the classical implementation of the CNOT gate. A $2 \times 3$ matrix is used as the input and the output states, where rows represent states and columns represent two qubits and a phonon.

### 8.1.3 CCNOT Gate

In analogy to subsection 7.2.3, we construct a unitary CCNOT gate with the unitary single-qubit rotation gates, the unitary Hadamard gates, and the unitary CNOT gates (see subsections 8.1.1 and 8.1.2), according to the circuit diagram shown in figure 7.3. We note that we still carry the phonon states, as the CCNOT gate is simulated to be unitary. The input and the output states are encoded in a $2 \times 4$ matrix, where the additional column, in comparison to the CNOT case (see subsection 8.1.2), is due to one additional control qubit.

### 8.1.4 SUM Gate

In conjunction to subsection 7.2.4, we implement the SUM gate by using the unitary CNOT gate (see subsection 8.1.2). We make use of the circuit diagram shown in figure 7.4 for the implementation. As in the case of the unitary CCNOT Gate, the input and the output matrix has the dimension of $2 \times 4$.

### 8.1.5 CARRY Gate

With the unitary CNOT gate and the unitary CCNOT gate developed in subsections 8.1.2 and 8.1.3, we implement the unitary CARRY gate as shown in figure 7.5. The input and the output matrix has the dimension of $2 \times 5$ as the unitary CARRY gate is operated on four qubits with one phonon.

## 8.2 Non-Unitary Gates

Before we introduce non-unitarity to our gates, we first show why keeping the unitarity, in general, is such a difficult task on a classical computer. Suppose we have a three-qubit system. If we apply a controlled-gate that manipulates the first and the second qubit of

the system with the third qubit as a control qubit, we are left with two possibilities of the input states of the third qubit: $|0\rangle$ and $|1\rangle$. Hence, if we try to classically simulate such a quantum gate by using the interference of the phases, given multiple input states, we must run a parallel operation on the two manipulated qubits for two different input states of the third qubit, which may result in different output states. Not only does this process demonstrate the efficiency of quantum computing, but also does it show the difficulty of keeping the unitarity on a classical computer: We are forced to simulate all number states, which grow exponentially, in the total number of qubits in our system.

In an attempt to remove as many states as possible from our simulation but to keep the computational results, we follow the proposed protocol in introducing non-unitarity. In this section, we construct the following four block gates, the ADDER gate, the ADD-MOD gate, the CTRL-MULTI-MOD gate, and the EXP-MOD gate, with the help of circuit diagrams (see figures 7.6, 7.7, 7.8, and 7.9) and the classically known input states, i.e. all number states and zeros in register I and II respectively. Since we can determine the classical path in the circuit when provided with a single number state as our input state, we make use of the classical knowledge of the ideal path of such a state as a reference in the process of introducing non-unitarity throughout this section.

### 8.2.1 ADDER Gate

The implementation scheme of non-unitarity in the ADDER gate is relatively straightforward. We separate the ADDER gate into three stages, as shown in figure 8.1:

- Stage(I) - Consecutive $\nu$ forward-CARRY (bar-right) gates,
- Stage(II) - The last forward-CARRY gate, the CNOT gate, and the forward-SUM gate (bar-right),
- Stage(III) - Consecutive $\nu-1$ forward-SUM gate followed by reverse-CARRY (bar-left) gate.

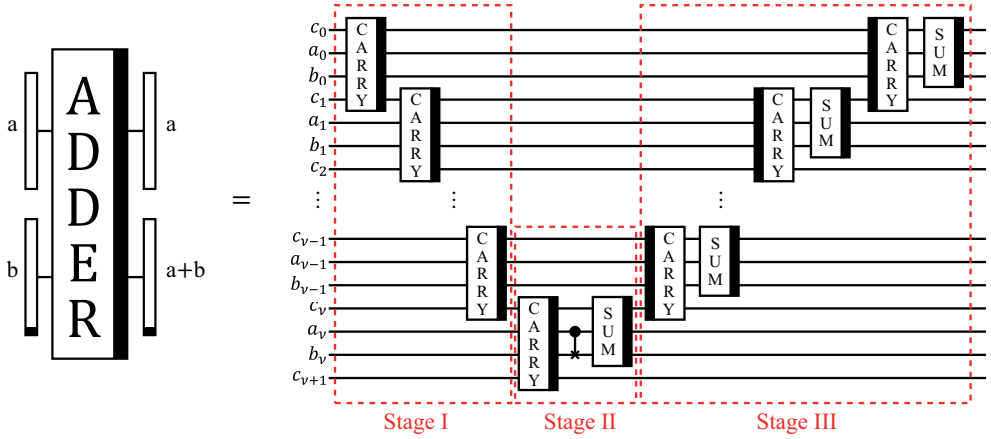



Figure 8.1: Stage-by-stage decomposition of the logic circuit of the ADDER gate. Non-unitarity is introduced systematically according to each stage. Notice the structural symmetry of the stages.

In the first stage, after each of the forward-CARRY operations, we keep the states only with the ideal output in the auxiliary qubits and in the phonon, since our protocol allows us to eliminate non-ideal results in the auxiliary qubit states or in the phonon states. The elimination process, then, disentangles the $j$'th pair of qubits $a_j$ and $b_j$ from other pairs of qubits, because the process is similar (but not identical due to re-normalization) to the measurement of such auxiliary qubits in the ideal state. We note that the two consecutive forward-CARRY operations in the ADDER gate may entangle many pairs of computational qubits via the shared output-input auxiliary qubit between them. By breaking the unitarity through the elimination of non-ideal output states in the auxiliary qubits, we exponentially reduce the number of states to be simulated.

Then, in the second stage, we combine the three operations, the forward-CARRY gate, the CNOT gate, and the SUM gate, to one combinatorial gate, which acts on the system with four qubits and a phonon. In this combinatorial gate, all qubits and phonon states are to be simulated unitarily except for the two auxiliary qubits, $c_\nu$ and $c_{\nu+1}$, which are subjected to the elimination process of the non-ideal output. After all the operations are performed, we, then, eliminate the non-ideal phonon state, or the excited phonon

state, to continue to the third stage. We note that this way of simulation allows us to avoid unnecessary loss of probability, despite longer execution time.

The last stage is done in a very similar manner from the first and the second one. We treat the forward-SUM gate and the reverse-CARRY gate as one combinatorial gate in the sense that the phonon mode is carried over, instead of being subjected to the elimination process in between the two gates. By employing the elimination method on the auxiliary qubits via the classical knowledge of the ideal path and by letting the phonon mode return to the ideal state, or the ground state, at the end of each combinatorial gate, we obtain the final output of the ADDER gate in the form of semi-separable states:

$$\begin{aligned} &ADDER_{non-unitary}|a\rangle|b\rangle|overflow(input)\rangle = |a'\rangle|a'+b'\rangle|overflow(output)\rangle \\ &= \left(\prod_{j=0}^{\nu}\{\lambda_{00}^{(j)}|0\rangle|0\rangle + \lambda_{01}^{(j)}|0\rangle|1\rangle + \lambda_{10}^{(j)}|1\rangle|0\rangle + \lambda_{11}^{(j)}|1\rangle|1\rangle\}_j\right)|c_{\nu+1}\rangle, \end{aligned} \tag{8.1}$$

where $a'$ and $b'$ is used to denote that the returned states may not be same as the input states, and the quantity $|\lambda_{00}|^2 + |\lambda_{00}|^2 + |\lambda_{00}|^2 + |\lambda_{00}|^2$ may not add up to 1 if probability is lost. We note the input-output matrix of the ADDER gate has the dimension of $(2 \times 2 \times N'') + 1$ according to equation (8.1), where $N''(\geq N))$ is the number of qubits in register II.

We remark that the reverse-ADDER gate, or the subtractor gate, is constructed in a very similar manner. The implementation is straightforward due to the structural symmetry present in the three stages of the ADDER gate. The form of the output states of the subtractor gate is, therefore, identical to that of the ADDER gate, which is shown in equation (8.1).

### 8.2.2 ADD-MOD Gate

Unlike the ADDER gate, the ADD-MOD gate cannot be easily broken down into stages that results in a perfectly symmetrical structure in the forward and the reverse order.

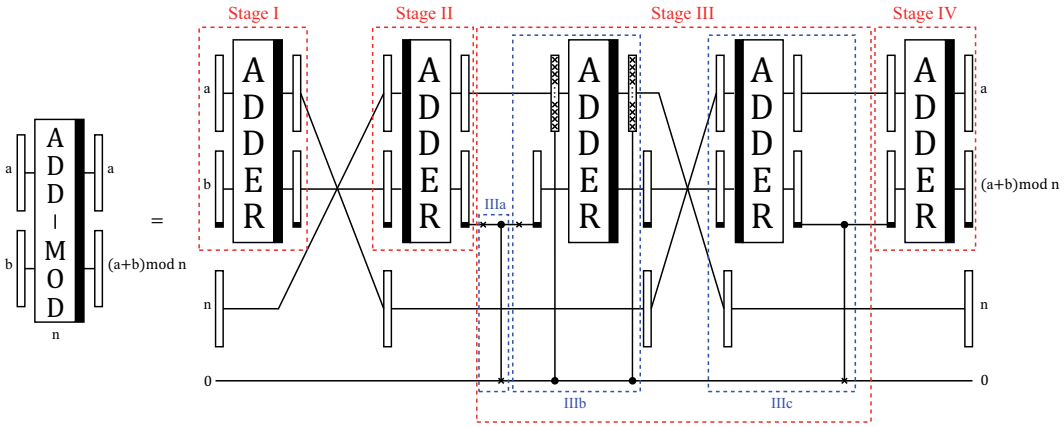



Figure 8.2: Stage-by-stage decomposition of the logic circuit of the ADD-MOD gate. Notice that stage(III) is divided into three parts. Each part contains one type of operation that involves the auxiliary qubit located at the bottom, which results in easier implementation on a classical PC when parallel operations are performed.

However, we can divide the forward ADD-MOD gate into stages that can be easily implemented backwards (see figure 8.2). The division includes the following stages:

- Stage(I) - The forward ADDER gate,
- Stage(II) - The reverse ADDER gate,
- Stage(IIIa) - The NOT-CNOT combinatorial gate,
- Stage(IIIb) - The NOT-CNOT-ADDER-CNOT combinatorial gate,
- Stage(IIIc) - The reverse ADDER-CNOT combinatorial gate,
- Stage(IV) - The forward ADDER gate.

For the reverse ADD-MOD, we simply flip the stage sequence.

We note that it is not necessary to implement the reverse ADD-MOD in our simulation, since we have shown that the restoration mechanism using the reverse CTRL-MULTI-MOD gate, which consists of the reverse ADD-MOD gates, may not be easily implemented due to the difficulty of finding $\xi_j$ in equation (7.7) (see subsection 7.3.3). As we

will use the forward CTRL-MULTI-MOD only, we may, therefore, omit the non-unitary implementation of the reverse ADD-MOD gate.

Implementing stage(I) and (II) is straightforward. We apply the non-unitary forward ADDER gate introduced in subsection 8.2.1 and discard all the non-ideal states in our non-unitary ADDER output in stage(I). Then, we swap the qubit arrays as shown in figure 8.2 and perform the non-unitary reverse ADDER gate for stage(II). We see that the swapping is easily simulated on a classical computer by simply replacing the input array.

In stage(IIIa), we first apply the unitary NOT gate (see subsection 8.1.1) on the highest carry qubit whose state is already idealized by the non-unitary ADDER gates (see subsection 8.2.1). Sequentially, we apply the unitary CNOT gate (see subsection 8.1.2), whose control and target qubits are the highest carry qubit and the additional qubit of the ADD-MOD gate, respectively. We, then, choose to suppress the non-ideal output in the phonon state, or the excited state. We note that we do not idealize the additional qubit, meaning the state of the highest carry qubit is entangled with the additional qubit.

As the first step in stage(IIIb), we apply the NOT gate on the highest carry qubit and eliminate the non-deal ouput. We do this in a parallel sense because we have two possible states in the additional qubit. Within the framework of such parallel operations, we apply the unitary CNOT gates to conditionally erase the preloaded value $n$ in the top qubit array as shown in figure 8.2. We eliminate any non-ideal phonon states after each of the consecutive CNOT operations in order to avoid potential "illegal" paths, since we obtained the states of qubits from stage(I)-(IIIa) by enforcing the phonon state to return to the ideal state. By repeating the consecutive CNOT operations to conditionally restore $n$ in the top qubit array after the non-unitary forward ADD-MOD gate, we complete stage(IIIb).

Once stage(IIIb) is done, we force our states in the top qubit array to have the value $n$ by discarding all the states that do not represent $n$. This step is necessary because we would like to recycle the array in the next ADD-MOD gate with the implemented value of $n$. We, then, replace the top qubit array that is to be recycled with the displaced output states from stage(I), as shown in figure 8.2. While we are still in the regime of parallel operation due to the additional qubit, we apply the non-unitary reverse ADDER gate followed by the unitary CNOT gate, whose control and target qubits are the idealized highest carry qubit and the additional qubit. Since the additional qubit is supposed to be recycled with the state $|0\rangle$, and there is no more operation left to be done, we eliminate the non-ideal phonon state and the output $|1\rangle$ of the additional qubit after the CNOT operation is complete.

As our last step, we perform the non-unitary forward ADDER gate. This finishes the implementation of the non-unitary ADD-MOD gate with the output

$$\begin{aligned} &ADD-MOD_{non-unitary}|a\rangle|b\rangle|overflow(input)\rangle|n\rangle|0(additional)\rangle \\ &= |a'\rangle|\left((a+b) \bmod n\right)'\rangle|overflow(output)\rangle|n\rangle|0(additional)\rangle, \end{aligned} \tag{8.2}$$

whose format is identical to equation (8.1), the output format of the ADDER gate. Hence, the dimension of the input and the output matrix stays the same as in the case of the ADDER: $(2 \times 2 \times N'') + 1$.

### 8.2.3 CTRL-MULTI-MOD Gate

The introduction of non-unitarity in the CTRL-MULTI-MOD gate can be decomposed into a few stages. Using the methodology described in section 7.1 and the circuit diagram of the CTRL-MULTI-MOD (see figure 8.3), we dissect our circuit in the following way:

- Stage(Ia($k$)) - Load $\alpha_k$ into the auxiliary qubit array using the CCNOT gates,
- Stage(Ib($k$)) - Operate the $k+1$'st forward ADD-MOD gate,

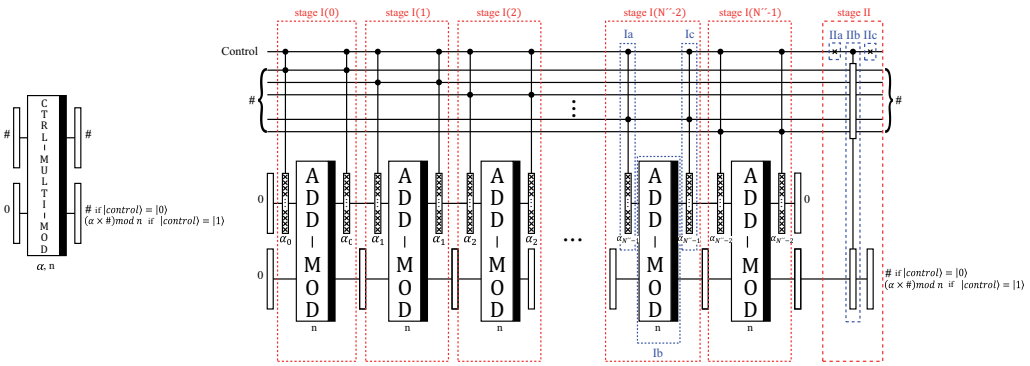



Figure 8.3: Stage-by-stage decomposition of the logic circuit of the CTRL-MULTI-MOD gate. The repeating structure of CCNOT, ADD-MOD, and CCNOT is designated as stage(I). The argument of stage(I($k$)) comes from loading and erasing $\alpha_k$, using CCNOT gates of the stage, as marked in the diagram.

- Stage(Ic($k$)) - Erase $\alpha_k$ from the auxiliary qubit array using the CCNOT gates,
- Stage(IIa) - Perform the NOT gate on the control qubit,
- Stage(IIb) - Perform $N'' - 1$ consecutive CCNOT gates,
- Stage(IIc) - Restore the control qubit using the NOT gate,

where $\alpha_k$ is defined in section 7.1.

Throughout stages (I) and (II), we force the states of the control qubits to remain on the ideal path by eliminating the non-ideal output. This allows us to use separable states between the control qubits and the target qubits. Since the control qubits in the CTRL-MULTI-MOD gate are either ($i$) register I qubits or ($ii$) auxiliary qubits in the output state, the idealization process strictly follows the protocol.

In stage(Ia($k$)), we use the unitary CCNOT gate (see subsection 8.1.3) to load $\alpha_k$. Since only the ideal output of the control qubits are to be kept, we have a potential probability loss in this stage, where such a loss will be propagated through the phases of the auxiliary target qubits. We note that the entire stage is under parallel operation due to two possible states of each qubit in the bottom-most array (see figure 8.3), which

are in a semi-separable relation with the auxiliary qubits (see equation (8.2)).

Stage(Ib($k$)) is straightforward to implement as we already constructed the non-unitary forward ADD-MOD gate in subsection 8.2.2. Since the output states of the non-unitary ADD-MOD gate is, again, in the semi-separable form (see subsection 8.2.2), the following stage(Ic) must be conducted in a parallel manner, too.

We complete stage(I($k$)) by applying the unitary CCNOT gates on the auxiliary qubits to remove the implemented state $\alpha_k$. In stage (Ic($k$)), in addition to the idealization of the control qubits, we eliminate any non-ideal state in the auxiliary qubit array, which leads to yet another potential probability loss. We note that we combine the potential probability loss from both the input states of the unitary CCNOT gates and the idealization process and impose the total loss on the phases of the bottom most qubit array, or register II in the output state (see figure 8.3).

The implementation of the entire stage(II) is straightforward as well. First, we apply the unitary NOT gate to the control qubit that belongs to register I (see subsections 7.3.2 and 7.3.3) and idealize the output. Then, we apply $N''$ consecutive unitary CCNOT gates to conditionally copy the states from the old register II to the new register II (see figures 7.8 and 7.10, and subsections 7.3.2 and 7.3.3), followed by the unitary NOT gate on the control qubit in register I, to restore the state. Finally, we eliminate any non-ideal states in the control qubit and the old register II (top output array, see figure 8.3) as auxiliary qubits (see figure 7.10) and extract the phases in the new register II (bottom output array).

Since the input and the output states have one idealized control qubit from register I and $N''$ qubits from register II, the dimension of the input and the output matrix is $2 \times 1$ for the control qubit and $2 \times N''$ for the register II, where the rows represent states, $|0\rangle$ and $|1\rangle$. We remark that the non-unitary CTRL-MULTI-MOD does the following

operation:

$$\begin{aligned} &CTRL-MULTI-MOD_{non-unitary}(x)|r_j(control)\rangle|\#\rangle|0(aux)\rangle|0(regII)\rangle \\ &= |r_j(control)\rangle|\#\rangle|0(aux)\rangle|(\# \times \alpha^{r_j} \bmod n)'\rangle, \alpha \equiv x^{2^j}, \end{aligned} \tag{8.3}$$

where $x$ is the seed.

### 8.2.4 EXP-MOD Gate

Using subsection 7.3.3 and figure 7.10, we can immediately implement the non-unitary EXP-MOD gate on a 32-bit classical PC. Since we have already idealized the auxiliary qubits' output in the non-unitary CTRL-MULTI-MOD gate, all we need now reduces to updating the control qubit in register I, which is trivial to implement since our control qubit state is separable from register II qubits as shown in equation (8.3). We need not worry about introducing new qubits with an integer state $|0\rangle$, since, by default, the non-unitary CTRL-MULTI-MOD is constructed to have one (see subsection 8.2.3).

## 8.3 Non-Unitary Shor's algorithm without Noise

### 8.3.1 Non-Unitary $f$-mapping

In this subsection, we present the probability spectrum of measuring an equivalence class element from the non-unitary implementation of the EXP-MOD gate (see subsection 8.2.4). As our protocol demands that the ideal path is kept at all times, we expect to obtain an ideal spectrum of equivalence class elements with the periodicity $\omega$, or the order (see Chapters 3-5).

In figure 8.4, we see a probability spectrum that shows us how likely it is for a state in register I to be in the equivalence class of $|1\rangle$ in register II. We chose the equivalence class with the state $|1\rangle$ in register II because the state $|1\rangle$ always exists in register II:

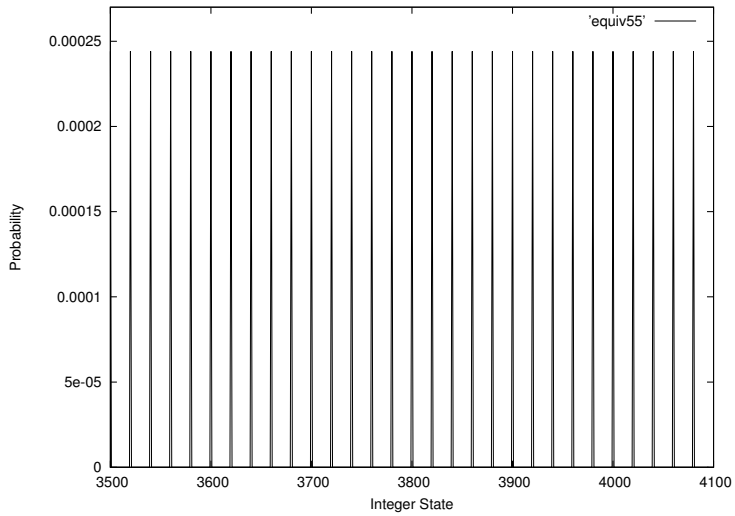



Figure 8.4: Probability spectrum of an equivalence class obtained from a complete, non-unitary implementation of Shor's algorithm. The semi-prime number to be factored is 55 and the seed value is 2. The probability is not normalized to 1 but to $\frac{1}{\omega}$, which is the probability of measuring the equivalence class $|1\rangle$ in register II. For a better visual representation, we show peaks corresponding to integer states higher than 3500 only.

the 0'th power of any seed is 1. We immediately observe the periodicity is the order $\omega$ as expected.

### 8.3.2 Unitary Peripheral Shor's algorithm and Non-Unitary $f$-mapping without Noise

As we now have $(i)$ the unitary peripheral Shor's algorithm (see Chapter 5) and $(ii)$ the non-unitary quantum $f$-mapping implemented on a classical computer, we can simulate a quantum computer running a complete non-unitary Shor's algorithm on a classical PC. The only alteration that needs to be made in the peripheral Shor's algorithm implementation is to take the non-ideal $f$-mapping results into account. Since the $f$-mapping is non-unitary, we cannot use a re-normalization process via the measurement of the state of register II. Instead, we run the peripheral Shor's algorithm for all equivalence classes

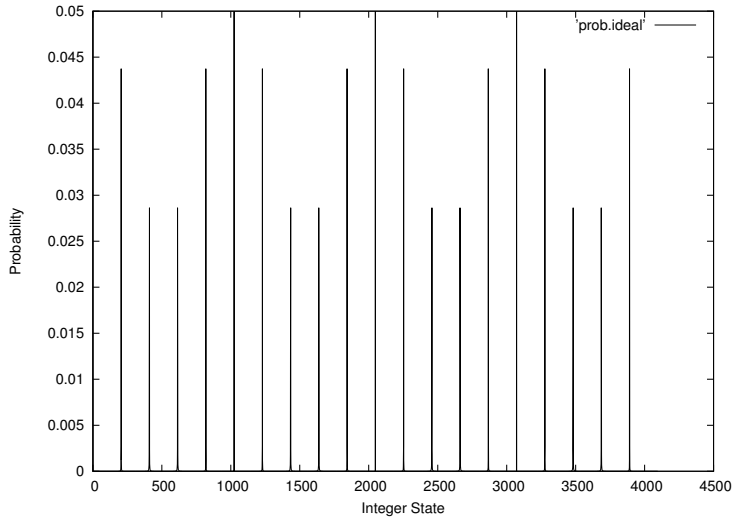



Figure 8.5: Probability spectrum of Shor's algorithm via the complete, non-unitary implementation. The semi-prime number factored is 55 and the seed value is 2. All equivalence classes are used; the norm of the probability is 1.

to obtain the final probability spectrum of Shor's algorithm.

The probability spectrum of the complete non-unitary Shor's algorithm, simulated on a classical PC, factoring the semi-prime number 55 with seed 2, is shown in figure 8.5. In order to check the simulation, we compare the resultant probability spectrum in figure 8.5 with that obtained from the evaluation of the probability equation (3.3). We immediately observe that both probability spectra (see figures 4.1 and 8.5) are identical. Since nowhere in the simulation code is the order $\omega$ used, **the implementation of the entire quantum Shor algorithm on a classical PC is complete.**

## 8.4 Scaling of the number of basic quantum gates

We note that the number of basic simple gates, such as the single-qubit rotation gate, the Hadamard gate, the NOT gate, and the CNOT gate, scales polynomially with respect to the number of qubits of a quantum computer. With the number of qubits $N'$ and $N''$

in register I and II respectively, the different parts of the algorithm require the following resources.

1. Initialization process: $N'$ Hadamard operations (see subsection 5.2.1);
2. Quantum Fourier transform: $N'$ Hadamard operations and $\frac{N'(N'-1)}{2}$ single-qubit rotation gates (see subsection 5.2.2);
3. Quantum $f$-mapping: $2N'(N''+1)$ NOT gates, $N'(44N''^2 - 18N'')$ Hadamard gates, $N'(176N''^2 - 72N'')$ single-qubit rotation gates, and $N'(154N''^2 - 52N'')$ CNOT gates (see Chapter 7);

- One EXP-MOD gate: $N'$ CTRL-MULTI-MOD gates;
- One CTRL-MULTI-MOD gate: $N''$ ADD-MOD gates, $2N''^2 + N''$ CCNOT gates, and 2 NOT gates;
- One ADD-MOD gate: 5 ADDER gates, $2N'' + 2$ CNOT gates, and 2 NOT gates;
- One ADDER gate: $2N'' - 1$ CARRY gates, $N''$ SUM gates, and 1 CNOT gate;
- One CARRY gate: 2 CCNOT gates and 1 CNOT gate;
- One SUM gate: 2 CNOT gates;
- One CCNOT gate: 6 CNOT gates, 2 Hadamard gates, and 8 single-qubit rotation gates.

# Chapter 9

# Complete Implementation of Shor's Algorithm with Noise

While we have implemented a complete non-unitary Shor's algorithm on a classical PC in Chapter 8, we have not yet introduced any noise to the system, in particular, in regards to the quantum $f$-mapping. In this chapter, therefore, we investigate the influence of the noise on the probability spectrum of the equivalence class and of the $\tilde{y}$ (see section 3.3) via a simulation of the complete non-unitary Shor's algorithm.

## 9.1 Noise Introduction

The introduction scheme of noise in our simulation is the same as discussed in section 6.1. Since we have already developed the way to introduce noise in the single-qubit rotation gate and the Hadamard gate in section 6.1 as shown in equations (6.1) and (6.2), in order to introduce noise throughout the simulation, only the NOT and the CNOT gate needs to be subjected to the introduction of noise.

Introducing noise to the NOT gate is straightforward: We apply a noise margin $\Delta\kappa$ to

$\kappa = \frac{\pi}{2}$, so that a noisy NOT gate becomes

$$\begin{pmatrix} \cos(\kappa + \Delta\kappa) & i\sin(\kappa + \Delta\kappa) \\ i\sin(\kappa + \Delta\kappa) & \cos(\kappa + \Delta\kappa) \end{pmatrix}.$$

Similarly, we introduce noise to the CNOT gate by providing an error margin to each of the Pauli-$x$ and Pauli-$y$ pulses (see subsection 7.2.2). Since we are not to consider probability leakage that is through non-computational states, as described in the protocol shown in Chapter 8, we do not manipulate the $2\pi$ Rabi cycle in our noise introduction scheme.

## 9.2 Probability Spectrum

In order to determine the effectiveness of Shor's algorithm under realistic conditions, we study the probability spectrum of the complete, non-unitary Shor's algorithm with noise in this section. We remark that not only do we need to pay attention to where the peaks form on a probability spectrum, but we also need to keep track of the norm of the probability in order to make sure there is enough norm left to be physically meaningful, as our simulation is coded under non-unitary conditions.

### 9.2.1 Equivalence Class

As the protocol for breaking the unitarity shown in Chapter 8 ensures that register I qubits are ideal, we have the following structure of the transformation performed by the implementation of the non-unitary quantum $f$-mapping:

$$|j'th\ qubit\ state\rangle_{regI} \rightarrow |j'th\ qubit\ state\rangle_{regI}\{\sum_{\eta=0}^{\omega-1} \zeta_\eta^j |\eta\rangle_{regII}\},$$

where $\eta$ denotes the integer state in register II and $\zeta_\eta^j$ denotes the phase factor of the state $|j'th\ qubit\ state\rangle_{regI}|\eta\rangle_{regII}$. We note that the sum of absolute square values of all

phase factors may not add up to 1, as we introduce noise to our system that results in probability leakage.

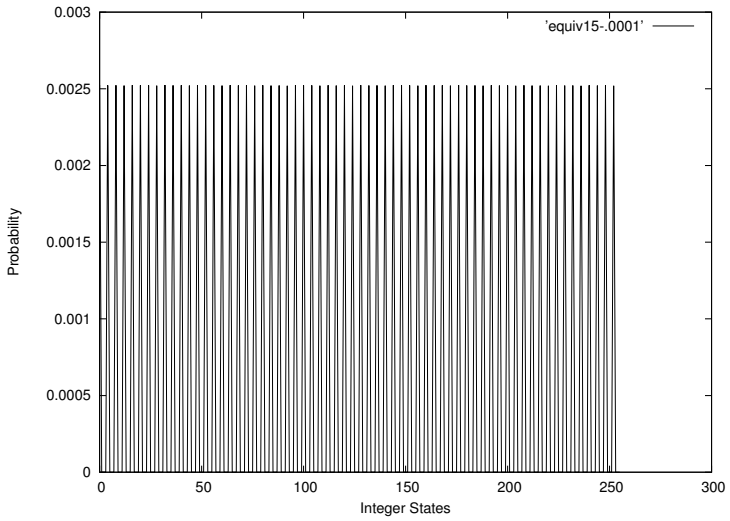
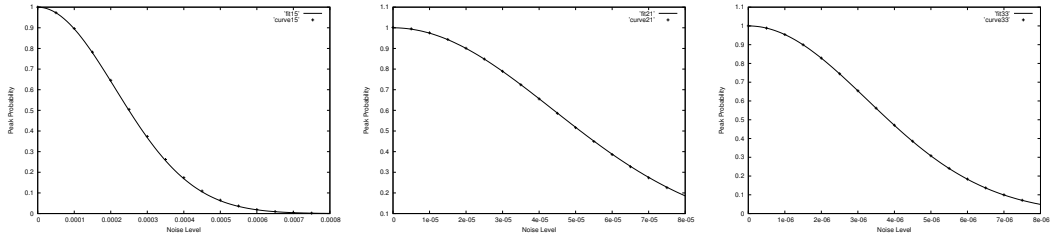



Figure 9.1: Probability spectrum of equivalence class with noise via the complete, non-unitary implementation. The semi-prime number to be factored is 15 and the seed value is 2. The norm of the total probability is 16.142%. The peak probabilities are all of the same height.

When the four basic quantum gates, the single-qubit rotation gate, the Hadamard gate, the NOT gate, and the CNOT gate, are introduced with an error margin of .0001 radian, for instance, for the case of factoring $15 = 3 \times 5$, we obtain nearly 65% probability of achieving correct $f$-mapping results, on the qubit-by-qubit level. An equivalence class probability spectrum of this case is shown in figure 9.1. The sum of probability compared to the ideal value $\frac{1}{\omega}$ is found to be near $(.65)^4$, as we have 4 qubits in register II. We note that the peak positions in register I for an equivalence class are unaltered despite the noise, due to our protocol that conserves the ideal path, which results in the fact that the peak probability is identical to the total probability.

The behavior of the peak probability of an equivalence class with respect to noise is shown in figure 9.2. We observe a remarkable resemblance to figure 6.2: the fitted function has the form of a Gaussian function.




(a) $n = 15$,$x = 2$ (b) $n = 21$,$x = 2$ (c) $n = 33$,$x = 5$

Figure 9.2: Peak probability of non-unitary Shor's algorithm vs. noise level. The fitted functions are Gaussian; $e^{-\frac{x^2}{\sigma}}$. $\sigma$ values are $9 \times 10^{-8}$, $3.8 \times 10^{-9}$, and $2.12 \times 10^{-11}$ for the cases 15, 21, and 33, respectively.

We remark that the probability loss due to non-unitarity in our simulation is significant when noise is introduced to the code. Due to expensive computation requirements for simulating the non-unitary $f$-mapping, however, the scaling behavior of total probability loss with respect to the number of qubits with a fixed noise level was unattainable. Such a scaling behavior is not physically meaningful anyway in that the non-unitarity in our simulation only picks up the loss of probability out of the ideal path, not the gain of probability from non-ideal states. In order to study the true system in an absolute sense, we must simulate in a unitary regime, which can be done if provided with proper resources.

### 9.2.2 Final Measurement

As a demonstration of the simulation of the complete non-unitary Shor's algorithm with noise, we present the probability spectrum of $\tilde{y}$ in figure 9.3. With the semi-prime number 21 and the seed 2, a complete simulation of the non-unitary Shor's algorithm with noise level .00003 radian in the $f$-mapping, .2 radian in the Hadamard gate of the peripheral Shor's algorithm, and .25 radian in the single-qubit rotation gate was run. The limit of the quantum Fourier transform was set to four maximal spacings between the reference qubit and the operated qubit. Since we cannot use the renormalization

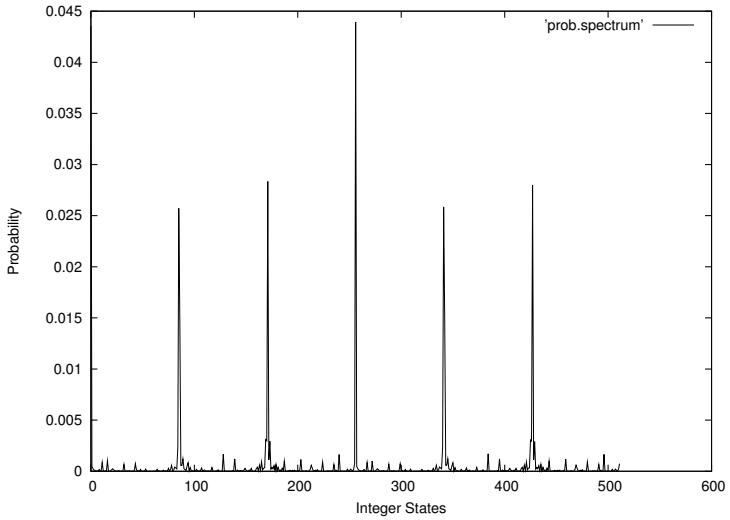



Figure 9.3: Probability spectrum of Shor's algorithm with noise via the complete, non-unitary implementation. The semi-prime number to be factored is 21 and the seed value is 2. The norm of the total probability is 32%. The peak probability amounts to 60% of the total probability.

scheme via measurement in register II, as described in subsection 8.3.2, we performed the quantum Fourier transform on all equivalence classes to obtain the final probability spectrum for reading $\tilde{y}$. As shown in figure 9.3, it turns out that under these conditions the probability spectrum has a well-defined peak structure that enables reliable quantum factoring. This shows that even with relatively large noise levels quantum factoring is possible.

# Chapter 10

# Dicussion

Throughout the thesis, we have seen how a quantum computer achieves its power through entanglement and superposition. The massive parallelism that is inherent in a quantum computer boosts the computing power exponentially, compared to a classical PC. It was also the digital nature of measurement that enabled the "infinite precision" (see Chapter 4) of a quantum computer, which returns an *integer* value.

Despite a concern of the vulnerability of a quantum computer with respect to noise, such a concern turned out to be exaggerated. In this thesis we showed that especially the quantum Fourier transform is extremely robust against the noise, allowing noise that is far larger than the gate itself, e.g. a noise level that reaches 260000% of the smallest single-qubit rotation gate in the case of factoring 323, yet attaining nearly 50% probability of landing on the correct peaks. This is so, because even the noise is introduced in a parallel sense due to the quantum mechanical nature of superposition and entanglement, which leads to an unexpected robustness of a quantum computer against noise.

Even in the presence of a physical limit, discussed in subsection 5.3.2 in combination with the noise, we see that the peak probability in the quantum Fourier transform

is not easily lost. In fact, the peak probability even rises in some cases as shown in figure 6.4. In addition, the limit on the quantum Fourier transform itself appears to be nearly ignorable: the analytical expression (5.6) predicts a fidelity of 95% for a 300-qubit quantum computer, only with the realization of the single-qubit rotation gates that can rotate $\frac{2\pi}{2^9}$ radian, which is well within the precision limit of currently available experimental tools. A nearly unbreakable quantum Fourier transform, then, leads to the following important paradigm: Any problem that can be turned into a period-finding problem can be $(i)$ efficiently and $(ii)$ reliably solved by a quantum computer.

We remark that the entire quantum computing process is closely related to the decoherence of the quantum states. This is so, because, if we define our universe to consist of the states of a quantum computer, the quantum computing process can be understood as the interaction between the ideal classical paths and the non-ideal paths. The way in which we introduced the non-unitarity via the protocols proposed in Chapter 8, however, includes the repeated elimination of the non-ideal paths in the $f$-mapping, so the decoherence behavior is not fully captured.

# Chapter 11

# Conclusion

This thesis provides several new insights that may come in handy for future work in the field of quantum information. The exact mechanism on how the quantum computer achieves its power over the classical counterpart and how Shor's algorithm takes the maximal advantage of them provides us with inspirations to develop a new quantum software. The analytical result on the limit of the quantum Fourier transform provides us with the means for substantial savings in quantum computer hardware. The level of robustness of the quantum computer demonstrated in the thesis gives us a quantitative idea of the maximal noise level allowed in successful quantum computing.

The extension of our investigation includes the following directions. We suspect the noise-level dependence of the peak probability, being approximated as a stretched Gaussian function, may perhaps be a part of a universal behavior of the peak probability, provided with the data that always shows such a behavior. In addition, accomplishing the only missing piece, the implementation of the unitary quantum $f$-mapping, will result in a complete, unitary implementation of Shor's algorithm on a classical computer.

We note that a crucial part in investigating both aspects, in particular the unitary $f$-mapping, is the availability of better resources. With the currently available machines

at hand, a 32-bit PC, it is simply impossible to proceed further. However, progress and advancement can be made with the help of a 64-bit classical PC equipped with enough RAM. We remark that the unitary $f$-mapping of the semi-prime number 15 can be run with available market products such as a MSI X79A-GD45(8D) motherboard, which supports up to 128 GB of RAM.

Given this extraordinary demand on resources, if Shor's algorithm is run "honestly", i.e. including a unitary implementation of the $f$-mapping, we can say with confidence that contrary to published claims [43], with presently experimentally demonstrated universal quantum computers that contain up to 7 qubits [31, 43], not even the number 15 can "honestly" be factored. Quantum factorization of $n = 15$ can only be claimed by relying on additional classical ("forbidden") knowledge, as explained in Chapter 5. Such a short-cut factorization works for small $n$, where this classical knowledge is readily available, but does not work for the "interesting", 300-digit numbers (code-breaking) at which Shor's algorithm is mainly directed. In this case, the $f$-mapping becomes of central importance and its reaction to noise has to be studied seriously, as was done in this Thesis.

# Appendix A

# Unitary N xN Transformations

In this appendix, the following theorem is proved: any infinitesimal $N$-dimensional unitary transformation can be decomposed into two-dimensional unitary transformations. Since finite unitary transformations result from repeated applications of infinitesimal unitary transformations, we obtain the corollary that any finite unitary transformation can be decomposed into a finite number of two-dimensional unitary transformations. Possible applications of the theorem include a new algorithm for diagonalizing a matrix (see appendix B) and the solution of quantum mean-field problems, such as the computation of the ground state of an atom or a nucleus. For the purpose of quantum computation, the theorem is used for the realization of quantum circuits, using exclusively $2 \times 2$ unitary transformations.

The theorem is motivated by a theorem on Euler rotations, which states that in three-dimensional space any rotation can be performed with three two-dimensional rotations. By viewing the full 3-D rotation in two different ways, ($i$) three active rotations in three orthogonal two-dimensional planes, ($ii$) rotations with each of three orthogonal reference axes (see figure A.1), the analogy to decompose an $N$-dimensional matrix into many two-dimensional matrices becomes clear: choose each one of $N$ axes as a reference axis of

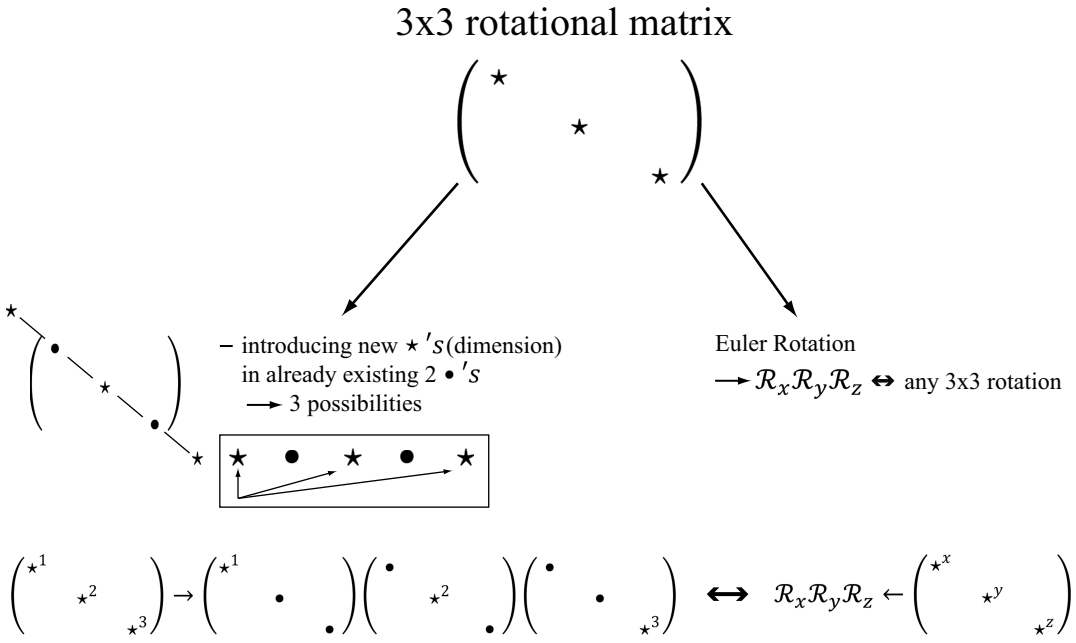



Figure A.1: Two different ways to understand an Euler rotation. The left-hand side demonstrates the axis-selective point of view and the right-hand side demonstrates the active, two-dimensional point of view. Notice that they yield the same result.

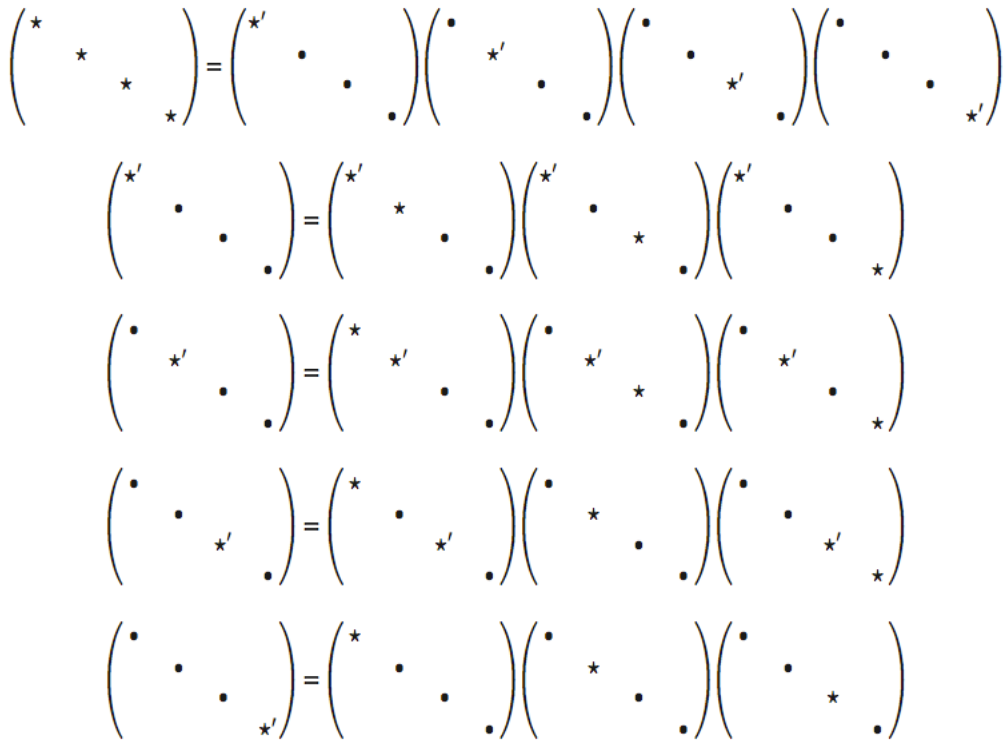


Figure A.2: Visualization of the decomposition process. This diagram shows how a $4\times4$ matrix can be decomposed into four $3\times3$ matrices, then into twelve $2\times2$ matrices by choosing different reference axes, denoted as $\star'$ and $\star$. Notice that repetitions of the same $2\times2$ matrices occur, which implies a possible reduction of the number of parameters needed for a full decomposition.

rotation to decompose an $N$-dimensional rotation into many $N-1$ dimensional rotations and iterate the process until everything becomes two-dimensional rotations (see figure A.2). Since a rotation is a real unitary (orthogonal) transformation, a complex rotation must be considered to properly describe a general unitary transformation.

With the help of a finite rotation angle $\phi$, a finite $2 \times 2$ rotation may be parametrized as

$$\begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}.$$

Using the Euler-rotation-like analogy developed above, a $3 \times 3$ finite, real rotation can be constructed as

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\phi_x & -\sin\phi_x \\ 0 & \sin\phi_x & \cos\phi_x \end{pmatrix} \begin{pmatrix} \cos\phi_y & 0 & -\sin\phi_y \\ 0 & 1 & 0 \\ \sin\phi_y & 0 & \cos\phi_y \end{pmatrix} \begin{pmatrix} \cos\phi_x & -\sin\phi_x & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} \cos\phi_x\cos\phi_y & \begin{matrix} -\cos\phi_z\sin\phi_x \\ +\cos\phi_x\sin\phi_y\sin\phi_z \end{matrix} & \begin{matrix} \cos\phi_x\cos\phi_z\sin\phi_y \\ +\sin\phi_x\sin\phi_z \end{matrix} \\ \cos\phi_y\sin\phi_x & \begin{matrix} \cos\phi_x\cos\phi_z \\ +\sin\phi_x\sin\phi_y\sin\phi_z \end{matrix} & \begin{matrix} -\cos\phi_z\sin\phi_x\sin\phi_y \\ -\cos\phi_x\sin\phi_z \end{matrix} \\ -\sin\phi_y & \cos\phi_y\sin\phi_z & \cos\phi_y\cos\phi_z \end{pmatrix}.$$

Since an Euler rotation and the associated analogy are general, the construction method of a real unitary (orthogonal) transformation is now complete.

For the complex case, a general unitary transformation in two-dimensional space is proposed according to

$$\begin{pmatrix} e^{i\theta}\cos\phi & -e^{i\alpha}\sin\phi \\ e^{-i\alpha}\sin\phi & e^{-i\theta}\cos\phi \end{pmatrix}.$$

Such a matrix is, indeed, general up to a global phase. In analogy to the real case, a three-dimensional complex, general, unitary transformation can now be attempted to be

constructed, as

$$\begin{pmatrix} FINITE & COMPLEX \\ 3D & ROTATION? \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & e^{i\theta_x}\cos\phi_x & -e^{i\alpha_x}\sin\phi_x \\ 0 & e^{-i\alpha_x}\sin\phi_x & e^{-i\theta_x}\cos\phi_x \end{pmatrix} \begin{pmatrix} e^{i\theta_y}\cos\phi_y & 0 & -e^{i\alpha_y}\sin\phi_y \\ 0 & 1 & 0 \\ e^{-i\alpha_y}\sin\phi_y & 0 & e^{-i\theta_y}\cos\phi_y \end{pmatrix} \begin{pmatrix} e^{i\theta_z}\cos\phi_z & -e^{i\alpha_z}\sin\phi_z & 0 \\ e^{-i\alpha_z}\sin\phi_z & e^{-i\theta_z}\cos\phi_z & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{A.1}$$

If we compute the matrix shown in equation (A.1), we obtain:

$$\begin{pmatrix} e^{i(\theta_y+\theta_z)}\cos\phi_y\cos\phi_z & -e^{i(\theta_y+\alpha_z)}\cos\phi_y\sin\phi_z & -e^{i\alpha_y}\sin\phi_y \\ e^{i(\theta_x-\alpha_z)}\cos\phi_x\sin\phi_z & e^{i(\theta_x-\theta_z)}\cos\phi_x\cos\phi_z & -e^{i(\alpha_x-\theta_y)}\sin\phi_x\cos\phi_y \\ -e^{i(\alpha_x-\alpha_y+\theta_z)}\sin\phi_x\sin\phi_y\cos\phi_z & +e^{i(\alpha_x-\alpha_y+\alpha_z)}\sin\phi_x\sin\phi_y\sin\phi_z & \\ e^{-i(\alpha_x-\alpha_z)}\sin\phi_x\sin\phi_z & e^{-i(\alpha_x+\theta_z)}\sin\phi_x\cos\phi_z & e^{-i(\theta_x+\theta_y)}\cos\phi_x\cos\phi_y \\ +e^{-i(\theta_x+\alpha_y-\theta_z)}\cos\phi_x\sin\phi_y\cos\phi_z & -e^{-i(\theta_x+\alpha_y-\alpha_z)}\cos\phi_x\sin\phi_y\sin\phi_z & \end{pmatrix}.$$

We immediately notice that we have too many parameters at our hands. Determining that the equation (A.1) is indeed the general 3D complex rotation, then, becomes an extremely difficult task as the associated notation is very cumbersome. Hence, continuing with finite angles for the general case is not desirable. For a much simpler treatment, therefore, an infinitesimal regime must be considered.

Consider an infinitesimal real rotation, or an infinitesimal unitary transformation, in two dimensions. Using a first-order approximation of the Taylor expansion, such a rotation matrix is constructed as

$$\begin{pmatrix} 1 & -\phi \\ \phi & 1 \end{pmatrix}.$$

We note that we need to be careful when using the misleading term, "an infinitesimal unitary transformation," as the unitarity is not conserved in the infinitesimal case: the

term is only to indicate that the unitary transformation is achieved in the limit of an infinite number of infinitesimal transformations.

A general 3-D infinitesimal, real unitary transformation is obtained through three orthogonal 2-D planar rotations

$$\begin{pmatrix} 1 & -\phi_z & -\phi_y \\ \phi_z & 1 & -\phi_x \\ \phi_y & \phi_x & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & -\phi_x \\ 0 & \phi_x & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -\phi_y \\ 0 & 1 & 0 \\ \phi_y & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -\phi_z & 0 \\ \phi_z & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

It is trivial to see that the matrix is general since the matrix is identical to a perturbed identity matrix. Since every possible perturbation is considered, such a matrix is the most general infinitesimal, real unitary (orthogonal) transformation.

It is important to note that the infinitesimal transformation approach provides several other major advantages over the finite approach in the following way. First of all, an infinitesimal rotation has the truncation condition to be of first order in the Taylor expansion, which leads to a much simpler notation in comparison to a full, finite rotation. This property is particularly useful in that an $N$-dimensional generalization is to be obtained. In addition, the operations involved in the infinitesimal approach commute, whereas those for the finite transformation do not, in general, for instance in the case of a regular three-dimensional Euler rotation. Since a finite rotation is achieved with infinitely many infinitesimal rotations, generalities that are obtained for an infinitesimal case carry over to the finite case.

With the analogy developed earlier in this appendix, the generalization of an infinitesimal real case to $N$ dimensions is straightforward. This means it is possible to construct any real $N \times N$ infinitesimal rotation with those of $2 \times 2$'s. Conversely, any $N$-dimensional real, unitary matrix can be decomposed into $2 \times 2$ real, unitary matrices.

In an analogous way, a complex unitary transformation can be explored in the infinitesimal regime. Taylor expanding the 2-D complex rotation and truncating at the first order, an infinitesimal 2-D complex rotation is obtained, where $\phi$ is now complex.

$$\begin{pmatrix} 1+i\theta & -\phi \\ \phi & 1+i\psi \end{pmatrix}$$

$$\begin{pmatrix} 1+i(\theta_y+\theta_z) & -\phi_z & -\phi_y \\ \phi_z^* & 1+i(\psi_z+\theta_x) & -\phi_x \\ \phi_y^* & \phi_x^* & 1+i(\psi_y+\psi_x) \end{pmatrix}$$
$$= \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1+i\theta_x & -\phi_x \\ 0 & \phi_x & 1+i\psi_x \end{pmatrix} \begin{pmatrix} 1+i\theta_y & 0 & -\phi_y \\ 0 & 1 & 0 \\ \phi_y & 0 & 1+i\psi_y \end{pmatrix} \begin{pmatrix} 1+i\theta_z & -\phi_z & 0 \\ \phi_z & 1+i\psi_z & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{A.2}$$

Then, a 3-D case is obtained in analogy (see equation (A.2)), and the verification method through perturbation theory confirms the generality. Due to the infinitesimal condition, different local phases for each axis are allowed. Again, with an immediate generalization to $N$ dimensions, a general $N$-dimensional complex unitary transformation is achieved and any $N$-dimensional complex unitary matrix can now be decomposed into $2 \times 2$ complex unitary matrices.

It must be noted that the complex case is over-complete. In regards to the generality of the matrix, the real case requires $\frac{N(N-1)}{2}$ real parameters to fully describe an infinitesimal unitary transformation due to perturbation theory (see the real matrix of diagram A.3), whereas the complex case requires $N^2$ real parameters due to the normalization and the orthogonalization conditions (see the complex matrix of diagram A.3). When the matrix is decomposed into lower dimensional matrices, while the number of real parameters needed in the real case does not change, since all the diagonal elements are 1's, those for the complex case do change by $N$, as the phases in the diagonal elements of the original matrix are divided, which introduces extra parameters. The complex matrix of figure A.3 clearly shows this property.

The theory investigated above is of particular interest due to its universality in any $N \times N$ matrix. For instance, the method developed to decompose an $N \times N$ matrix into

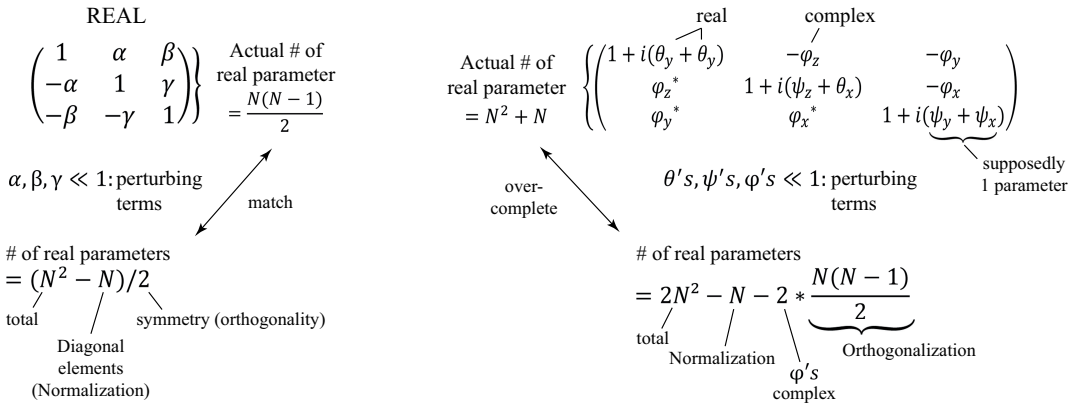



Figure A.3: Infinitesimal unitary transformations and the number of real parameters. The left-hand side shows the real case and the right-hand side shows the complex case. The number of real parameters needed in the real case matches that of the decomposed case. However, there exists a discrepancy for the complex case.

many $2 \times 2$ matrices is directly applicable to developing a new method to diagonalize a matrix. Such a diagonalization method is introduced in the following Appendix B.

# Appendix B

# A New Method for Matrix Diagonalization

In this appendix, a new method for diagonalizing an $N \times N$ real symmetric matrix is presented. With the tool developed in appendix A and the method of steepest descent, it is possible to achieve an $N^2$ scaling algorithm that produces the lowest eigenvalue. Discussion including difficulties and special cases involved in the method is also introduced.

In the previous appendix, the method for decomposing an $N \times N$ matrix into many $2 \times 2$ matrices was demonstrated. This, then, leads to the important property that any vector in $N$ dimensions can be constructed with $2 \times 2$ Jacobi rotations acting on an arbitrary vector, which has the same norm as the desired vector. Mathematically, such an arbitrary vector can be written as $J_1 J_2 J_3 ... J_M$ acting on a unit vector, (100...0) for example, where the $J$'s are Jacobi rotational matrices and $M$ is $\frac{N(N-1)}{2}$ for a real case with normalization.

$$\vec{z} = \left.\begin{pmatrix} \sin a_1 * \sin a_2 * \sin a_3 * ... * \sin a_{N-2} * \sin a_{N-1} \\ \sin a_1 * \sin a_2 * \sin a_3 * ... * \sin a_{N-2} * \cos a_{N-1} \\ \sin a_1 * \sin a_2 * \sin a_3 * ... * \cos a_{N-2} \\ . \\ . \\ . \\ \sin a_1 * \sin a_2 * \cos a_3 \\ \sin a_1 * \cos a_2 \\ \cos a_1 \end{pmatrix} \right\} N. \tag{B.1}$$

With $2 \times 2$ Jacobi rotations, the maximal number of elements in a commuting set of rotations is $\frac{N}{2}$. This is easily understood since any element in a commuting set must live in an independent space, in other words, overlaps in rotation axes must not occur, so that the elements are mutually orthogonal to one another. Since a total of $\frac{N(N-1)}{2} J$'s exist, there are $N-1$ groups of commuting $J$'s. Hence, only $N-1$ $J$'s matter in constructing a vector in $N$ dimensions. The result is also evident from the fact that most $J$'s hit the zeros of (100...0). The resulting vector after applying the $J$'s is shown in equation B.1.

Due to the way that the resulting vector is constructed, $z_1^2 + z_2^2 + z_3^2 + ... + z_N^2 = 1$, as the norm is conserved. In addition, the total number of parameters is $N-1$; $a_1, a_2, ..., a_{N-1}$. Once provided with these $a$'s, the number of calculations needed to generate the vector $\vec{z}$ scales like $N$.

Now, define $f$ to be $\vec{z}^T H \vec{z}$, where $H$ is a real, symmetric $N \times N$ matrix. The number of operations to compute $f$ scales like $N^2$, since computing $H\vec{z}$ requires $\sim N^2$ operations, and applying $\vec{z}^T$ on $H\vec{z}$ requires $\sim N$ operations. The reason behind the $f$'s definition is the following. $f$ is constructed to be $\vec{z}^T H \vec{z}$ because the minimum of $f$ is the lowest eigenvalue of $H$. Suppose $H$ has eigenvectors $\vec{x_1}, \vec{x_2}, ..., \vec{x_N}$ and corresponding eigenvalues $\lambda_1, \lambda_2, ..., \lambda_N$, where $\lambda_1 < \lambda_2 < ... < \lambda_N$. Then, an arbitrary vector can be expanded in

eigenvectors, $\vec{z} = \sum_{j=1}^{N} A_j \vec{x_j}$. $f$ is, therefore, $\sum_{j'=1}^{N} \sum_{j=1}^{N} (A_j \vec{x_j})^T H (A_{j'} \vec{x_{j'}})$. Using the orthogonality relations between eigenvectors, $f$ becomes

$$f = \sum_{j=1}^{N} \sum_{j'=1}^{N} A_j A_{j'} \vec{x_j}^T H \vec{x_{j'}} = \sum_{j=1}^{N} \sum_{j'=1}^{N} A_j A_{j'} \lambda_{j'} \vec{x_j}^T \vec{x_{j'}} = \sum_{j=1}^{N} \sum_{j'=1}^{N} A_j A_{j'} \lambda_{j'} \delta_{j,j'} = \sum_{j=1}^{N} A_j^2 \lambda_j.$$

Since $\lambda_1$ is the smallest eigenvalue, $f$ must be larger than $\sum_{j=1}^{N} A_j^2 \lambda_1 = \lambda_1$. For $\vec{z} = \vec{x_1}$, the minimum $f = \lambda_1$ is actually attained. Hence, $f$ has the minimum value of $\lambda_1$.

So far, a systematic method to parametrize and to generate an arbitrary vector is presented, and the function $f$ that has the minimum value of the lowest eigenvalue of a real symmetric matrix $H$ is identified. It is well known that a good indicator of where to look for the minimum value of a function is its gradient. The method of steepest descent, e.g., is a way to use the gradient for approaching the minimum value of $f$. A sketch of the method is presented in figure B.1.

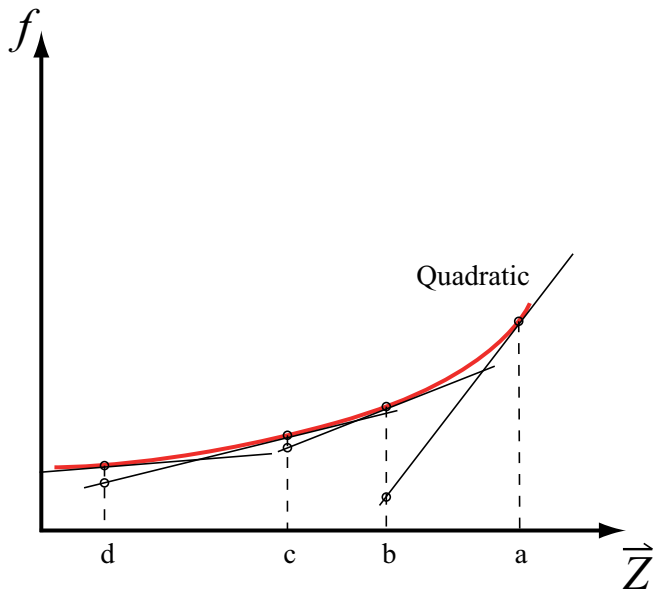



Figure B.1: Sketch of the steepest descent method in a quadratic function. Descending along the gradients in vector space, from $a$ to $d$, $f$ approaches the lowest eigenvalue and the random starting vector $\vec{z}$ approaches the eigenvector associated with the minimum eigenvalue.

For a quadratic minimum, it is evident how the steepest descent method leads to the minimum value of $f$. In the spirit of this example, the following procedure is proposed to find the minimum value of $f$.

1. Find the gradient of $f$ with respect to the $(N-1)$ parameters $a_j$, $j = 1, ..., N-1$.
2. With a specified step size, move along the gradient in the vector space.
3. Recalculate $f$ with newly found vectors and the associated parameters.
4. Iterate the above procedure until a minimum is reached within a pre-specified accuracy.

For step 1, gradients of $f$ can be found in the following way. First, $f$ is parametrized by the $N-1$ parameters $a_k$, $k = 1, ..., N-1$. The gradient of $f$, then, is twice of $\vec{z}^T H (\nabla_k \vec{z})$, where $k$ runs from 1 to $N-1$, since $H$ is symmetric. Since $\vec{z}^T$ and $H$ are known, the only part that needs to be found is $\nabla_k \vec{z}$. As shown in B.1, the resulting vector $\vec{z}$ contains numerous trigonometric functions. Therefore, computing the gradient becomes the simple task of interchanging $\sin(a_k)$ and $\cos(a_k)$ in $\vec{z}$. Counting the number of operations to compute $\vec{z}^T H (\nabla_k \vec{z})$, $\vec{z}^T H$ costs $\sim N^2$ operations and $\nabla_k \vec{z}$ costs $\sim N$ operations for a specified $k$, which means, for all $k$, it scales like $N^2$. Since multiplying $\vec{z}^T H$ and $\nabla_k \vec{z}$ only costs $\sim N$ operations, the entire calculation scales with less than $\sim N^3$ operations. Since the method is an iterative procedure, the algebraic part of the method is finished. Hence, the algebraic part beats the conventional modern method of diagonalizing an $N \times N$ matrix, which scales like $N^3$ (see figure B.2).

Once the gradient of $f$ is found, many iterative steps with the steepest descent method should be made to reach a minimum value of $H$ with a finite step size that is on the order of the desired precision to obtain the lowest eigenvalue of $H$. This entails several difficulties that occur in using the method: The number of iterative steps and trapping at local extrema. Since the step size is proportional to the desired minimum error of the resultant eigenvalue, a large number of iterations may be needed depending on the

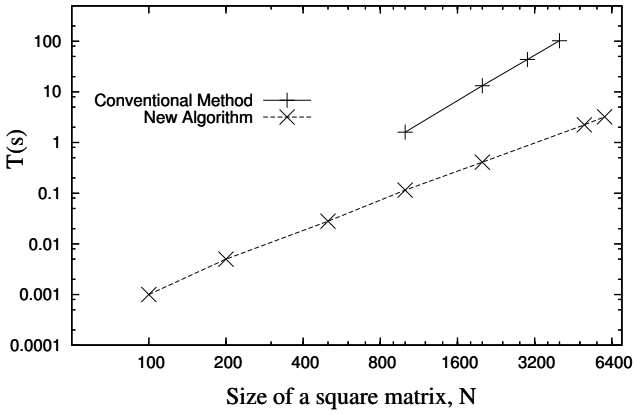
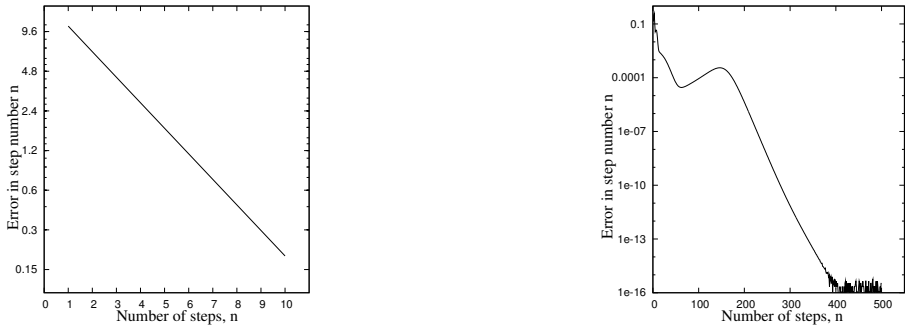



Figure B.2: Comparison of the execution time for diagonalizaing an $N \times N$ matrix between the conventional method and the new method. Power-law behavior of the execution time $T$ vs. the square-matrix size $N$ is obtained by both a modern diagonalization algorithm and the new algorithm. The conventional method scales $\sim N^3$, whereas the new method scales $\sim N^2$.

required precision. Hence, if the number of steps needed to reach the lowest eigenvalue exceeds $N$, the advantage of $N^2$ scaling in the algebraic part is lost. Trapping by a local minimum can occur, for instance, if the step size is too small to jump over the rim of the local well in the function $f$. For a simple function such as a quadratic one, the convergence scheme is exponential as shown in figure B.3a. However, if the function happens to be a complicated one, the exponential convergence does not appear in the early steps, even though such exponential behavior does show up eventually, as shown in figure B.3b. This is because of how the error is measured; the error is defined to be the absolute value of the difference between a previous value and a propagated value. Figure B.4 shows how an increase in error can occur due to the geometry of the function. Irregular gradients in many different parameters, indeed, cause an increase of the error.




(a) Quadratic function

(b) Random function

Figure B.3: Convergence behavior of the steepest descent method. The quadratic minimum in (a) has exponential convergence behavior, whereas a random function exhibits such behavior only after $n > 140$. Notice the error in the step number $n$ actually rises between $60 < n < 140$. The "gras" in the $n > 400$ regime is due to machine error. It may be reduced by using quadruple precision.

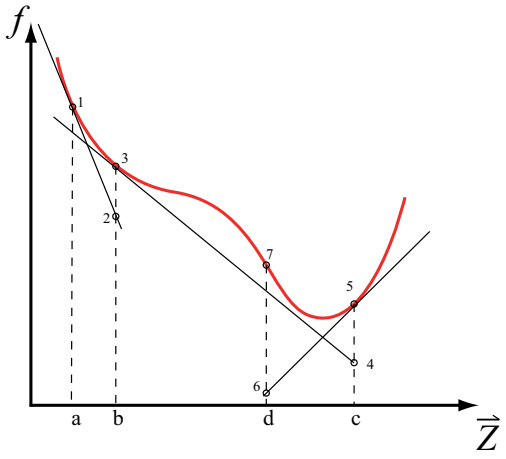



Figure B.4: A possible geometry of a function that may cause an increase in the error in step number. Given the geometry of $f$, the algorithm takes the route from 1 to 7. It is clearly shown that $\overline{f_a f_b} < \overline{f_b f_c}$, which are the measures of the error value. Hence, the value of an error in step number would have increased before an exponential behavior sets in.

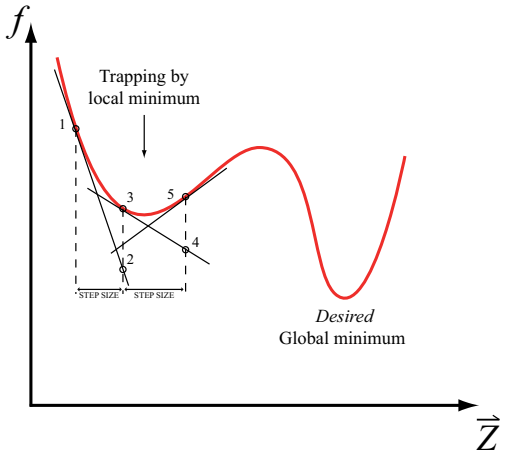



Figure B.5: Trapping by local minima in the steepest descent method. The method fails due to the small step size. The size is not large enough to jump over the rim of the local well. Step sizes shown in the figure are not to scale, since the step is taken in the vector space, not on the $\vec{z} - f$ graph (step sizes that appear in the above graph may not be the same at all).

The geometry of the function $f$ can also cause trapping. Figure B.5 shows how such trapping may occur at local minima of a function. This trapping mechanism can be shown analytically. Generally, an arbitrary vector $\vec{z}$ living in the space of $H$ can be expanded in the eigenvectors of $H$: $\vec{z} = \frac{\sum_{j=1}^{N} A_j \vec{x_j}}{\sqrt{\sum_{k=1}^{N} A_k^2}}$. Then, since the minima of $f$ are the targets of interest, a partial derivative of $f$ with respect to $A_m$ should be set to zero. With the use of orthogonality between the eigenvectors and the quotient rule from calculus, the partial derivative of $f$ with respect to $A_m$ reads,

$$\frac{\partial f}{\partial A_m} = \frac{\partial}{\partial A_m}\left(\vec{z}^T H \vec{z}\right) = \frac{\partial}{\partial A_m}\left(\sum_{j=1}^{N}\sum_{j'=1}^{N} \frac{A_j \vec{x}_j^T}{\sqrt{\sum_{k=1}^{N} A_k^2}} H \frac{A_{j'} \vec{x}_{j'}^T}{\sqrt{\sum_{k=1}^{N} A_k^2}}\right)$$
$$= \frac{\partial}{\partial A_m}\left(\sum_{j=1}^{N} \frac{A_j^2 \lambda j}{\sum_{k=1}^{N} A_k^2}\right) = \frac{S * 2A_j\lambda_j\delta_{j,m} - A_j^2\lambda_j * 2A_m}{S^2} = 0,$$

where $S \equiv \sum_{k=1}^{N} A_k^2$. This means either $A_m = 0$ or $A_m = 1$ and the rest of the $A$'s are zero. The former case is obvious as the derivative of zero is zero, which means other $A$'s should be investigated for the descent. The latter case clearly shows that the vector

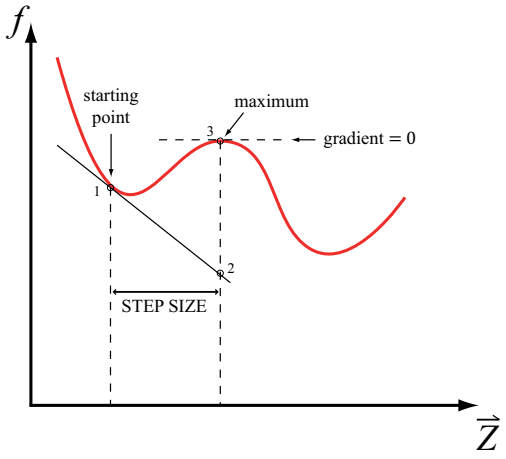



Figure B.6: Trapping by maxima in the steepest descent method. If the starting point (1) and the step size (2) happen to cause the newly computed point (3) to be placed at the exact maximum point, then the gradient of the function becomes zero, which leads to trapping at the maximum.

$\vec{z}$ is $\vec{x}_m$, which leads to the conclusion that the local minima (or maxima) are actually eigenvalues of $H$, and they cause trapping in the steepest descent method.

Due to the steepest descent method, minima behave as attractors and maxima behave as repellers. However, if the starting vector and the step size happen to lead one of the descending points to land on the exact point of a maximum, the gradient becomes zero and trapping occurs. A sketch of this special case can be found in figure B.6.

Despite a few difficulties demonstrated above, the new method is extremely powerful for the following reasons. In physics, it is very often the case that the ground state is of interest. In addition, usually, in order to obtain only the first few eigenvalues with reasonable accuracy, a tremendously large matrix needs to be diagonalized, where all the other eigenvalues are usually inaccurate and useless anyway (see figure B.7). While the conventional method, Householder's diagonalization method, requires solving for every single eigenvalue of a matrix that scales as $N^3$, the new method allows access to the lowest eigenvalue without computing the others, which leads to an $N^2$-scaling

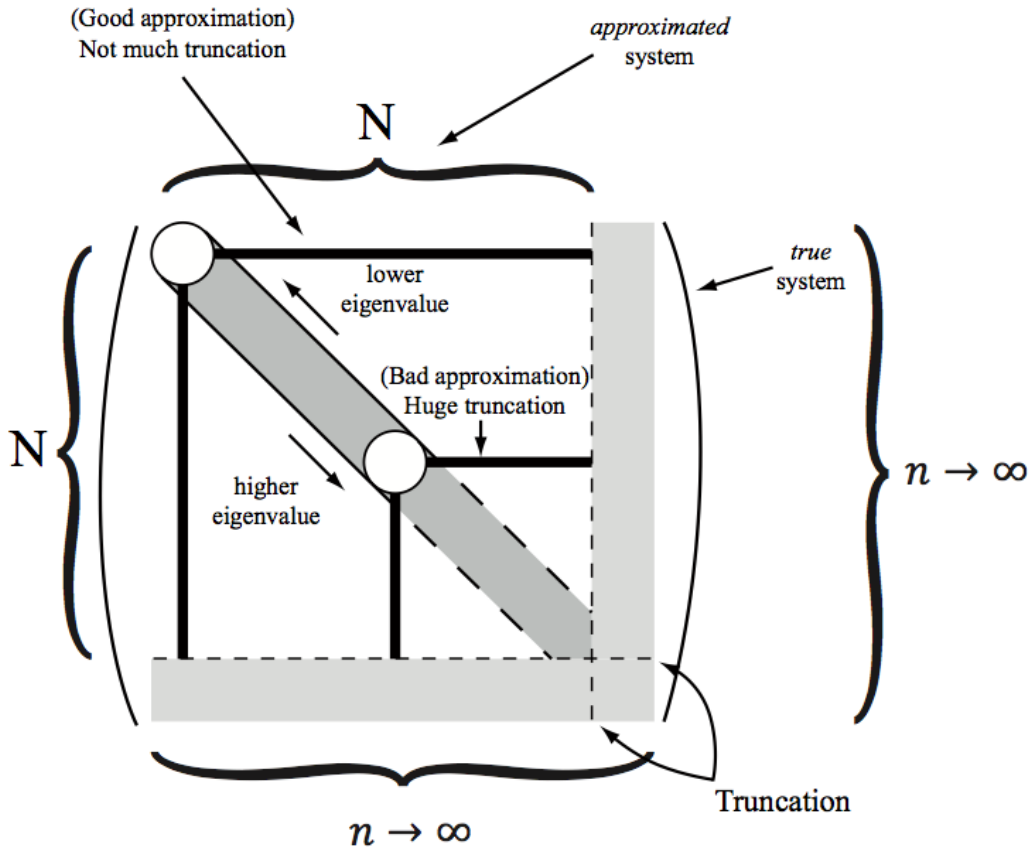



Figure B.7: $N \times N$ matrix representing an approximated system. Due to the truncation of the matrix, only the first few eigenvalues that are located in the upper left corner of the diagonals have enough accuracy. The other ones lack too many off-diagonal influential terms to have any accuracy.

scheme.

Regarding the second and the third lowest eigenvalues, the new algorithm can be easily modified. Once the eigenvector for the lowest eigenvalue, $\vec{x_1}$, is determined, the second lowest eigenvalue, $\lambda_2$ can be computed by using an arbitrary vector from which the $\vec{x_1}$ component is subtracted. This is to say,

$$\vec{y} = \frac{\vec{z} - (\vec{z} \cdot \vec{x_1})\, \vec{x_1}}{|\vec{z} - (\vec{z} \cdot \vec{x_1})\, \vec{x_1}|}.$$

Constructing $f(\vec{y}) = \vec{y}^T H \vec{y}$, where $f$ now has the minimum value of $\lambda_2$, the second lowest eigenvalue can be obtained by the same algorithm. Analogously, the third and the fourth lowest eigenvalues may be computed.

It is true that the new method has difficulties such as the number of steps needed, the convergence behavior, and the chance of trapping. However, this is usually not a problem for the following reasons. First of all, a rough approximation of the eigenvector that corresponds to the lowest eigenvalue is usually known. For instance, perturbation theory produces such an approximation. Hence, the method can be used in the vicinity of the ground state of the system, which is favorable for the new method as the exponential convergence will set in at once; therefore the number of iterative steps necessary significantly decreases. In addition, most Hamiltonians of a physical system have the form $H_0 + V$, where $V$ may be large. Usually, $H_0$ is analytically known, which means the exact forms of eigenvectors and the precise values of the corresponding eigenvalues are known. By introducing a switch parameter, $\sigma$, that can slowly turn on and off $V$, iterative procedures can be developed to fully take $V$ into account. Using this adiabatic switching technique, the following sequential procedure is proposed to obtain the lowest eigenvalue with $H = H_0 + V$.

1. Start with a known $H_0$ and the associated $\vec{x}_{1,0}$ and $\lambda_{1,0}$.
2. Set $\sigma$ to be an incremental value (adiabatically switching on).
3. Choose an arbitrary vector $\vec{z}$ near $\vec{x}_{1,0}$.
4. Use the $\vec{z}$ to define $f$.
5. Run the new diagonlization algorithm to obtain $\lambda_{1,1}$ and $\vec{x}_{1,1}$.
6. Raise the value of $\sigma$ by another incremental value and replace $\vec{x}_{1,0}$ in step 3 with the newly found $\vec{x_{1,1}}$ to compute $\lambda_{1,2}$ and $\vec{x}_{1,2}$.
7. Iterate these steps until $\sigma$ becomes 1 corresponding to a fully switched on $V$.

These steps are natural for the new method as the arbitrary vector $\vec{z}$ will be guaranteed to be near the ground state of the system, which ensures the exponential convergence, thereby making the new method efficient.

It is advantageous to use adiabatic switching since the behavior of the ground state energy, for example, is of great interest to physicists when the system is subjected to an external potential. Hence, the Jacobi rotations are useful not only for quantum computing but also for efficiently determining the ground state of quantum systems in general. It should also be noted that the algorithm may be easier to implement on parallel computers. With the numerous advantages demonstrated above, further research is expected to improve upon the implementation of the algorithm.

# Appendix C

# Staged Teleportation Scheme and Quantum Network

In this appendix, a quantum protocol is proposed to demonstrate the possibility of the realization of quantum networks and quantum servers. First, the requirements for quantum communication, which are very different from those of classical communication, are identified. Then, we investigate the means of transporting the quantum information in a way that does not violate any of these requirements, namely via teleportation, using EPR pairs. With this scheme, we show that the construction of efficient quantum networks is feasible. In addition to the hardware description of this network and the way the information travels, major advantages in using the protocol will be discussed.

This appendix emphasizes the efficient networking ability of the protocol. Sophisticated networks are demanded whenever more than two networking devices need to be connected. The one-to-one connection is primitive and therefore inefficient, since it needs as many direct connections between the senders and the receivers of the information as the square of the number of networking devices. By means of introducing centralized quantum servers in between, it is possible to transport quantum information in a system-

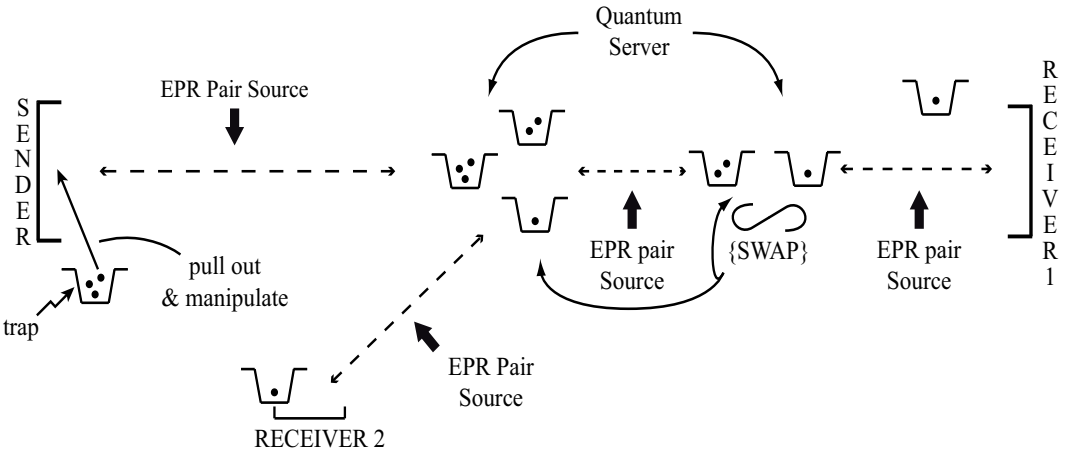



Figure C.1: A schematic diagram of a realizable quantum network. A sender (on the left), broadcasting stations, and receivers (bottom and right) have storage devices to hold many EPR pairs. Such EPR pairs are produced by the EPR sources shown above. With the help of SWAP gates, information can travel via "spooky action at a distance."

atic way with a smaller and more convenient personalized quantum networking device similar to the current, classical networking schemes. Such efficient quantum networks are sketched in figure C.1.

In figure C.1, each sender, broadcaster, and receiver have storage devices for EPR pairs to be used in teleportation as a quantum communication channel. The teleportation scheme with EPR pairs is of particular interest in that it preserves three important properties that must be fulfilled for quantum communication:

- Entanglement
- Superposition
- No-Cloning theorem

These are the three requirements for quantum communication that differ fundamentally from the classical counterpart due to the quantum mechanical nature of quantum information. The use of teleportation through EPR pairs indeed satisfies the three points

above, since $(i)$ the pairs are entangled, $(ii)$ no intermediate measurements are performed to collapse superposition states, and $(iii)$ no amplification is needed.

The hardware in the networks is installed as follows. EPR-pair sources are placed in such a way that the intermediate broadcasting stations can receive the pairs and store them in the storage devices. In a similar way, the network users, who may be senders or receivers of the information, are connected to their local servers. Then, the central server has EPR connections to numerous local servers via the broadcasting stations, leading to a universal connection for all the users through the network. These servers and users are also connected through classical networks, which is critical for the information transport process. The EPR devices have the ability to check the arrival of the pairs from the source using mirrors, for example, and to distinguish each pair from one another. Such an ability does not compromise the information transfer, since these sorts of confirmation processes do not make use of the information carrying part of EPR pairs.

Once the networking hardware is installed, the information transfer occurs in the following order. The sender of quantum information produces the output result through a quantum computer. Then, this information is placed on one of the already-stored EPR pairs that is in the sender's storage device. Now, the sender contacts the local server to notify which EPR pair carries the quantum information. At the local server, a SWAP gate is operated to interchange the information between the information carrying pair and the pair connected to the broadcasting stations that are connected to the central server. In the same manner, the information is transported to the next one in the line of networks. The selection of the route is made by the sender who specifies the receiver, where this route plan is classically delivered to the intermediate servers.

The advantage of using the proposed networks with servers is the following. From a practical point of view, producing many small EPR sources is easier than manufacturing a few but gigantic ones to connect long-distance, which makes the short-distanced server-oriented networking scheme much more favorable than the direct user-user connections.

Also, the construction of the network in avoiding physical obstacles in the midst of the path of EPR pairs becomes more flexible with many servers. In this way, all the users are connected to one another with a number of connections that scales as the number of users. In addition, this enables a smaller personal device, since one does not need to trap as many EPR pairs as the number of users, just to be ready to potentially communicate with other users. Furthermore, the use of portable devices may be realizable without the need to carry the EPR pairs at all times, as the one with the portable device only needs to contact the nearest local server to receive EPR pairs whenever any information needs to be transferred.

It should be noted that none of the three bullet points above is violated during the information transport. Since eavesdropping of the information is not possible due to the checking protocol on the arrival of EPR pairs, a secured networking regime of quantum information is established as long as the route of the information transport is not altered. Also, due to the nature of an EPR pair with respect to teleportation, i.e. “spooky action at a distance”, the information of interest is instantaneously teleported between the storage devices. With the introduced infrastructure and localized EPR pairs that are ready to be used, an instantaneous transport of quantum information in a secure manner with fidelity near 100 percent is achieved.

# Bibliography


[1] R. P. Feynman, *Simulating Physics with Computers*, Int. Journal of Theor. Phys. **21**(6/7), 467-488 (1982).

[2] D. Deutsch, *Quantum Theory, the Church-Turing Principle and the Universal Quantum Computer*, Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences **400**(1818), 97-117 (1985).

[3] M.A. Nielsen, I.L. Chuang. *Quantum Computation and Quantum Information*, (Cambridge University Press, New York, 2000).

[4] R. Blümel *Foundations of Quantum Mechanics: from photons to quantum computers*, (Jones and Bartlett Publishers, Sudbury, 2010).

[5] P.W. Shor, *Algorithms for Quantum Computation: Discrete Log and Factoring*, Proceedings of the 35th Annual Symposium on the Foundations of Computer Science (IEEE Computer Society Press, New York, 1994), p.124.

[6] N.D. Mermin *Quantum Computer Science: An Introduction*, (Cambridge University Press, New York, 2007).

[7] C.H. Bennett, G. Brassard, *Quantum cryptography: Public key distribution and coin tossing*, Proceedings of IEEE International Conference on Computers Systems and Signal Processing **175**, 175-179 (1984).

[8] A.K. Ekert, *Quantum cryptography based on Bell's theorem*, Phys. Rev. Lett. **67**(6), 661-663 (1991).

[9] W.-Y. Hwang, *Quantum Key Distribution with High Loss: Toward Global Secure Communication*, Phys. Rev. Lett. **91**(5), 057901 (2003).

[10] R. Alleaume, *Quantum Key Distribution and Cryptography*, arXiv:quant-ph/0701168v1 (2007).

[11] F. Xu, B. Qi, H.-K. Lo, *Experimental Demonstration of phase-remapping attack in a practical quantum key distribution system*, New J. Phys. **12**, 113026 (2010).

[12] C. Wiechers, L. Lydersen, C. Wittmann, D. Elser, J. Skaar, Ch. Marquardt, V. Makarov, G. Leuchs, *After-gate attack on a quantum cryptosystem*, New J. Phys. **13** 013043 (2011).

[13] R. Landauer, *Irreversibility and Heat Generation in the Computing Process*, IBM J. Res. Develop. **5**(3), 183 (1961).

[14] C. H. Bennett, *Logical Reversibility of Computation*, IBM J. Res. Develop. **17**, 525 (1973).

[15] T. Toffoli, *Reversible Computing*, in Automata, Languages and Programming, (J. W. de Bakker and J. van Leeuwen ed.), (Springer, New York, 1980), p.632-644.

[16] P. Benioff, *The Computer as a Physical System: A Microscopic Quantum Mechanical Hamiltonian Model of Computers as represented by Turing Machines*, J. Stat. Phys. **22**(5), 563-591 (1980).

[17] D. Deutsch, *Quantum Computational Networks*, Proc. R. Soc. Lond. A. **425**(1868), 73-90 (1989).

[18] S. Lloyd, *A Potentially Realizable Quantum Computer*, Science **261**(5128), 1569-1571 (1993).

[19] B. Schumacher, *Quantum coding*, Phys. Rev. A **51**(4), 2738-2747 (1995).

[20] A. Barenco, C.H. Bennett, R. Cleve, D.P. DiVincenzo, N. Margolus, P.Shor, T. Sleator, J. Smolin, H. Weinfurter, *Elementary Gates for Quantum Computation*, Phys. Rev. A **52**, 3457-3467 (1995).

[21] J. I. Cirac and P. Zoller, *Quantum Computations with Cold Trapped Ions*, Phys. Rev. Lett. **74**, 4091-4094 (1995).

[22] A. Steane, *The ion trap quantum information processor*, Appl. Phys. B **64**(6), 623-643 (1997).

[23] A. Steane, *Quantum Computing*, Rep. Prog. Phys. **61**(2), 117-173 (1998).

[24] C. Monroe, D. M. Meekhof, B. E. King, W. M. Itano, D. J. Wineland, *Demonstration of a Fundamental Quantum Logic Gate*, Phys. Rev. Lett. **75**(25), 4714-4717 (1995).

[25] Q. A. Turchette, C. J. Hood, W. Lange, H. Mabuchi, H. J. Kimble, *Measurement of Conditional Phase Shifts for Quantum Logic*, Phys. Rev. Lett. **75**(25), 4710-4713 (1995).

[26] L.-M. Duan, H.J. Kimble, *Scalable Photonic Quantum Computation through Cavity-Assisted Interactions*, Phys. Rev. Lett. **92**(2), 127902 (2004).

[27] T.D. Ladd, F. Jelezko, R. Laflamme, Y. Nakamura, C. Monroe, J.L. O'Brien, *Quantum computers*, Nature **464**(4), 45-53 (2010).

[28] P. W. Shor, *Scheme for Reducing Decoherence in Quantum Computer Memory*, Phys. Rev. A **52**(4), R2493-2496 (1995).

[29] L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, Proceedings, STOC, 1996, p. 212-219.

[30] M.W. Johnson, M.H.S. Amin, S. Gildert, T.Lanting, F. Hamze, N.Dickson, R. Harris, A.J. Berkley, J. Johansson, P. Bunyk, E.M. Chapple, C. Enderud, J.P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M.C.

Thom, E. Tolkacheva, C.J.S. Truncik, S. Uchaikin, J. Wang, B. Wilson, G. Rose, *Quantum annealing with manufactured spins*, Nature **473**(7346), 194-198 (2011).

[31] J.T. Barreiro, M. Müller, P. Schindler, D. Nigg, T. Monz, M. Chwalla, M. Hennrich, C.F. Roos, P. Zoller, R. Blatt, *An oepn-system quantum simulator with trapped ions*, Nature **470**(24), 486-491 (2011).

[32] M. Steffen, *Towards a scalable superconducting qubit architecture*, APS March Meeting 2012 abstract J3.00003.

[33] J.P. Pollard, *A Monte Carlo Method for Factorization*, BIT Num. Math., **14**(3), 331-334 (1975).

[34] C.P. Schnorr, H.W. Lenstra Jr., *A Monte Carlo Factoring Algorithm With Linear Storage*, Math. Comp., **43**(167), 289-311 (1984).

[35] H.W. Lenstra Jr., *Factoring integers with elliptic curves*, Ann. Math., **127**, 649-673 (1987).

[36] A.K. Lenstra, J.H.W. Lenstra, *The devlopment of the Number Field Sieve*, Lecture Notes in Mathematics, **1554**, 95-102, (Springer-Verlag, Berlin, 1993).

[37] A.K. Lenstra, J.H.W. Lenstra, *Prime Numbers: A Computational Perspective.*, Lecture Notes in Mathematics, 227-244, (Springer, Berlin, 2001).

[38] B. Verkhovsky, *Generalized Baby-Step Giant-Step Algorithm for Discrete Logarithm Problem*, Advances in Decision Technology and Intelligent Information Systems, IIAS, **6**, 88-89, (2008).

[39] C. Ottaviani, D. Vitali, M. Artoni, F. Cataliotti, P. Tombesi, *Polarization Qubit State Gate in Driven Atomic Media*, Phys. Rev. Lett. **90**(19), 197902 (2003).

[40] V. Vedral, A. Barenco, A. Ekert, *Quantum networks for elementary arithmetic operations*, Phys. Rev. A **54**(1), 147-153 (1996).

[41] C. Monroe, D.M. Meekhof, B.E. King, W.M. Itano, D.J. Wineland, *Demonstration of a Fundamental Quantum Logic Gate*, Phys. Rev. Lett. **75**(25), 4714-4717 (1995).

[42] F. Schmidt-Kaler, H. Häffner, M. Riebe, S. Gulde, G.P.T. Lancaster, T. Deuschle, C. Becher, C.F. Roos, J. Eschner, R. Blatt, *Realization of the Cirac-Zoller controlled-NOT quantum gate*, Nature **422**(6930), 408-411 (2003).

[43] L.M.K. Vandersypen, M. Steffen, G. Breyta, C.S. Yannoni, M.H. Sherwood, I.L. Chuang, *Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic resonance*, Nature **414**, 883-887 (2001).